INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

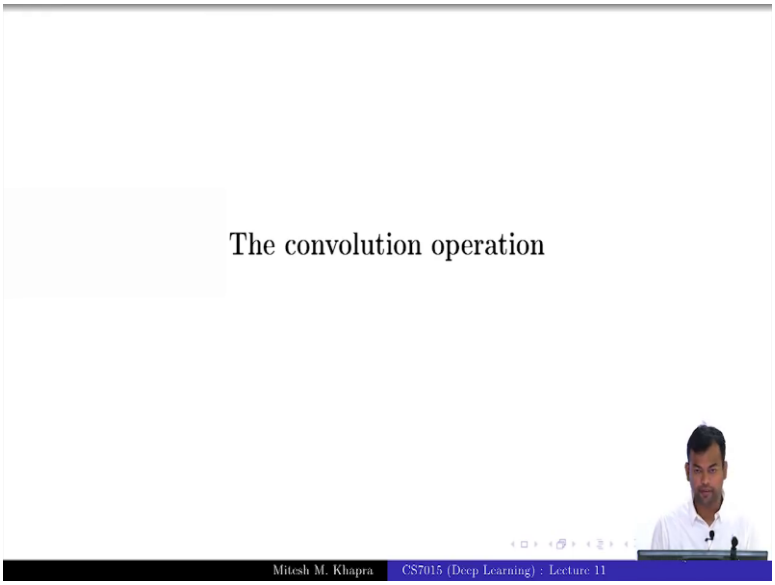
Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 11
The convolution operation

So, why let us start. So, far in the course we have looked at feed forward neural networks, we have seen how to train them and we have seen 2 special cases of feed forward neural networks, one was the auto encoders for learning, representations or learning latent representations of inputs and the other thing that we had seen was how to use a feed forward neural network to learn word representations where we saw this word to wake algorithm and it is different variants, it was continuous bag of words, skip gram model graph and so on. So, those are all since some since applications of the feed forward neural network.

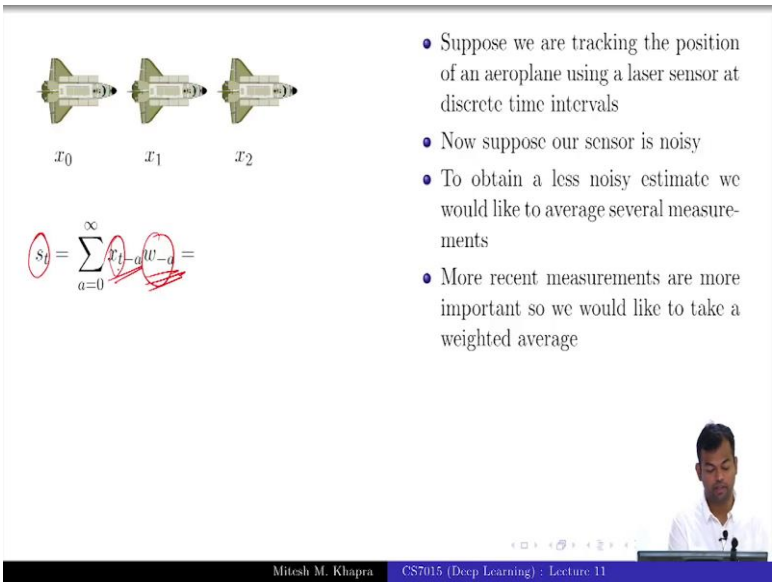
And now we will move on from there, though we will look at different type of neural network today, which is convolutional neural networks and we look at some specific architectures, which have become popular over the past few years. Ok.

(Refer Slide Time: 01:07)



So with that, I will start this lecture on convolutional neural networks. So, in the first module we will look at the convolution operation. Ok.

(Refer Slide Time: 01:11)



So, let us see. So, suppose we are tracking the position of an aeroplane using a laser sensor at discrete time intervals, right. So, you have this ok. So, you have this aeroplane, suppose it is going from say Chennai to Delhi and at discrete time intervals, you are seeing the tracking the position of the aeroplane right, how far it is from Chennai at this point right may be it is 50 kilometers, 100 kilometers and so on.

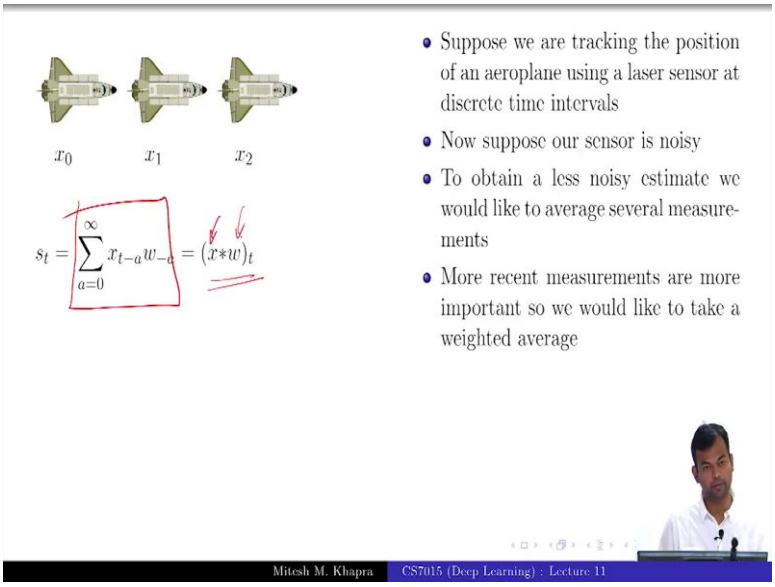
And now your laser you think that it might be noisy, it might not be giving you very accurate measurements. So, you would be taking these measurements and say intervals of course, it is not in practice you would not do that, but just indulge me for the purpose of illustration that say, you are taking these measurements every 5 seconds or 10 seconds or something like that.

Now, since your sensor is noisy instead of relying on a single measurement, you would probably want to take the average of the past few measurements that you have taken. So, that would give you a more accurate representation of what the current position is. Does that make sense like, you are taking multiple measurements and taking averages of those right and of course, more recent measurements are more important as compared to the previous measurements right. So, this is suppose at time step t, say this was t minus 5 seconds and this was t minus 5 minutes suppose.

So obviously, you would not want to take give a very large weightage to the measurement that you are taken t - 5 minutes back right because, the plane would have

moved by a lot by that time. So, it rely more on the recent measurements and less on the previous measurements right. So now, the mathematical way of lighting, this is that you know the positions or the readings that you have taken at time steps 1, 2, 3, up to time step t, you are interested in the revise estimation of this measurement right. So, you have taken some measurement at time step t and you want a revised measurement of that and the way you are going to compute, that is you are going to take a weighted average. So, w is the weight of all the previous measurements that you have taken right.

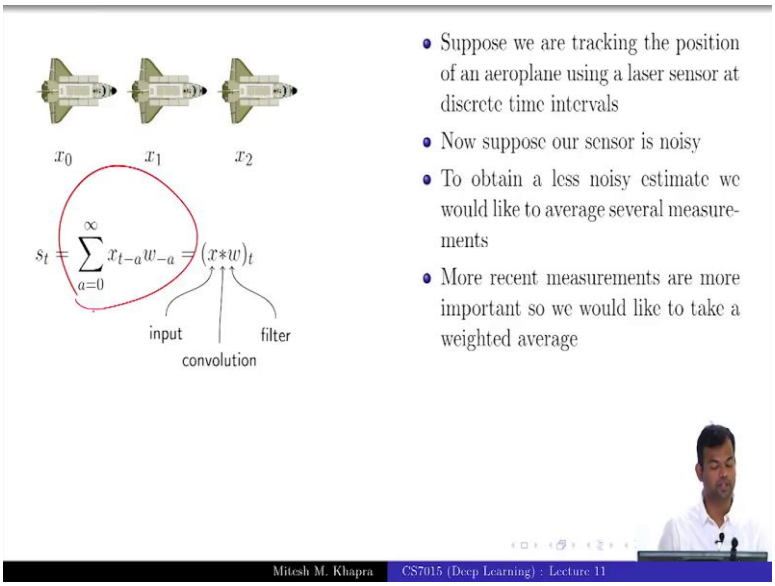
(Refer Slide Time: 03:17)



So, the measurement that you take a t - 1, t - 2, t - 3 all the way up to t minus infinity and for each of them would have a weight associated with this.

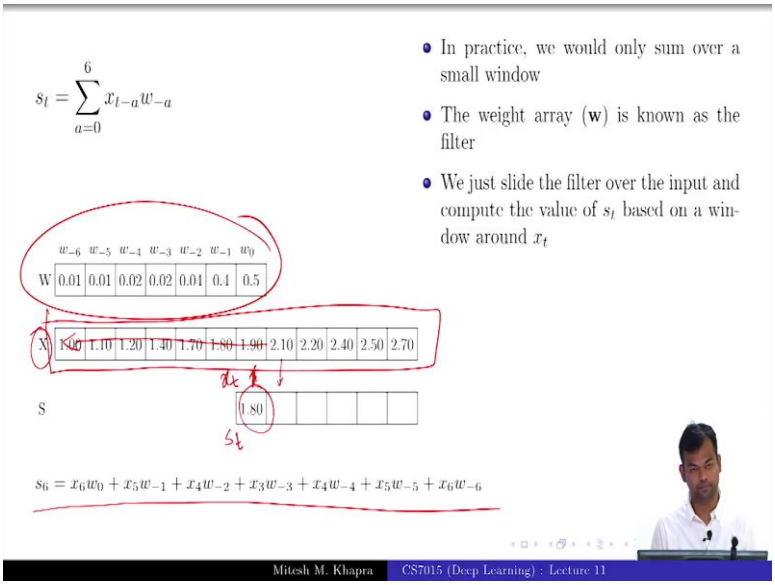
So, this operation right this thing, you can write it as the following operation that you have a vector of measurements or an array of measurements, which is x and you have an array of weights associated with these measurements, the farther the measurement from the current time step hopefully smaller is the weight assigned to that and this operation is known as the convolution operation right.

(Refer Slide Time: 03:37)



So, you have X which is the input, W is known as the filter and the operation that is defined as this equation is known as a convolution operation right.

(Refer Slide Time: 03:49)



Look, but of course, in practice you would not do this from infinity right, you would probably keep a window, you will say I will rely on the previous 6 measurements; that means, whatever I took at t - 1 second, t - 2 second up to t - 6 seconds right, beyond that it does not really make sense. So let us see, how this computation happens. So this is

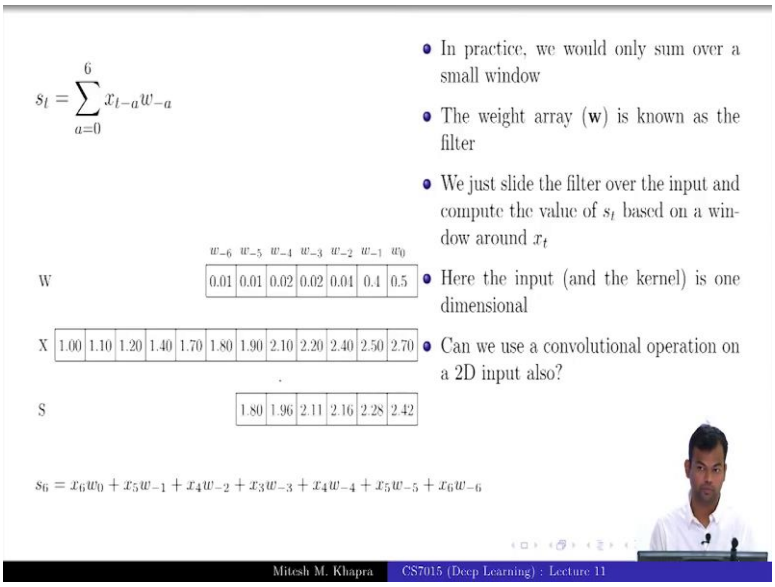
weight array so now, what would be the dimension of this weight array? How many entries would it have?

Student: 7.

7 right 0 to 6. So, 7 entries ok. And this is what my situation looks like right. So, this is the X the measurements, which I have taken using the laser ok. So, I have taken some measurements. Now, I am at a particular time step and I want to make a revised estimate. So, I have this x_t and from that I want to compute s_t and the way, I am going to do that is by taking a weighted average of all these previous measurements. Is the setup clear to everyone, ok?

And now this is what my formula is going to be. So, the revised estimate of s_6 is going to be whatever was x_6 into w_0; that means, the weight assigned to the current time step; x_5 into w minus 1; that means, weight assigned to the time step - 1; x_4 into - 2 and so on. So, you get this ok. So, I have these 7 weights and I will multiply with them with the 7 previous readings, 1 is to 1 multiplication and I will get the weighted average and using that I get a revised estimate.

(Refer Slide Time: 05:29)

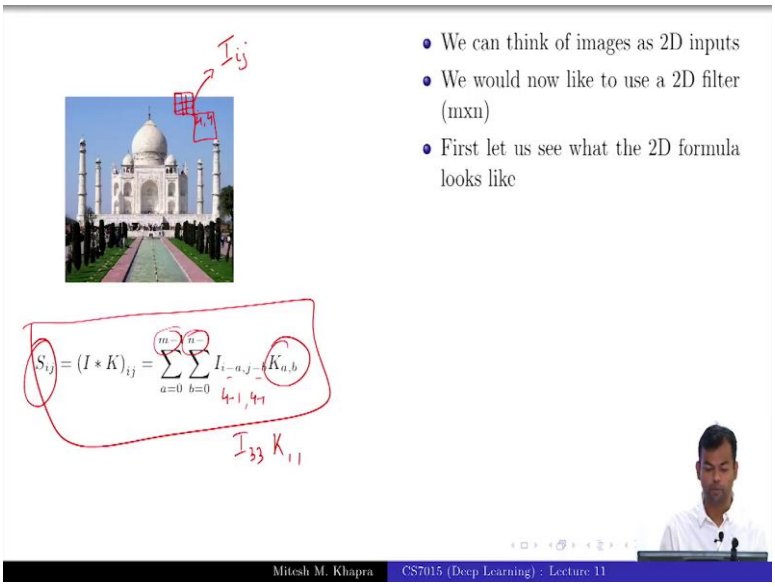


Now, I want to get a revised estimate for the next entry, how will I get it? I will just slide this weight matrix right.

So, I will just slide it by 1, I will again do the same computation and get the revised measurements. Again for the next entry, I will slide it by 1, slide it by 1, slide it by 1 and I will keep getting these entries ok. So, everyone gets the setup, how do you do the convolution operation? It is basically, a weighted average of the previous entries fine.

So, here the input as well as the kernel is kind of 1 dimensional right you. So, you have it is. So, you do not have a 2D input here, you just have a single dimensional input here.

(Refer Slide Time: 06:01)



Can you use a convolution operation on a 2D input also? Do you know of any 2D inputs? Images, right? So, we can think of images as 2D inputs. Now again I am trying to do the same thing, the setup is the same, the story just changes from laser to a camera now. So, I have taken an image maybe the image was captured and I am not very confident about all the pixels that I have captured ok.

So, now for any given pixel, I want to re-estimate it, using it is neighborhood that is what I want to do ok. So, this is the pixel, I am going to look at some neighborhood around it right. So, every cell here is 1 pixel, just assume that every cell here is 1 pixel. So now, I am going to re-estimate this pixel by taking a weighted average of all its neighborhoods right. So now, can you tell me, what is my filter going to look like in this particular case? My filter would be just 3×3, right. So, whatever neighbors I want to average on for every neighbor, I want a weight. So, if I am going to average on a neighborhood of 3×3

then for each of these, I will want a weight. So, my filter would also be of size 3×3, how many of you get this? Ok.

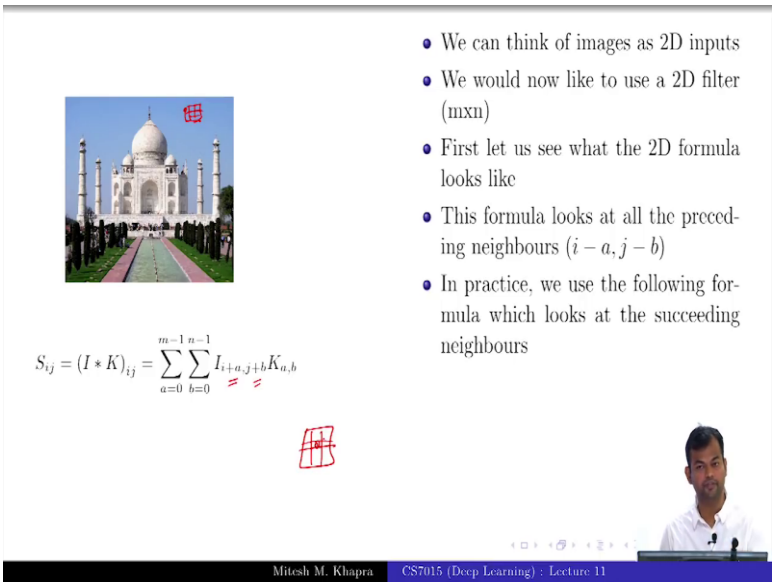
So, we now like to use a 2D filter, which would be m cross n, ok. And in general it would be $m \times m$. So, it would always be a square filter, but I am just taking the case. Now what this nasty looking formula is doing, right? So, I have a particular pixel. So, this is an image. So, I will refer to this pixel as I ij right. So, it is the ith ij^{th} entry in the image, I want a revised estimate for that I want an Sij for that.

So the way, I am going to do that is I am going to look at m rows and n columns before it right. So, I am going to look at this neighborhood of m cross n, ok. And for each of these, I would have a weight associated with it. So, if I am looking at say for example, this was 4 , 4, this pixel was 4, 4 then, I will look at 4 – 1, 4 - 1. So, that would be I 3, 3. So, I will look at that neighbor and with that neighbor, I would have some weight associated, do you get that how this formula expands?

So, this formula would have m cross n terms, for every term, you would have a have a weight and that weight, you can just represented as this filter matrix. So, you get this what this formula is doing? It looks a bit nasty, but it is just the weighted average of all the neighborhood that you have and the neighborhood is a 2 dimensional neighborhood, in this case. How many if you get this properly, ok?

Now this, in this formula actually, I am looking at minus a and minus b; that means, I am looking at previous neighbors right. Now you should have these questions right, why previous neighbors, why not future neighbors? So, why am I not looking at this neighborhood?

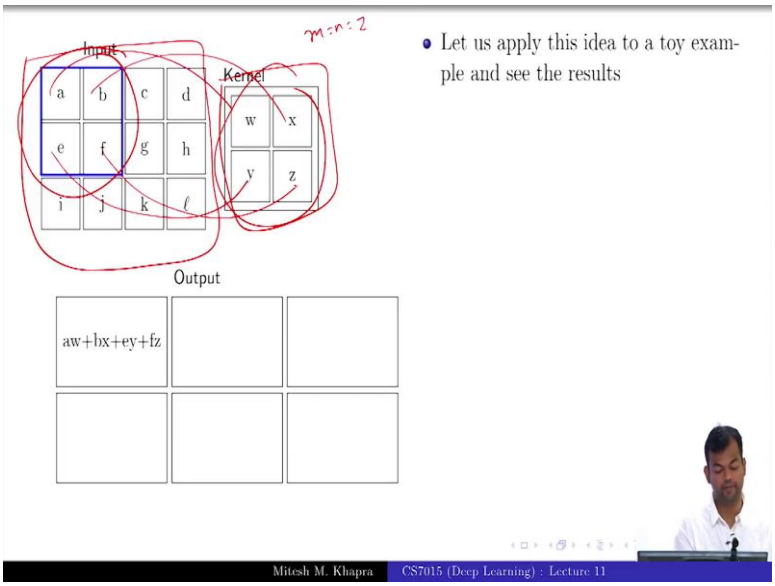
(Refer Slide Time: 08:51)



So there is no correct answer here, different convolution operations, I mean different packages use different convolution operations, but the most standard one, I believe is when you look at the next neighborhood right; that means, you at this pixel and you will look at this neighborhood, the neighborhood after it right not the before it ok.

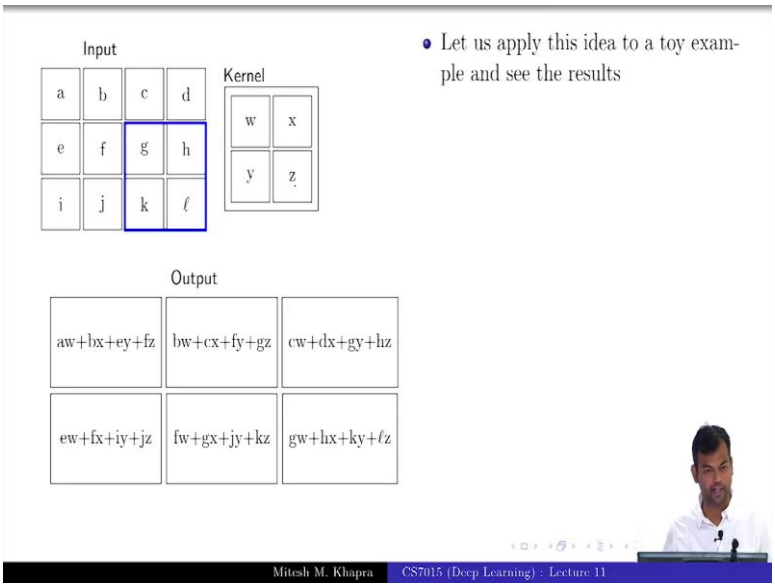
And in fact, so this is the formula that, I am going to look at plus j and plus p; that means, I am looking at pixels in the rows after this and in the columns after this pixel, all of you get this instead of before? Now what is even more natural to do? The names surrounding thing right. So, I will have this pixel and I will look at it is such a way that, this pixel is the center of the neighborhood right. So that is what I am going to go towards after a couple of slides and that is what, I will use for all my convolution operations, but in terms of textbook definitions these are the definitions that, you will find in textbooks ok.

(Refer Slide Time: 09:35)



So, let us let us apply this to a toy example. So, I have this input, which is 2 dimensional input, I have a kernel which is a 2×2 kernel. So, my m is equal to n is equal to 2. So, I am going to place this kernel at this location, ok. And then what will I get as the output?

(Refer Slide Time: 10:01)

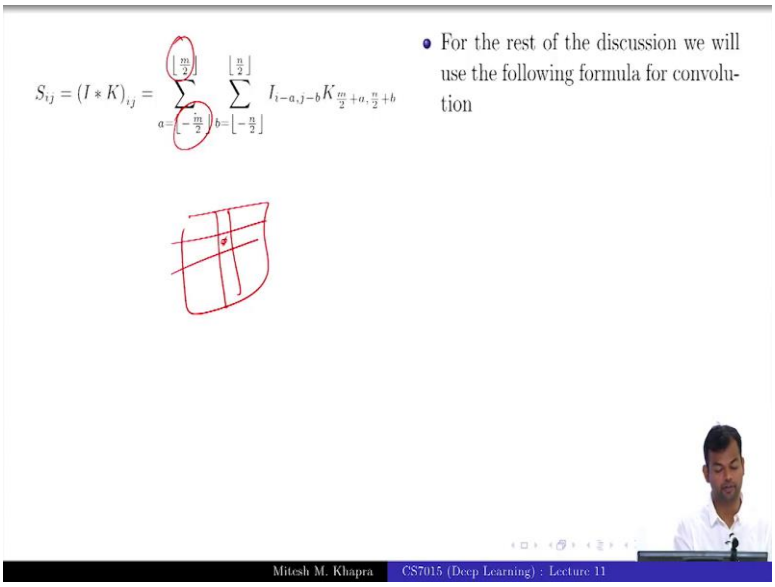


a into w plus b into x plus e into y plus f into z right and I will keep sliding this to get the other entries. Do you observe something about the input and the output?

Student: (Refer Time: 10:07).

Size, the output size has reduced, why? We will get back to this.

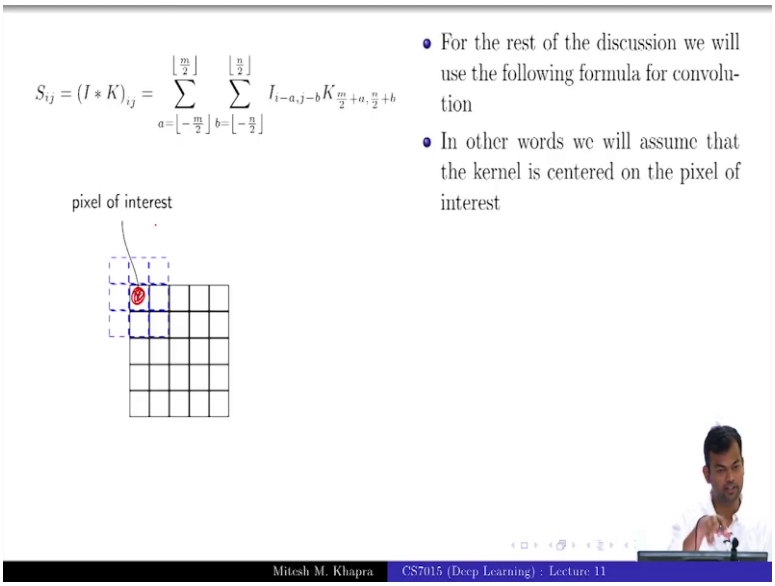
(Refer Slide Time: 10:19)



So right now, I just you to notice it is obvious nothing great about it, but I will just get back to it more formally later on.

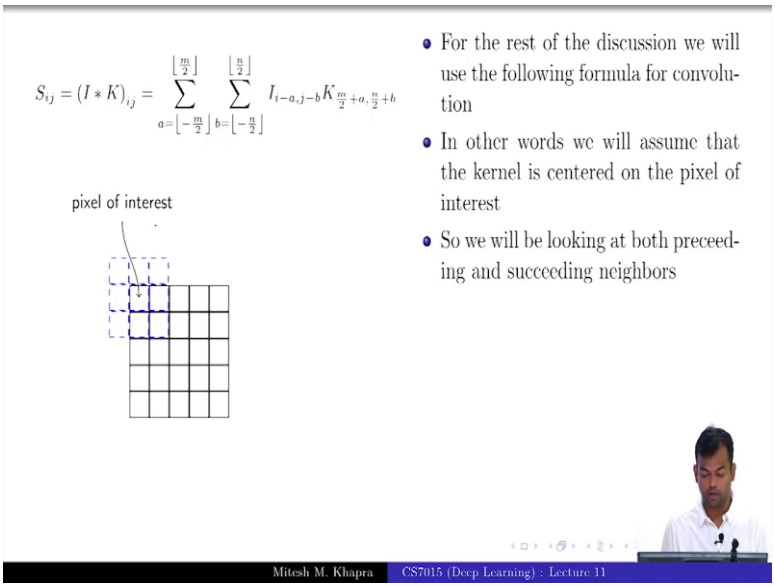
So, for the rest of the discussion, we will use the following formula for convolution, which is the centered formula right. So, $\frac{m}{2}$ to $\frac{n}{2}$; that means, I will be looking at a neighborhood, which is centered on the pixel of interest that is why this - $\frac{m}{2}$ to $+\frac{m}{2}$ Is that fine? Ok?

(Refer Slide Time: 10:35)



So, this is how I am going to look at it. So, this is how I will place, if this is the pixel of interest, which I want to re estimate, I will replace the kernel such that it, this pixel lies at the center of the kernel, ok.

(Refer Slide Time: 10:49)



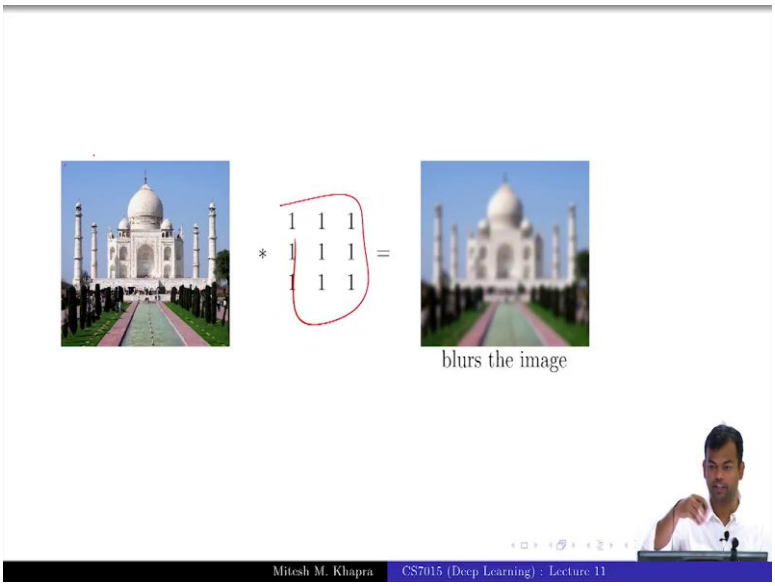
So, we will be looking at both preceding and succeeding neighbors, ok.

(Refer Slide Time: 10:51)



So, let us see some examples of 2D convolutions applied to images.

(Refer Slide Time: 10:57)



So this is an image, I decide to apply the following convolution operation to edge ok, tell me what the resulting image would be?

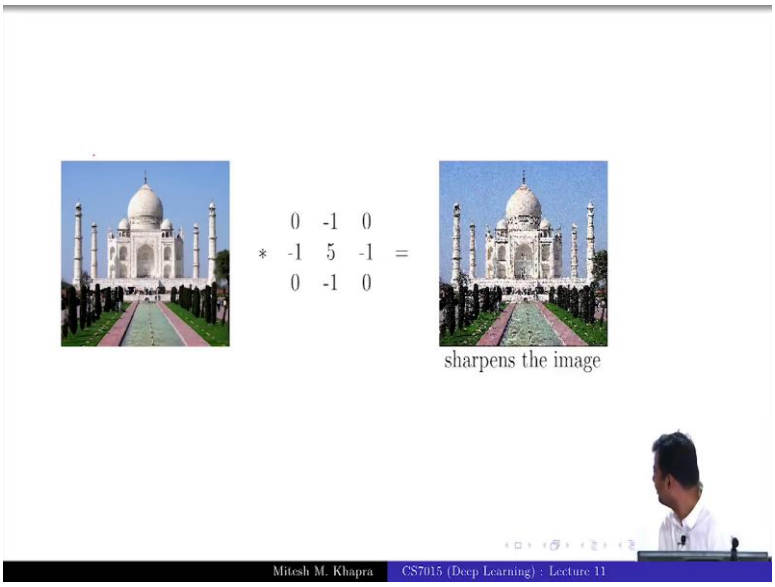
Student: Blurred.

Blurred, why blurred?

Student: We are taking average.

We are taking the average right. So, it would be blurred, you get the intuition. So this kernel basically, I have fitted at every pixel and I have computed the average around it and place at pixel by that average value and when we are going to take average things are going to get blurred right, because all the sharpness is gone ok.

(Refer Slide Time: 11:23)



Now let us look at this kernel, what will this do? Sharpen why? Because one was blurred the other has to be sharpened, what is happening here?

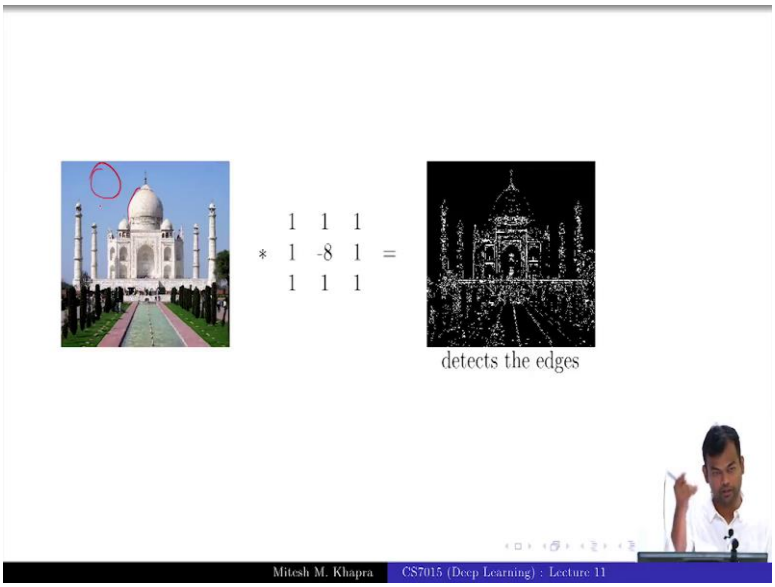
Student: (Refer Time: 11:29).

It is subtracting the neighbor's right. So you are taking 5 times the current pixel and subtracting the neighbors from it right. So, if the neighbors are similar, those would get subtracted and this would stand out really right, does that make sense?

This will result in a, but this in my on my laptop, this looks like a sharpened image, I do not know why it is looking like this here ok, it is a sharpened image just trust me, you can so actually are common right. So, people who have used adobe or any of these photo shopping software's right. So, you have this click button and where you say take an image sharpen and blur it. So, this is exactly what the tool is doing in the background, it is applying this convolution operation throughout the image.

So, when you say blur it is basically, placing that convolution operation throughout the image and computing the blurred image and same for sharpening and all these other spatial effects that, you have most of them come out of some convolution operation ok.

(Refer Slide Time: 12:23)



So for example, the next one, what would this do?

Student: (Refer Time: 12:26).

So, I will give you a hint, when will this result in a 0 output?

Student: (Refer Time: 12:32).

When all the neighbors are the same as this, right so then, when will it result in a nonzero output?

Student: (Refer Time: 12:37).

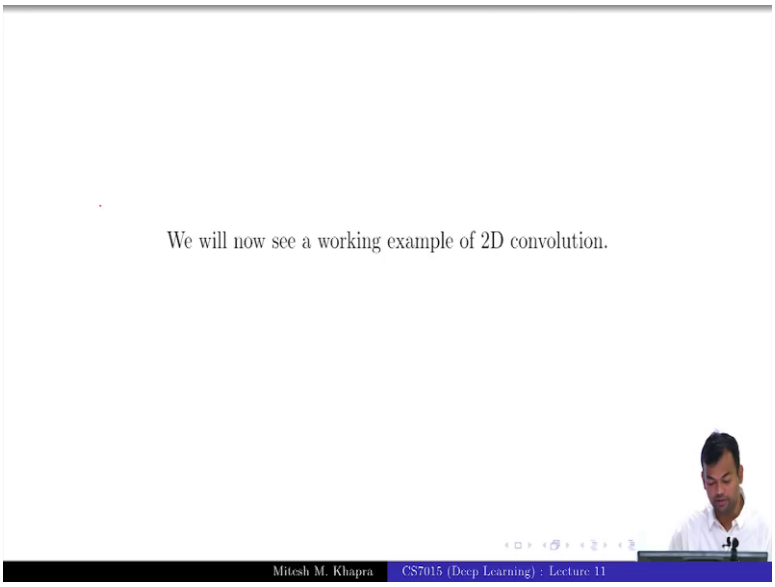
When there is a difference, when there is a difference. So, looking at this image tell me one place, where you know that it will result in nonzero output?

Student: (Refer Time: 12:45).

All the boundaries right. So, this is basically an edge detector in the slides, it appears properly ok. So, this is basically an edge detector and you get the intuition that these

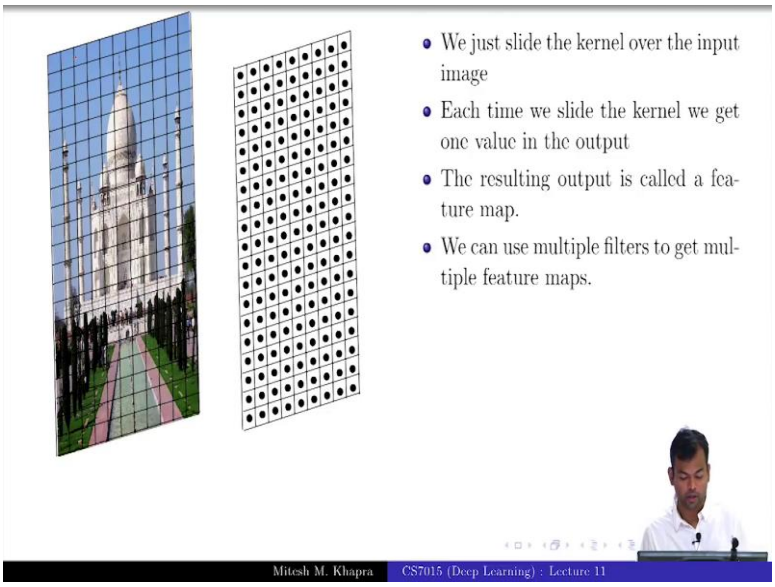
boundaries, whether neighbors are not the same as the current pixel, you will not get a 0 value. In this case, when all the neighbors are the same as the current pixel, so you are taking the sum of the 8 neighbors and subtracting the current value 8 times. So, that would be 0 right ok.

(Refer Slide Time: 13:11)



So enough of examples, so now, we will see a working example of a 2D convolution. So, I just want to drill this idea of what happens, when you do a 2D convolution.

(Refer Slide Time: 13:19)



So, what we are going to do is, we have this 3×3 kernel and assume that everything here is a pixel ok, everything here is a pixel. So, I am going to slide this 3 cross 3 kernel across this filter, now when I place the filter once on the image how many outputs do I get?

Student: 1.

1 output. So, if I keep sliding it across the image, I will keep getting 1 1 pixel in the output ok. So, what the resulting thing that I get is known as a feature map ok, because it is the original input that we have taken. For every pixel, you have tried to approximate it or whatever filter weights you have applied and it necessarily does not mean, that you are taking an average, it could be some weird average of your neighborhood right. So, you have extracted some features from there.

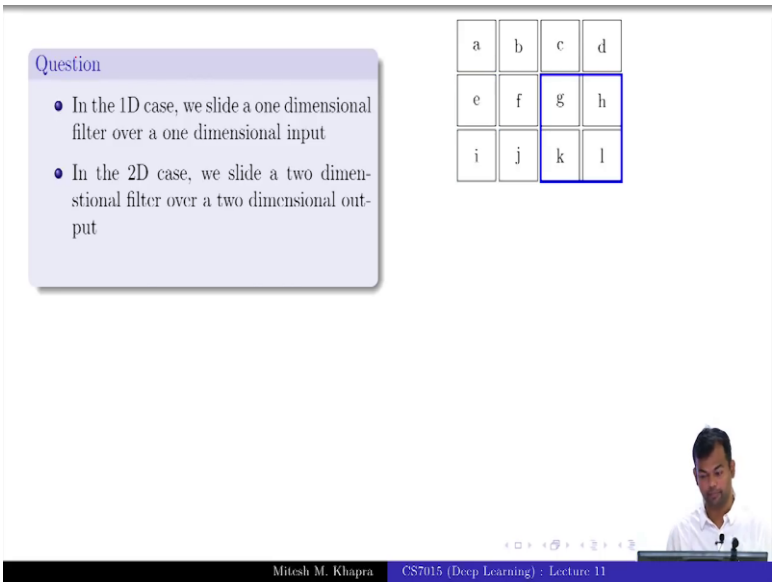
So for example, in the edge detector case, you could think of it that you have extracted the feature that this pixel does not lie at a boundary right, that is why you get the black pixel. Do you get that? You see this way of interpreting a convolution operation that, you are trying to extract some features from that neighborhood.

So, in this earlier example whenever you got a black you are basically extracting the feature that this pixel does not lay at a boundary. Is that ok, fine? So now, you could get one such feature map by using a single 3 cross 3 convolution operation ok. If I use multiple such convolution operations, what would happen? I will get multiple feature maps ok. So, let us try to understand this, what is the dimension of my original image, m cross n into 3. Why is it into 3?

Student: RGB channels.

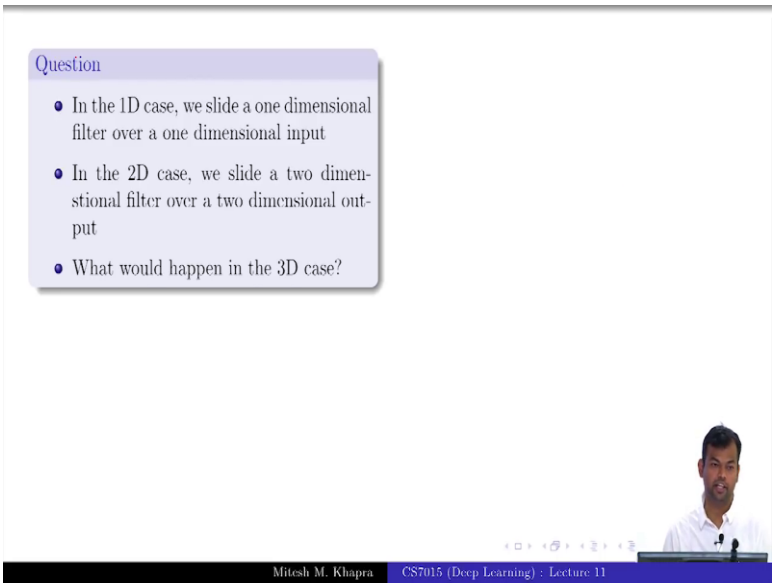
RGB channels ok. RGB is what we will have right. So, we will have this $3 \times m \times n$. So, we will return back to this idea and from now this one image by using a single kernel, so this, in fact in for this figure right, I am assuming that the input is 1 cross m cross and I am not assuming there are 3 channels although, it is a colored image, but just bear with me. So, it is a 1 cross m cross an image and when I apply a filter, I get a 1 feature map, if I apply k such filters, I will get k feature maps. So one feature map could be for the blurring one, one could be the sharpening one, one could be the edge detector and so on right. There are various such filters that you could apply.

(Refer Slide Time: 15:35)



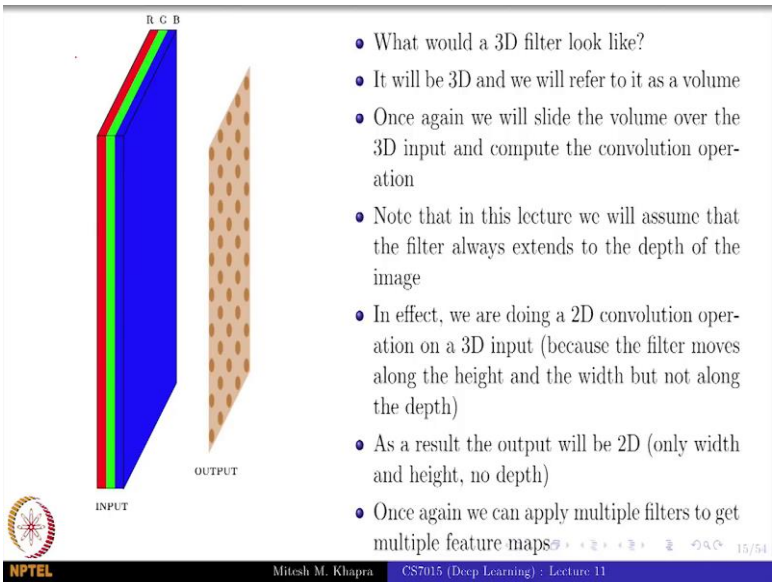
Now, in the 1D case, we slide a 1 dimensional filter over a 1 dimensional input, in the 2D case, we slide a 2 dimensional filter on a 2 dimensional input. What would happen in the 3D case?

(Refer Slide Time: 15:45)



So, now, we are going to this RGB images right. So, we will have 3 cross m cross n as the input, what would happen in the 3G, 3D case not 3G.

(Refer Slide Time: 15:55)



So, what would a 3D filter look like?

Student: Box.

Look like a cuboid, a box basically and we will call it a volume, why volume? Because it has a width, it has a height and it will have a depth. So, this is what a 3 D filter would look like. I will assume that it is depth is the same as the depth of your input. What is the depth of your input in this case?

Student: 3.

3. So, I will assume that the depth of the filter is the same as a depth of the input and the width and height could be 3×3, 5×5, 7×7, anything right. So, we will get into that in more details later on. So, once again we slide this volume across the entire image ok. What is the output going to be? 2D or 3D?

Student: 2D.

Why? So when I was 1D I was getting 1D output, when I was 2D I was getting 2D output, 3D again 2D output why? Because I have assumed that, no not width.

Student: (Refer Time: 16:49).

The depth of the filter is the same as the depth of the input. So now, you just imagine this if you can. Suppose the filter was of depth 2 instead of 3 then, I would slide it horizontally first, vertically and then across the depth also. So, then what would be output be in that case?

Student: 3 dimensional.

3 dimensional and it would have depth of 2.

Student: 2.

Everyone gets that right, but for this lecture I am always going to assume that the depth of the filter is equal to the depth of the input always right. And that is how it is for all the convolution neural networks, that we will see the depth of the input is going to be equal to the depth of the filter, the rather the depth of the filter is going to be equal to the depth of the input. So whenever, I apply a 3D filter I am actually doing a 2D convolution because, I am moving only along the width and the height, I am not moving along the depth. So, the output is going to be 2D. So now, can I have multiple such filters? Yes each filter will give me a 2D output, if I have k such filters I will have a.

Student: (Refer Time: 17:54).

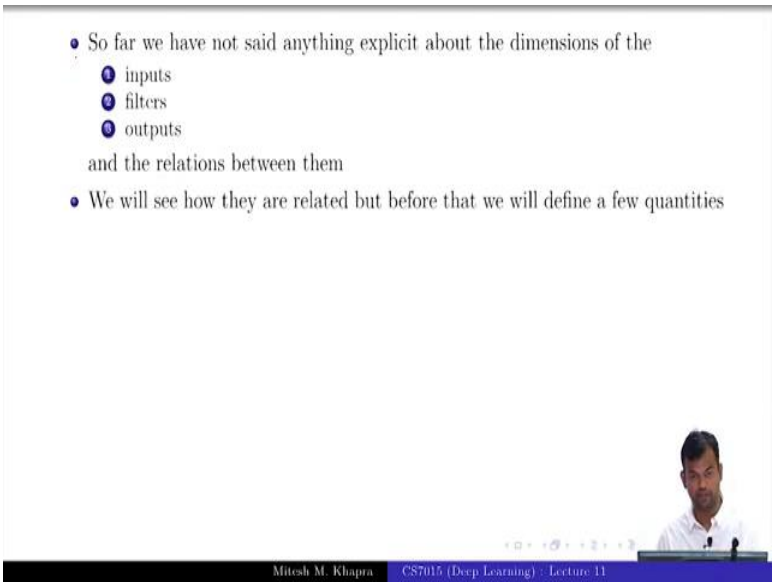
k $\times$2D output right k 2D outputs fine.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 11
Relation between input size, output size and filter size

Ok. So, now we will go to the next module where we will try to learn the Relationship between the input size, the output size and the filter size, ok.

(Refer Slide Time: 00:21)



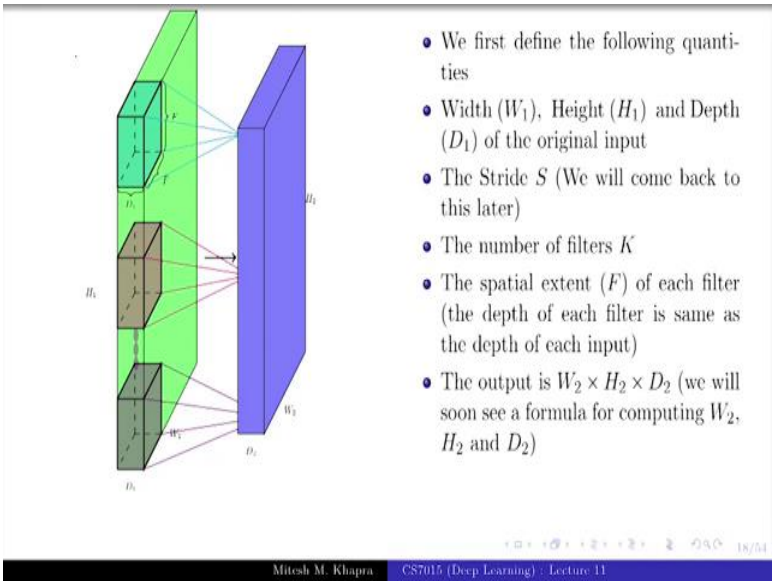
So, so far we have not said anything about the dimensions of the inputs I have just been very vague that its M $\times$ N and also for the filter I have just said 3 $\times$ 3; 5 $\times$ 5 and so on. And in fact, I am not even told you what the dimensions of the outputs are? Except that I be got some intuition that it seems that the output dimension is smaller than the input dimension right.

So, let us look at these in more detail and see what do these different outputs look like. Why do I care about these things? What do the inputs and the output sizes look like? Because these are your matrices that you will be dealing with this tell you these tell you how many parameters you are going to have.

These tell you what is the size of the memory that you need to load this entire network into your memory and so on it. So, that is why this computation is very very important

and I will have some more things to say about it towards the end of this lecture or some lecture ok.

(Refer Slide Time: 01:06)



So, if you first define the following quantities. So, we have the input which has a width W1, height H1, and depth D1. So, if you are looking at the original image then the depth is going to be three in most cases it is going to be RGB ok. There is something known as the stride S, I will come back to it later on.

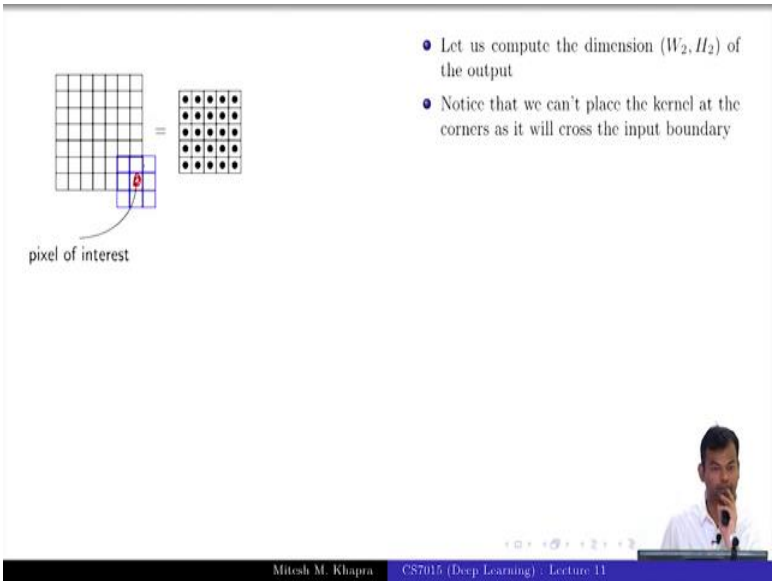
But I am just defining it here, or others just introducing it here. And then you have number of filters. So, I said that every filter that you apply you are going to get one feature map which is two dimensional. If you have K such filters, you will get K feature maps each of them is 2D right.

So, we will have something known as number of filters K and then you will have something known as the spatial extent of these filters. So, that is the number 3×3; 5×5 which I have been saying. So, that is known as the spatial extent I am going I am going to refer to it as F ok, and we are going to always assume square filters.

So, it is always going to be F$\times$ F. Is that fine? Ok, and the depth of the filter is one more thing which I need to worry about. But I have already said that I am going to assume that the depth of the filter is the same as the depth of the input ok.

Now, the output is again a volume which is of size W2, H2, and D2. And my quest is to find out how do I compute these W2s, H2s, and D2s that is what I want to figure out. It is not very difficult, but I just want to do it properly, so that is what the setup is right. So, we have defined the sizes of everything on the input and the filters now you want to look at how do we get the output sizes ok.

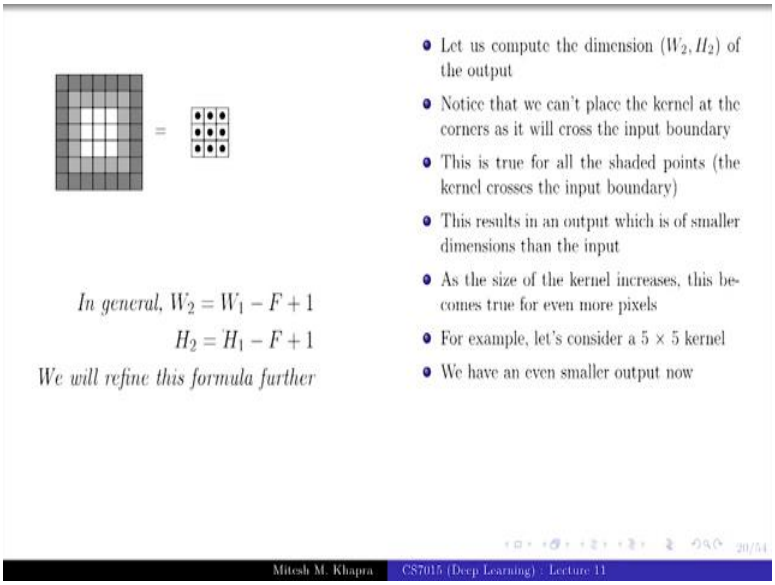
(Refer Slide Time: 02:39)



So, let us compute this for one example. So, this is my original image, so I am looking at a two dimensional input which I believe is 7×7. And I am applying a 3×3 filter to it. So, every time I slide the filter I will get one pixel in the output and I got the entire feature map.

Now it is obvious that the size of the output is less than the size of the input why is it so? Because there are certain pixels at which I cannot place the filter why? You will go out of the boundary right. So, I cannot if this is my pixel of interest, I cannot place my filter there. Because then the filter will go outside the image and I do not know what the average to come to how to compute the average right those values are undefined do you get that? Ok.

(Refer Slide Time: 03:26)



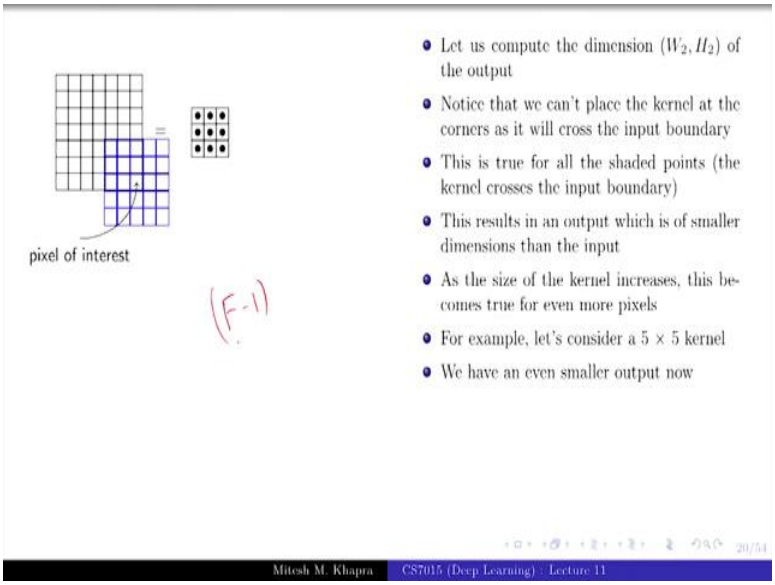
So, in general for let me see for the 3 cross. In fact, this is true for all these pixels which have been shaded or any of these pixels I cannot place the filter because you will go outside the boundary. So, now, for a 3 × 3 filter what is the reduction in the size of the output compared to the input? The width decreases by 2 and the height decreases by.

Student: 2.

2 right. So, can I be bold enough and say that the width and height decreases by not yet ok.

So, let us see if we had a 5 × 5 kernels ok. Then which are the places at which I will not be able to place the filter? These two shaded boxes and both these boxes I cannot place the filter because the filter will go outside the image.

(Refer Slide Time: 04:13)



So, now can you tell me how many? So, in this case the size reduce is by how much? The width reduces by.

Student: (Refer Time: 04:20).

No the width reduces by 4 sorry and the height reduces by?

Student: 4.

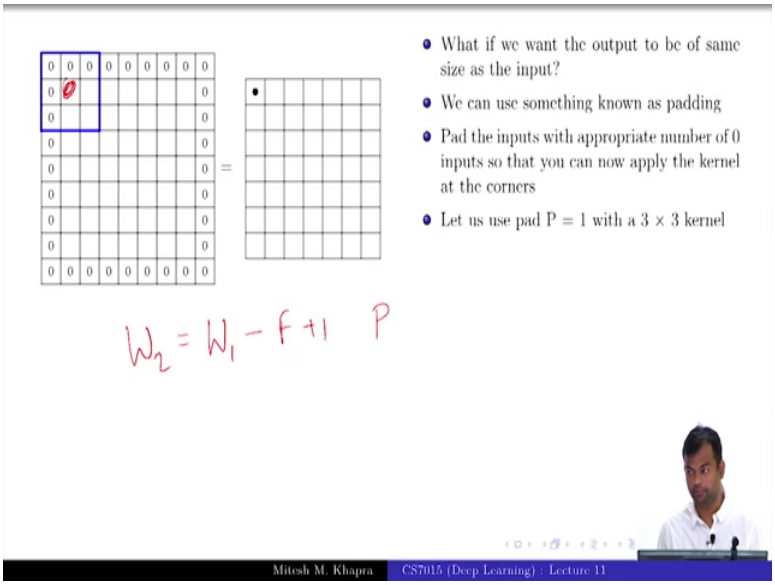
4. So now can I say that the width and height actually reduce by F minus 1, where F is the size of the filter is that ok? How many of you get this? So, you did not get this? No. You did not get this? Ok. So, in the 3×3 case you see that that is one row and one column on each side left and right which I cannot apply ok.

So, let us focus on the columns. So, columns give me the width right. So, when I have a 3×3 filter there are two columns which get subtracted, because these are the boundary columns. When I have a 5×5 filter how many columns get subtracted? 2 on the left hand side, 2 on the right side is a total of 4.

If I have a 7 cross 7 filter; 3 on the left hand side, 3 on the right hand side, so total of six. So, you see the relation it is always F - 1 right. So, your new width and height is always going to be W1 - F - 1 which is W 1 - F + 1 is that ok. Everyone gets this and same for

the height the width and height are going to be symmetric. Because the filter we have chosen to be symmetric it is F × F ok.

(Refer Slide Time: 05:28)



Now, but what if we want the output to be the same as the input? What do we do? Padding ok, you can use you know something known as padding. So that means, now I will have a boundary of 0's. So, I am saying that this is my original image and outside it there is a black border or a white border I do not know whether 0 stands for black, or white it is embarrassing, but.

Student: Black.

Black ok. So, it is a black border outside the image ok. And now I am going to take an average that way.

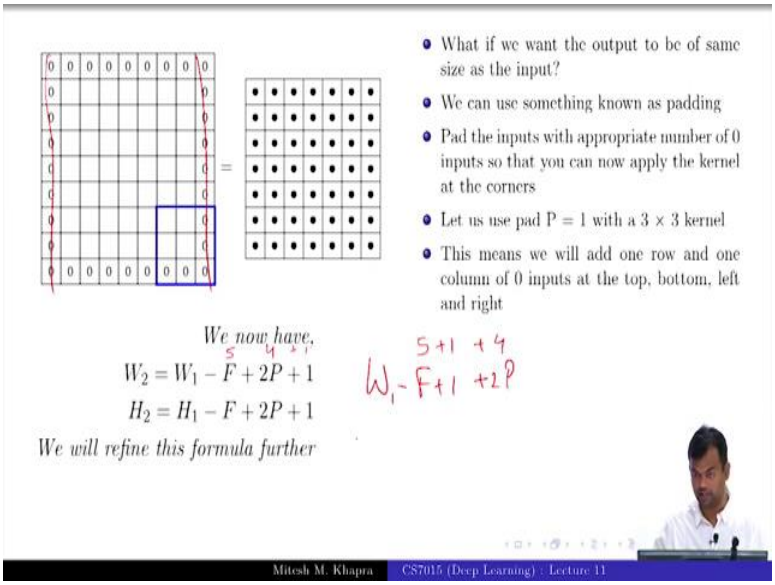
Now, this was the pixel earlier on which I could not place the filter. But now I can artificially place on it assuming that it there is a black boundary around it. So, now what would be the output size?

Student: (Refer Time: 06:08).

Same as the input. So, now can you tell me? So, I have W1, I have F and now I have something known as P also ok. So, I know that w output rather w2 is the output is 1.

Now when I add the padding what would the formula be? 2 P is that fine? Everyone gets that? Ok.

(Refer Slide Time: 06:33)



So, now if I have a 5 cross 5 filter and if I want the output size to be the same as the input size. What is the padding that I should use?

Student: P.

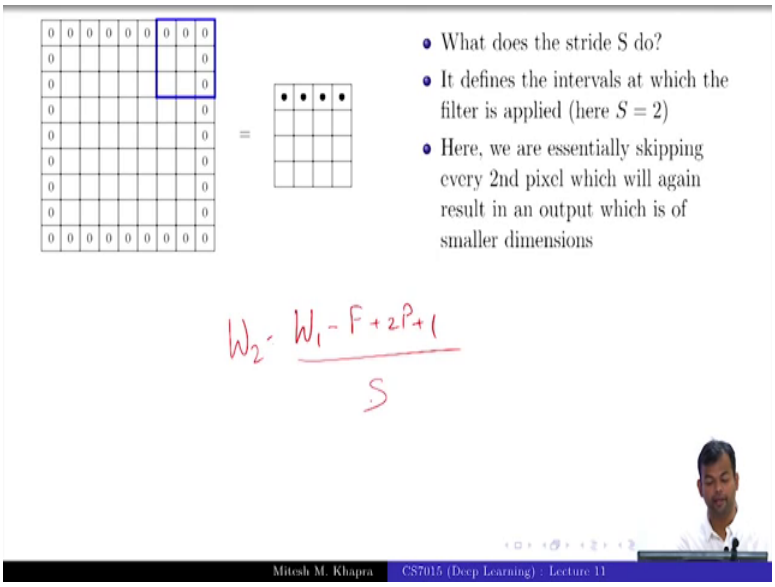
P is equal to 2 right that is clear from the example that we saw that there were two shaded columns and rows which were problematic. So, I need to do a padding of 2 and then if I substitute in this formula you can just see right. So, 5 + 4 + 1 right, so you will get back W1 is that fine is that ok. How many if you get this? How the formula works with the padding? How many of you do not get this please raise your hands? You do not get this ok.

So, remember in the 3 × 3 case there was one column on the left hand right which was a problem. So, when I say a padding of one I add one column to the left; one column to the right; one column to the bottom and one column to the top.

And that is exactly the problematic region in the 3 × 3 case right. So that means, this was my original formula ok. Now the new width is going to be plus 2 times the padding which I am going to use. Because I have used one padding here, and one padding here right. Now, in the 3 × 3 case that is ok.

Now in the 5 × 5 case how many columns are problematic? 2 so; that means, I have use a padding of 2, when I say a padding of 2 I add 2 on all the sides. So, now again if you substitute in this formula so you would have W1 - 5 + 1 + 4. So, that will give you back W1 right. So, that is how it is right.

(Refer Slide Time: 08:08)



Now, the question is you have if you have taken care of filter size and padding. Now the other thing that we need to look at is stride ok. So, the stride defines the interval at which the filter is applied. Now what do I mean by that?

So, remember that stride is basically a step right the same definition as it is applies to walking rate. So, right now what we were doing is we placed the filter at as this pixel at the center, then this pixel has the center, and then this pixel has the center. Instead of that I could take a that means, my stride is 1, I am taking one step at a time. So, if I do a stride of 2 what would happen?

Student: (Refer Time: 08:48).

I will apply two alternate pixels right. So, this is how I will move. So, then what would happen to my output size if my stride is 2?

Student: (Refer Time: 08:57).

What would happen?

Student: (Refer Time: 08:59).

It will become half ok. So, what would the formula be then? So, so far my formula was now if I have a stride of 2 what would the formula be?

Student: (Refer Time: 09:12).

They divide the whole thing by 2.

Student: (Refer Time: 09:16).

By S right. If it was S was 3 then this would have become one-third roughly right. If S was 4 this would have become one-fourth. So what should I divide by?

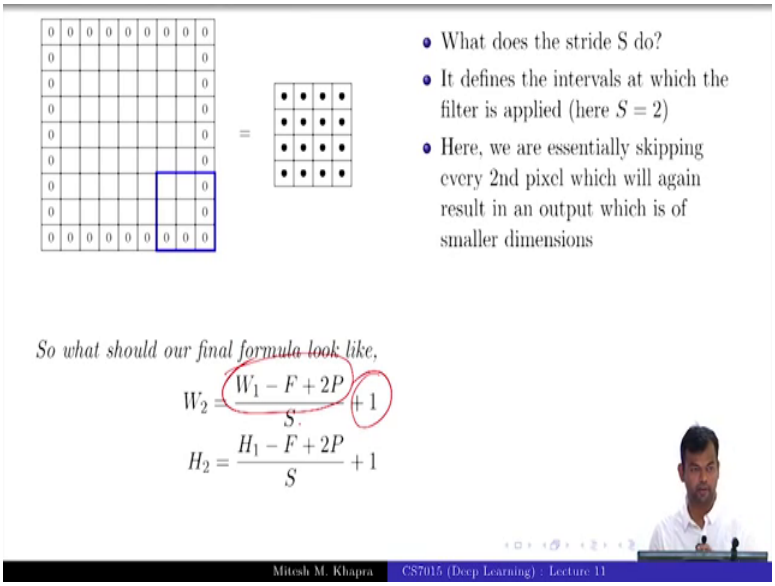
Student: S.

S. So, I should divide the whole thing by S.

Student: (Refer Time: 09:32).

S ok.

(Refer Slide Time: 09:36)

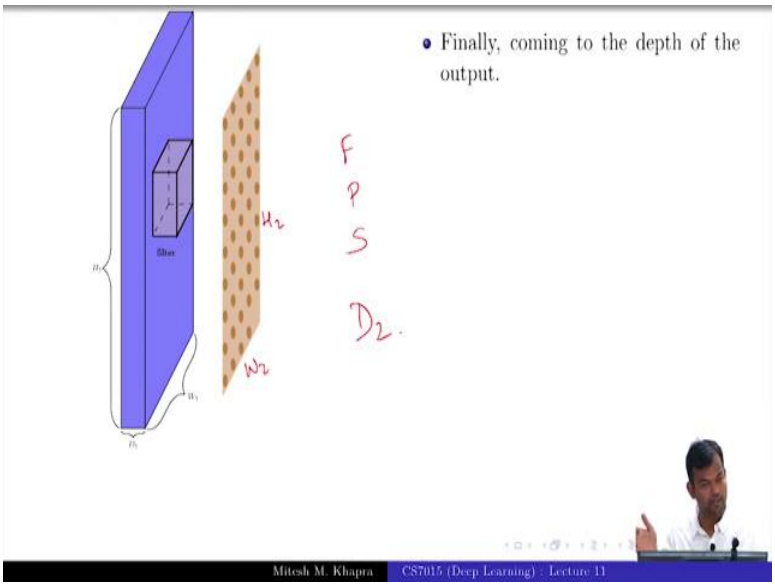


So, it turns out that is not exactly that, but you get the intuition and you can work out the formula. So you do not divide this by S and you will figure it out, it is easy to see because of some ceiling and flooring and things like that. So, you can go back and check

that out and basically you could just think of this that, this was your original weight in the absence of stride, or rather than the stride was one.

So, now if you are going to take longer strides you have two account for that if you take a stride of 2, stride of 2 your width is going to become half; you should take a stride of 4 your width is going to become one-fourth is that fine? Do not worry about this additional plus point you can go home and figure it out.

(Refer Slide Time: 10:06)



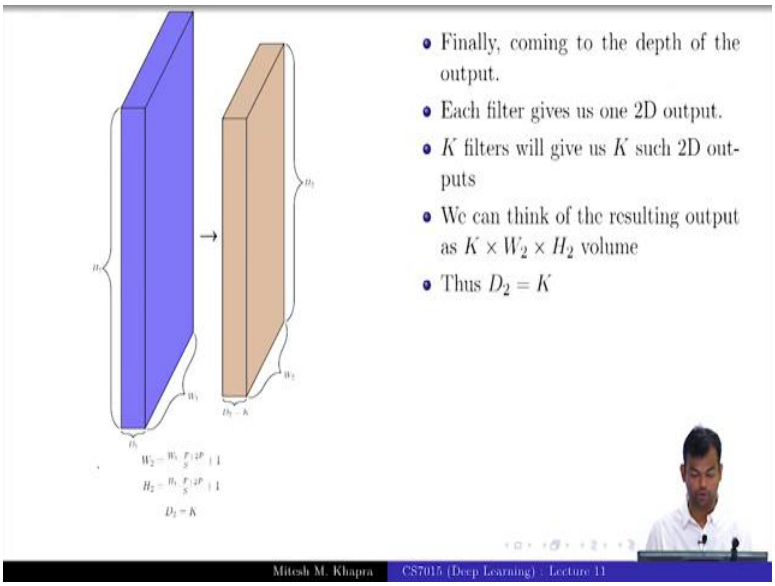
Finally coming to the depth of the output what would the depth of the output be? So, let me just see right now all our formulas were W2, H2 in the presence of filter padding and size, a stride sorry. But we never had a formula for D2, so that is what I am asking.

Now what is the depth of the output? Same as the every filter is going to give you one two dimensional output. If you have K filters.

Student: K.

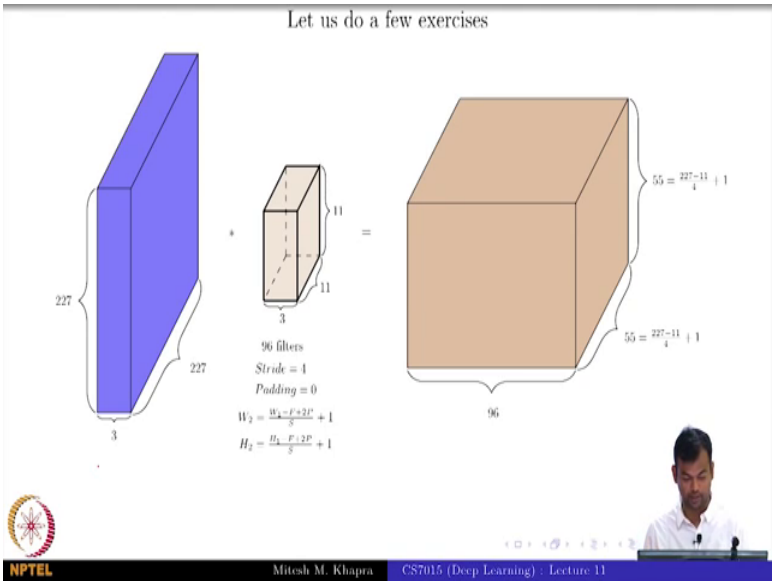
You will get K two dimensional outputs. That means, the depth of your output is K right.

(Refer Slide Time: 10:38)



So, the depth is very simple it is just equal to K the number of filters that you have. So, I want you guys to note down these three formulas.

(Refer Slide Time: 10:48)



Now, let us do some exercises. So, this is my original input which is 227 × 227 × 3. I have decided to apply 11 cross 11 filters and I am not going to tell you the depth of the filter, because it is going to be the same as the input ok.

And I have 96 such filters I have decided to use a stride of 4 there is no padding. Can you tell me what is the output volume going to look like? What are the dimensions of the output volume? Ok. So, D2 is simple. What is D2? 96. What is W2?

Student: 55.

55. H2? Ok you guys have the last class fine. So, similarly you can do it for so actually the computation which I had that this was just not some random computation. This is actually the first convolution layer from Alexnet right, so one of the popular architectures that we are going to cover later on right.

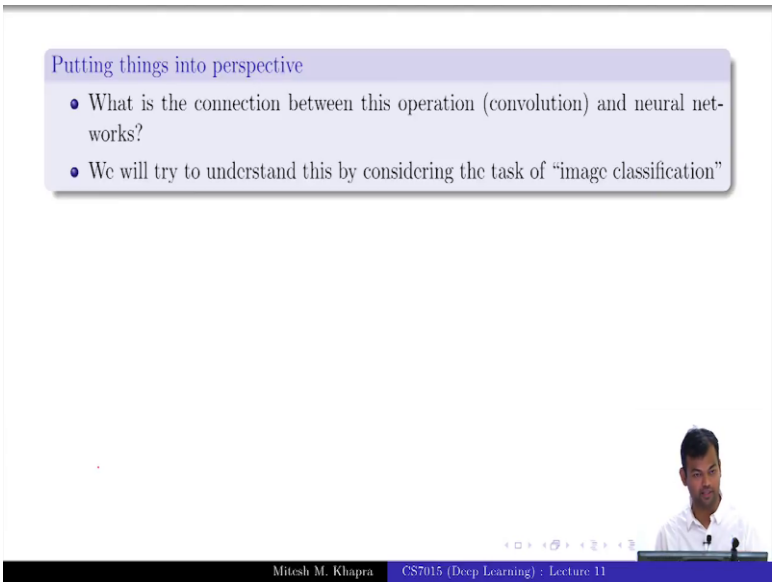
So, this is what Aalexnet does at its input. It takes the RGB input and gives you a volume of this slice this side and then there is something else with this volume right. So, we will see that later on. There is one more exercise you can do it later on, I do not want to do it now ok.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 11
Convolutional Neural Networks

Ok. So now, we will go to the next module, this is for the camera and this is on Convolutional Neural Networks.

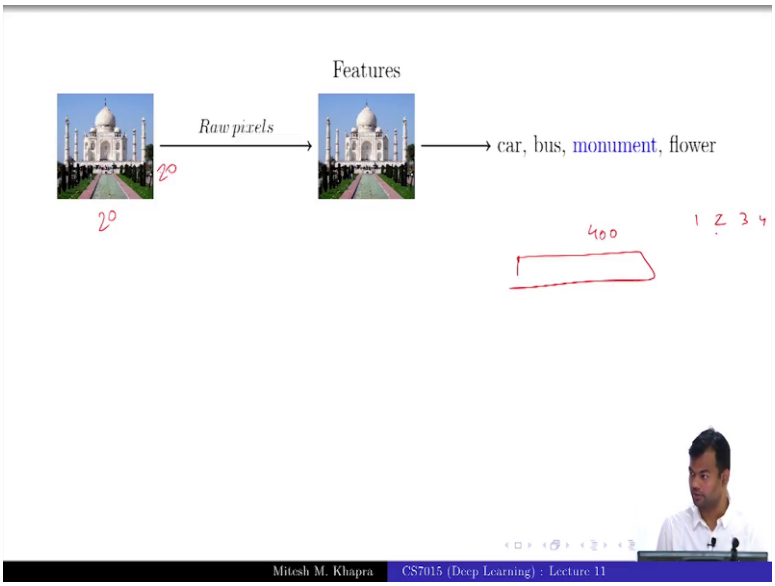
(Refer Slide Time: 00:17)



So far, we have done what we have just talked about a convolution operation, you just taken some input boxes and converted them to output boxes, what does this anything of this have to do with neural networks? I keep saying that is a course on neural networks right. So, everything has to link to that. So, what is the connection?

So, we will try to understand this by taking the example of image classification and I will use the same trick to get everyone's attention. So, the next few slides are going to tell you the difference between machine learning and deep learning ok. So now, everyone will pay attention.

(Refer Slide Time: 00:49)

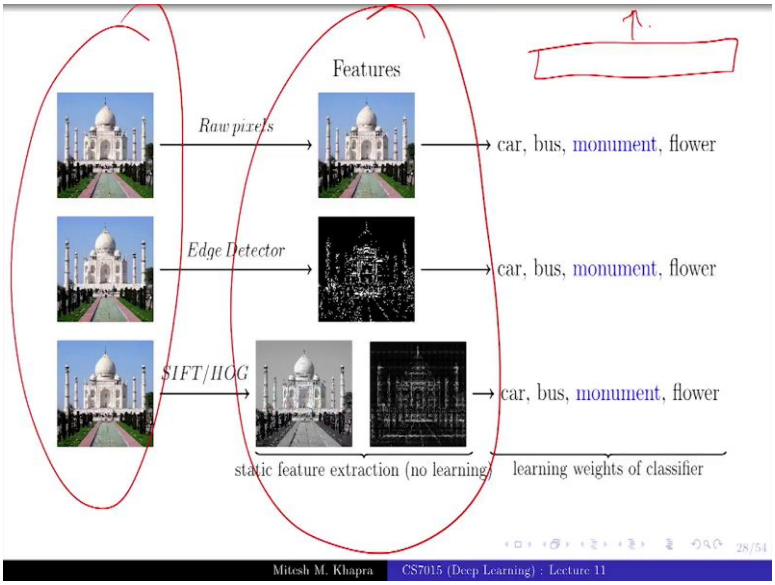


So, this is the task you have give given an image and you want to classify it into one of k categories and I am considering 4 categories here car, bus, monument, flower ok. What is the simplest thing that you can do? Suppose this is a 20 cross 20 image you know the simplest thing.

Student: Sir.

Given on the slide; you will just take this as a 400 dimensional input feature vector right and you will treat it as a 4 class classification problem. Train some multiclass SVM or anything on that right. So, you have a simple input so, you are given some 1 million images, all of these are 400 dimensional and they come from 1, 2, 3 or 4, these are the 4 classes, which is car, bus, monument and so on.

(Refer Slide Time: 01:43)



So, you can just treat this as an input feature vector and do your classification right that is the simplest thing that you do or else what you could do is; you could do some kind of feature engineering right, you could say that actually this entire blue sky is not really helping me in deciding anything, these entire green lawns and all this is not helping, if monument, car, bus and flower are the classes, what I care about is the shapes, I do not care about the details inside the shapes, I am not trying to decide whether the car is of a blue color or what model the car is and so on right, all I want to see is that this particular shape of a car is present or not?

Now, what kind of filter gives you the shape of the image?

Student: (Refer Time: 02:16).

Edge detector right so, I could use edge detector. So now, this is something that I have used based on my domain knowledge, that for these 4 classes actually, just detecting the shape is important. So, I will ignore everything else. So, there is a lot of details there right. So, I have actually sparcified my entire input, I have just looking at the edges in the input and now this is a better refined feature as compared to the earlier feature, how many of you agree that this is a more refined feature representation right.

But this was handcrafted, I actually hand coded the edge detector kernel, because I knew that it is 8 at the center and minus 1 everywhere else right, that is how I thought of it that that is what an edge detector is or at least I read about it somewhere right. So, that is how you would do it. So, this is feature engineering.

So, what is this? This is how you do machine learning right, you take an input you do some feature engineering and then you train a classifier on top of that, but now you could become even more creative with the feature engineering and that is what the computer vision community was doing largely before 2012. Come up with different ways of capturing better and better features from images. So, too popular in features from that era and that is I am just talking about 2012 not some like 500 years back, but from that era was SIFT and HOG features, which actually tell you how do the gradients of these pixels change across the image right.

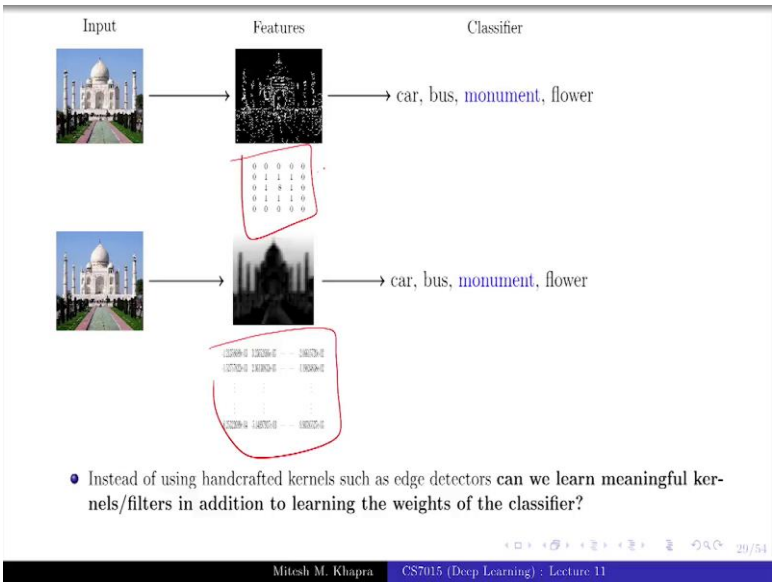
So, this is again try to capture something like how, what is the variation in the image from pixel to pixel right. So, that is the essence that how is you do not care about these entire blue patterns, because they are just telling you sky it is redundant right, if you have seen some 10 pixels or 20 pixels, which has sky you know that a large part of it is going to be sky.

So, these try to capture some abstractions from the image and these are better than the edge detectors and these features were extremely popular. So, what you would do is you take your original input, this is a deterministic algorithm, you apply the HOG algorithm or the SIFT algorithm and it gives you a transformed representation for the image and you can use this transform representation to do classification. And a lot of work prior to 2012, 2011 show that these features work extremely, well across a wide variety of across a wide variety of image tasks ok.

So again, what was happening here? This was feature engineering because someone realized that what I care about is this gradation in the input images and I can capture this by this nice algorithm called SIFT or HOG of course, someone came up with that algorithm, but it is still kind of feature engineering right.

So, this is how the learning is to happen is you are given some input, you do a static feature extraction no learning. So, feature extraction is deterministic you take the input pass it through one of these algorithms; either the edge detector or the blur detector or SIFT or HOG and you get some representations for the input. And the only learning that happens is on top of this transformed input. So, you now have a transformed input and on top of that you are going to train a classifier and you are going to learn the weights of the classifier. So, the only thing that you learn is the weights of the classifier.

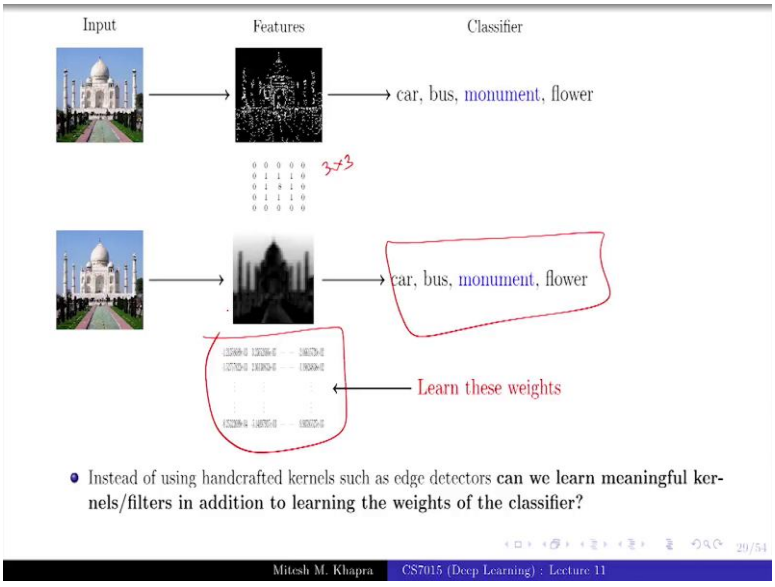
(Refer Slide Time: 05:25)



So, that is equivalent to learning only the soft max layer, in case of a feed forward network that is the output layer right

Now, instead of using these so, this is the question instead of using these handcrafted kernels or features, such as edge detectors or blur detectors or what not; can we learn meaningful kernels in addition to learning the weights of the classifier, do you get this? Question at least, whether the answer or not, but you get the question. So, what I am asking is that why should I hand code this edge detector ok.

(Refer Slide Time: 05:55)



Why not have after what is the edge detector; it is like a 3 cross 3 matrix right and I have seen tons of such matrices in my feed forward neural networks, I have dealt with very large matrices, which were called parameters of the network.

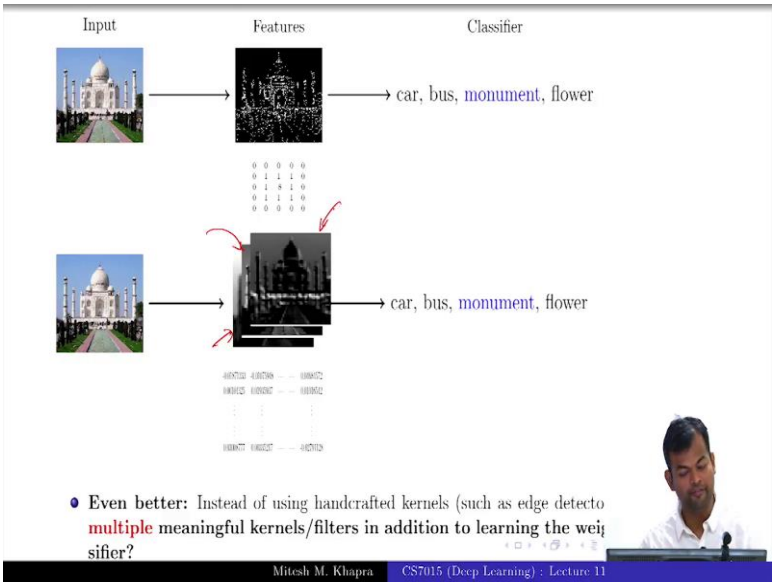
So, why not have a 3 cross 3 or a 5 cross 5 or whatever dimensional matrix and try to learn, what should be the right values in this matrix instead of hand coding the edge detector matrix. Do you get the idea? How to do that as still far, but at least do you get the idea that is what I am we are trying to do ok.

So, now instead of just learning the weights of the classifier, we also want to learn the weights of the kernels; that means, instead of using handcrafted features, I am now going to;

Student: (Refer Time: 06:37).

Learn the features. So, that is the difference between deep learning and machine learning. So, you had handcrafted features there and now you are going to learn the representations also by treating them as additional parameters in your network.How you will do that; we will see and it is very simple given that you understand, how to train feed forward networks.

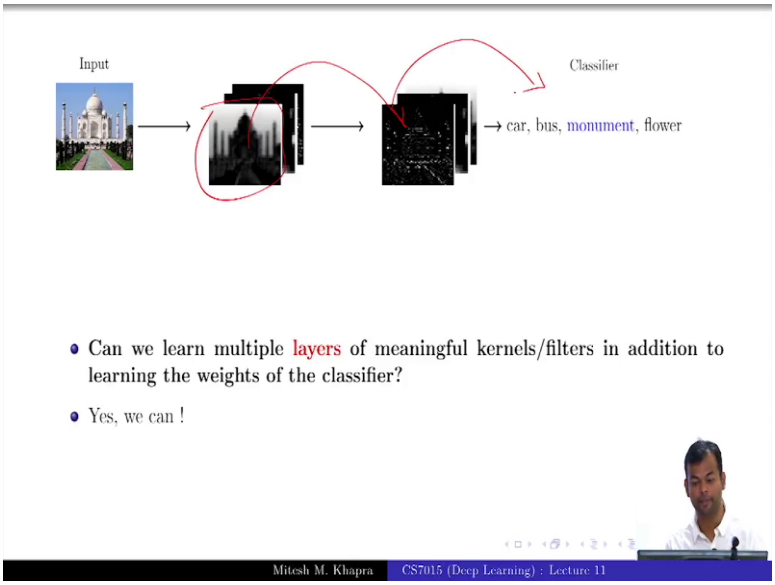
(Refer Slide Time: 07:01)



But then why the stop there? Why just have one feature representation for the input? Can I learn multiple such kernels right? I could have one 3 cross 3 matrix, which learned this

first representation another, 3 cross matrix, which learned this another representation and yet another 3×3 or 5×5 or 7×7 matrix, which learns this different representation. So, instead of learning one static representation from the input, I could learn multiple representations from the input.

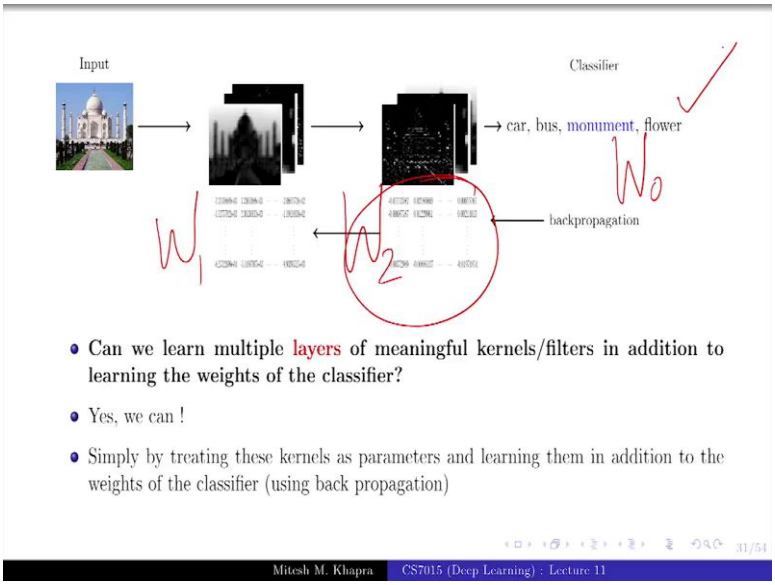
(Refer Slide Time: 07:29)



In fact, why not why just stop there, what is the next thing that I am going to try to do? Multiple layers of features right so; that means, at the first layer I learned this representation, now I could take this and try to learn an even more abstract representation and then keep doing this to make it deeper and deeper. Do you get this? ok.

So, at every stage now I have these parameters, which are helping me learn the representation of the input, I am learning multiple representations at every layer and then having multiple layers of these representations right and everything is learnable end to end ok. So, you get the difference between deep learning machine, learning now there is no handcrafting of features, you are learning the feature representation. I know, I understand there is some confusion about how you would do this.

(Refer Slide Time: 08:15)



But we will get to that just trust me on that that you will be able to figure out how to do this ok.

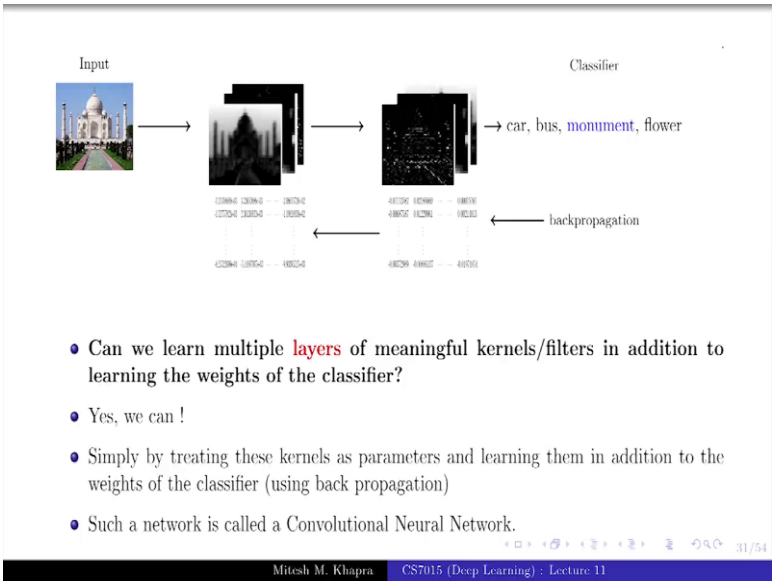
And all of this we have some weight matrices here, we have some weight matrices here, these are the layer 1 weight matrices, the other layer 2 weight matrices and these are the output layer matrices and you see this layer wise, arrangement of these weight matrices and they are very comfortable with this, because we have done feed forward neural networks, where we had these multiple layers and we knew how to back propagate from the last layer to the first layer.

Now, what I am trying to say is that, I would like to adjust these weights of filters in such a way that my classification loss is minimized and what is the loss function that I am going to use here?

Student: (Refer Time: 08:51).

Cross entropy, because this is a multi class classification problem ok.

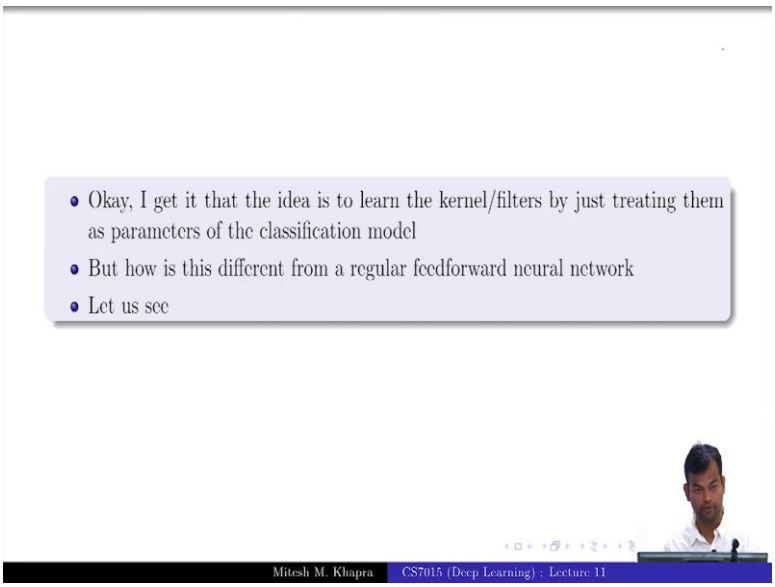
(Refer Slide Time: 08:59)



So, just hang on with this intuition and we will make it more clear fine.

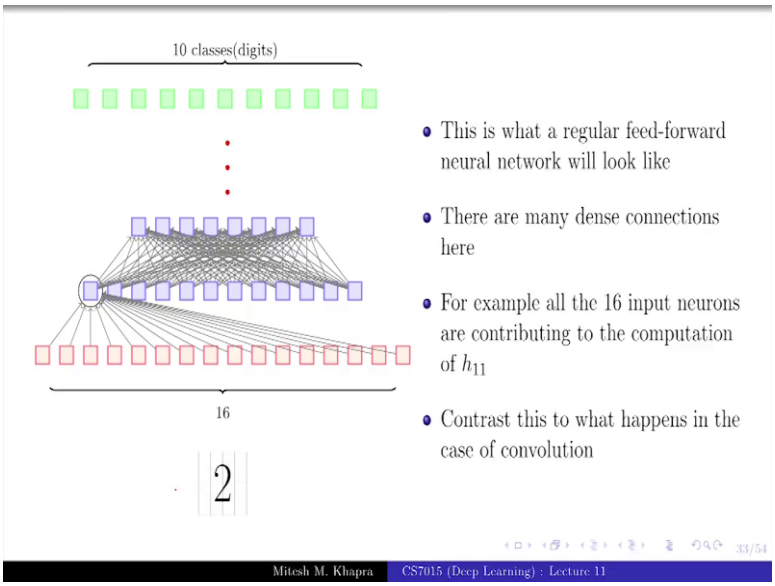
So, such a network which has these multiple convolution, learned convolution operations at every layer and then multiple such layers is known as convolutional neural network.

(Refer Slide Time: 09:09)



Ok fine. So, get this idea that we need to learn kernel filters by just treating them as parameters of the classification model ok, but how is this different from a regular feed forward neural network? You could have taken a regular feed forward neural network and I will show it to you on the next slide and what is the difference between that and a convolution operation?

(Refer Slide Time: 09:29)

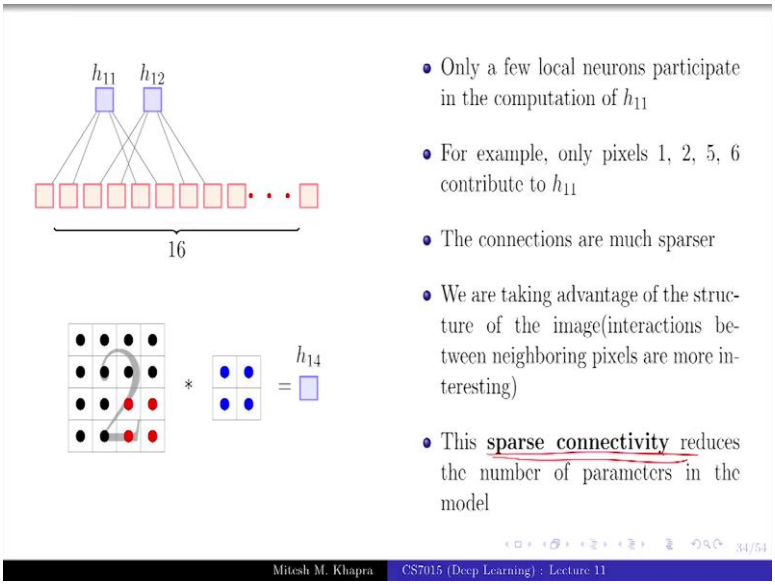


So, if you understand that then, you would be done for this lecture. Yeah.

So, we have an input. So, let us say now, I will take back the eminist case, where you are given an input as an image and these are digit inputs and you want to classify them into 1 of 10 inputs and I am going to assume that, my input is 4 cross 4; that means, I have 16 pixels ok.

So, the simplest thing that, I could have done or the feed forward neural network way of doing this is that, I would just flatten out this image, I will get 16 inputs, I need 10 outputs at the output layer. So, I have an output layer, which will have 1 of these 10 classes and then, I add as many layers that I want in between ok. This is what a feed forward neural network would look like and if I consider any one neuron in the first layer, it takes inputs from all the 16 inputs right, that is how a feed forward neural network is, you have these extremely dense connections, where every output depends on every input at every layer ok.

(Refer Slide Time: 10:31)



Now, so, this is the same story which I have said. Now, let us look at what a convolutional neural network looks like. So, again you have these 16 inputs, I am using a 2 cross 2 convolution ok. Now if I use a 2 cross 2 convolution, if I place it here, then I am using pixels 1, 2, 5 and 6 and computing one value. So, you see the difference between this and a feed forward neural network, in a feed forward neural network h_{11} would have depended on.

Student: (Refer Time: 11:03).

16 values; 16 inputs and a convolutional neural network it is depending on.

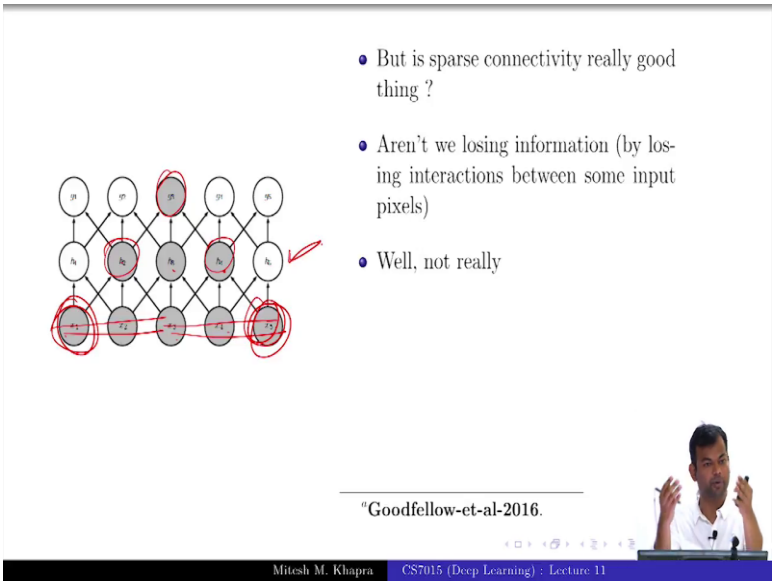
Student: (Refer Time: 11:09).

Only 4, only 4 neighbors, ok. and Similarly h_{12}, I am using a stride of 2 by the way right. So, I am not placing the filter here, I am just skipping one step. h_{12} would depend on pixels 3 4 7 8 , ok and so on right.

So, one thing is clear that as opposed to a feed forward neural network, you have sparser connections here. Is that clear? And why do we have sparse connections? Because we are exploiting our knowledge about images that in an image, you do not really care about the interactions between on between a pixel at the leftmost left top most corner and the right bottom corner right. So, there is sky here, there is ocean here or there are trees here, you would want to capture the neighborhood around that pixel not really capture it with the

entire image, that is why you do not want all of these 16 inputs to contribute, you only want a small neighborhood to contribute. Do you get that intuition ok.

(Refer Slide Time: 12:13)



So, this is the first property of a convolutional neural network that it has sparse connectivity ok. But its sparse connectivity really good? I just made a case for that. Now I am going to counter argue right, is it really good that you have these sparse connections? Because you are losing out information right, you are using out interactions between certain pixels. So, why can, why is that good?

Student: (Refer Time: 12:29).

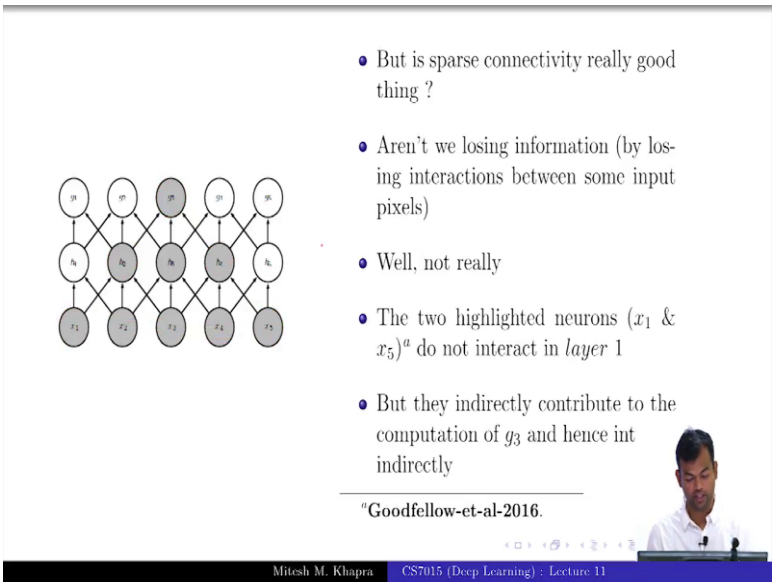
I am hearing a lot of interesting answers, but remember that you are always going to have multiple layers ok. So, consider these 2 pixels. In the first layer these 2 pixels did not interact, because h_2 only dependent on these 3 and h4 only dependent on these 3. There is no a there is no unit here which depends on both x1 and x5. Is that obvious? Because I am just using a window of size 3.

But now once I go to the next layer, once I go to g_3; g_3 depends on h_2, h_3, h_4 here, which in turn depends on x_1 x_2 x_3 x_4 x_5 right. So, even though at this layer x_1 and x_5 are not interacting with each other as you go deeper these interactions become obvious. Do you get that? Right. So, that is why you will always use a deep convolutional neural network,

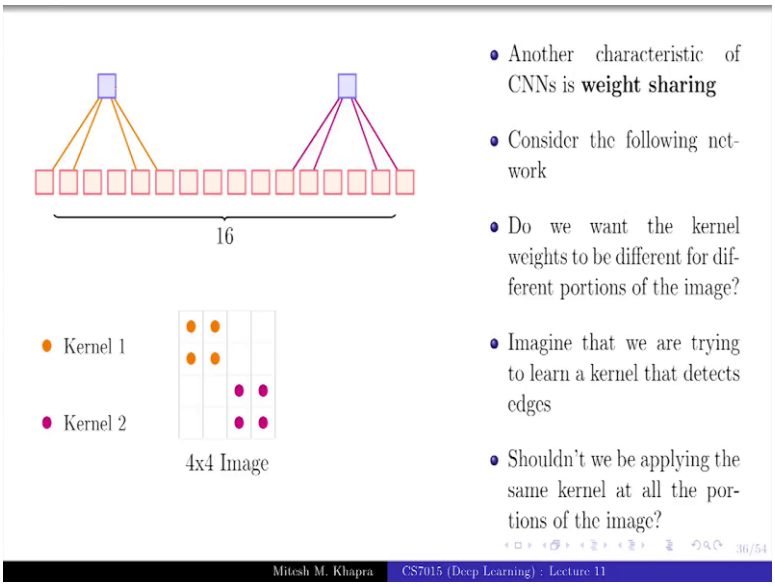
where all the pixels get to interact at a deeper layer, but at the more image, it layers you just want to capture the interactions between a neighborhood.

So, it is like you take this neighborhood find out something, then take neighborhoods of neighborhoods and then try to find out something at the next layer and keep continuing in this layer, how many of you get this? Right ok. So, this is what sparse connectivity looks like.

(Refer Slide Time: 13:43)



(Refer Slide Time: 13:47)



Another characteristic of CNNs is something known as weight sharing. So, let us see what it is? So, remember I had considered this 2 cross 2 kernel and I was placing it at these 4 pixels, which is pixels 1, 2, 5 and 6 and I was pacing another kernel at these 4 pixels, which is pixels 11, 12, 15 and 16 right, these 4 pixels. And I have used different colors for them indicating that these filters are different. So, they are both 2 cross 2 filters, but I am assuming at the values inside them are different. Does this have to be the case? Just think what a filter is trying to do.

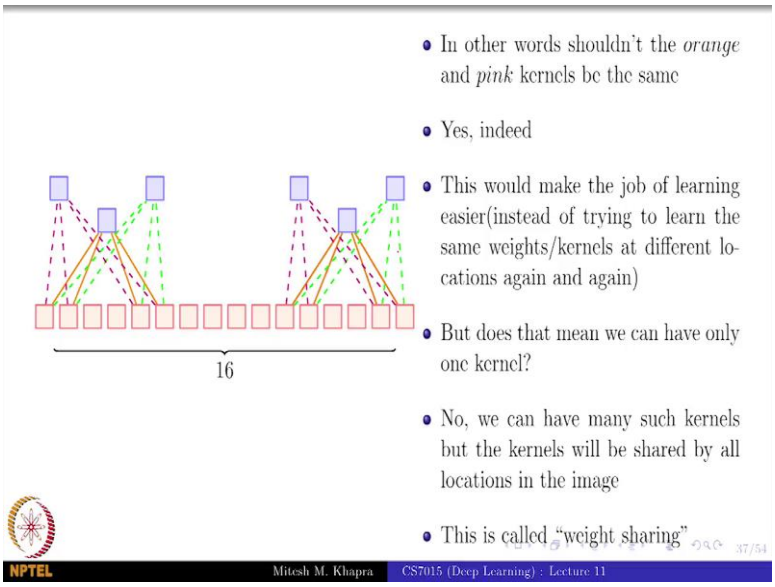
Student: (Refer Time: 14:27).

It is striding across the entire image. At every location I want to do the same operation. Remember, when we are doing blurring or edge detection or sharpening I had the same filter, which I was applying at every location. So, I want to see, what is the effect of this filter throughout the image? So, I do not really want to change this filter; that means these 4 weights would be the same as.

Student: pink weights.

The pink weights; how many of you get this? So, this is a question, do we want the kernel weights to be different for different portions of the image? So, imagine that we are trying to learn a kernel that detects edges. So, the same kernel configuration is required throughout the image because, that is the kernel configuration, which will detect an edge.

(Refer Slide Time: 15:17)



So, you want the same kernel to be there everywhere. So, we are going to share these weights, they should not be these pink and orange weights, we will just have the same weights everywhere ok. So, this is known as weight sharing.

So, now this is something ridiculous, if you think about it now how many weights do I have in layer 1?

Student: (Refer Time: 15:33).

4 weights that is all that looks too less right it would lead to.

Student: (Refer Time: 15:39).

Dash fitting.

Student: (Refer Time: 15:42).

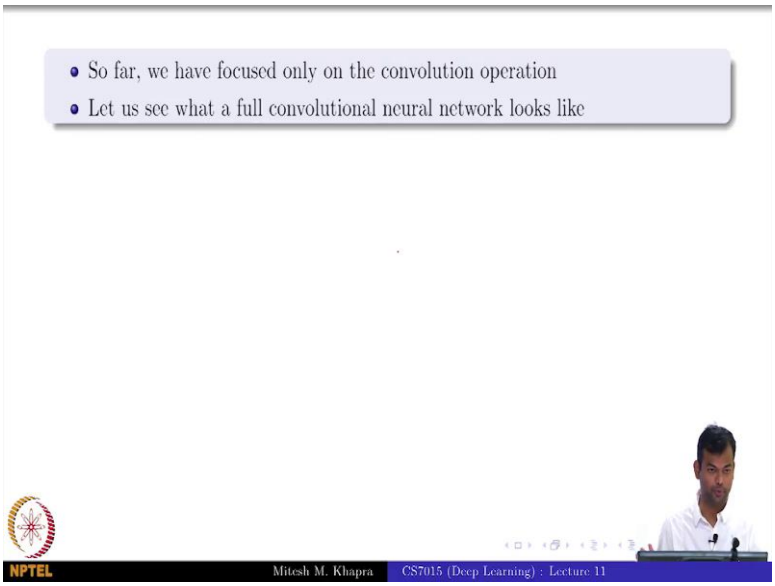
Under fitting because we have very few parameters so, how do I deal with the situation?

Student: (Refer Time: 15:45).

You will have multiple kernels right. So, you will have another kernel, which takes something else you will have one more kernel, which takes something else and you can have as many such kernels right. So, the more the number of kernels will have you will have that many into 4, as the number of parameters and that many outputs at layer 1. How many if you get this? Ok good.

So, these are the 2 important characteristics of convolutional neural networks, one is sparse connections and the other is weight sharing ok.

(Refer Slide Time: 16:17)



So, so far we have focused only on the convolution operation. Now let us see, what does a full convolutional neural network look like or maybe I will do this next time, I think this is.

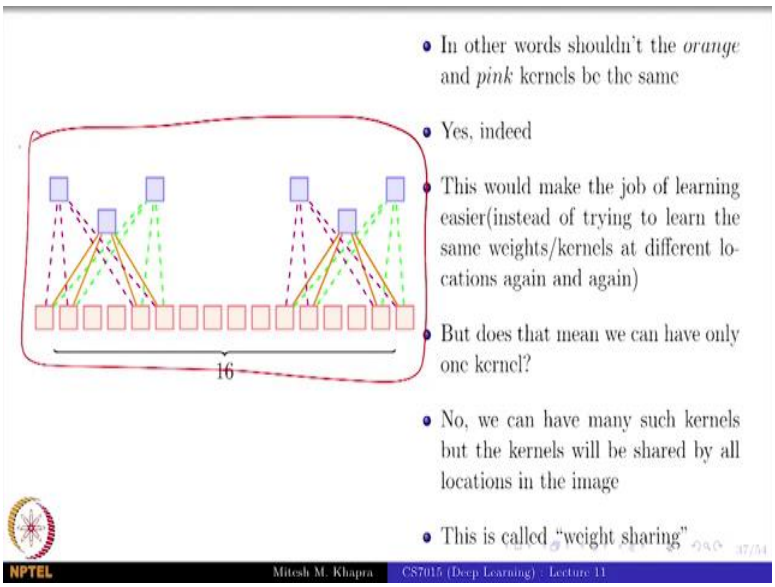
Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 11
Convolutional Neural Networks (Contd.)

(Refer Slide Time: 00:11)



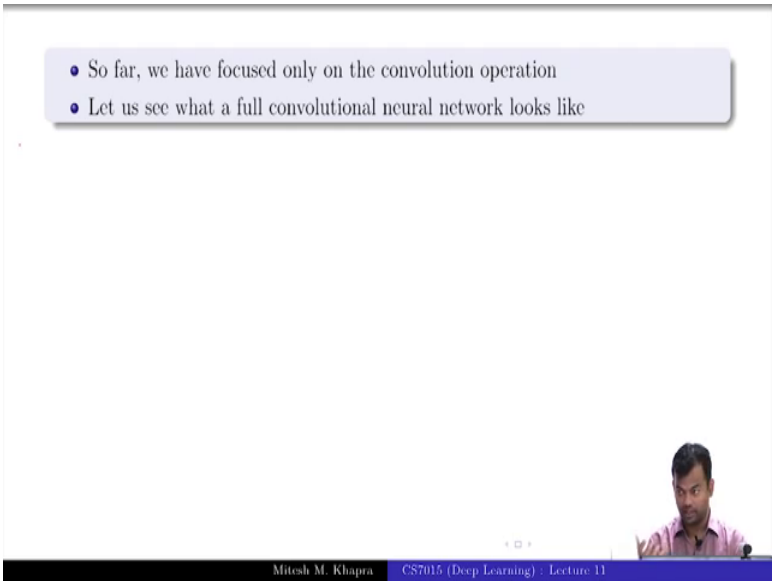
(Refer Slide Time: 00:17)



So, we will start from where we left yesterday. So, this is what we had seen yesterday, we saw what's the difference between a convolutional neural network and a feed forward

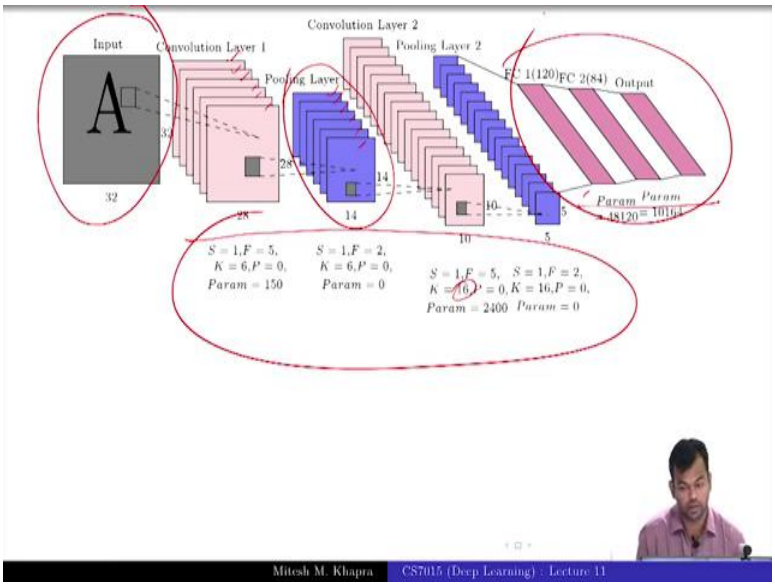
neural network and we focused on two main properties, one is sparse connectivity and the other was weight sharing. That is about it and then we saw that this representation of I mean, we saw this diagram about how to you could have multiple kernels and each kernel would apply across the entire image and the weights would be shared for that kernel.

(Refer Slide Time: 00:43)



So, far we have only focused on the convolution operation and even when you have seen the neural network or the convolutional neural network, we have only seen the convolution layers , right? So, there is something more in a typical convolutional neural network and that is what I was about to start yesterday. So, we just continue from there.

(Refer Slide Time: 00:59)



So, this is what a full convolutional neural network looks like. So, ignore these things for now, all these parameters etcetera as a just ignore for them, I will just walk you through, what is important in this diagram, right?.

So, your input is an image and the tasks that you are dealing with here is digit recognition or handwritten digit recognition right and what you see here is that you have taken an input which is a 2 dimensional input and then the next layer you actually see 1, 2, 3, 4, 5, 6 outputs. So, what does that mean?

Student: (Refer Time: 01:33).

You have use 6 filters apply them, throughout the filter, throughout the image each filter, gave you 1 feature map. And so in this layer, you have 6 such feature maps. So, the original 2 dimensional input has now become 6 2 dimensional outputs. ok. After that there is something known as a pooling layer, we will see what a pooling layer is in detail.

Now, what I want you to understand is lets assume that, what the pooling layer does is it does some kind of a shrinking, it takes the original output and shrinks it. How it shrinks it? We will see in a while. But let us see what happens after that. So now, you have 1, 2, 3, 4, 5, 6 as your input. So now, you have this is your input, this volume is now, your input I call it a volume, because it has depth height and width and now you are again

going to apply convolutions to that and what do you see, how many outputs do we have now? We have 16 outputs , right? So, what does that mean?

Student: (Refer Time: 02:27).

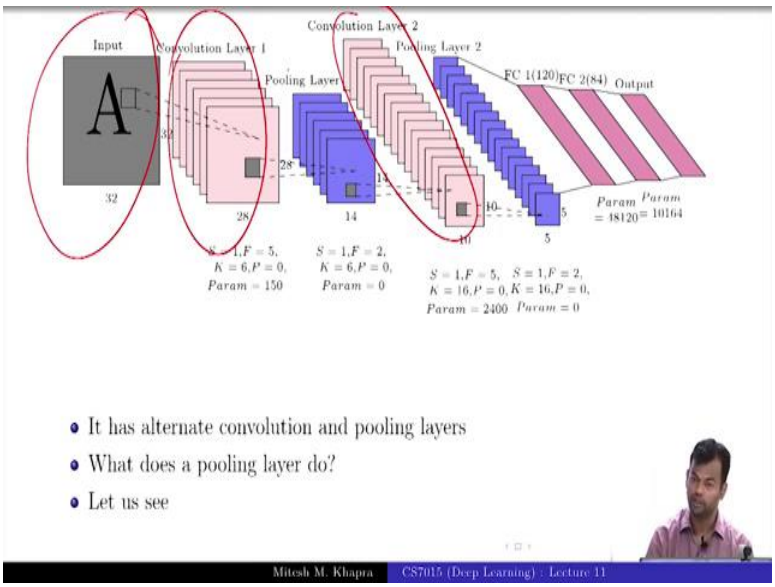
I took this 3D input applied 16 3D filters on it each 3D filter give me 1 feature map, one 2D feature map, why one 2D feature map?

Student: (Refer Time: 02:39).

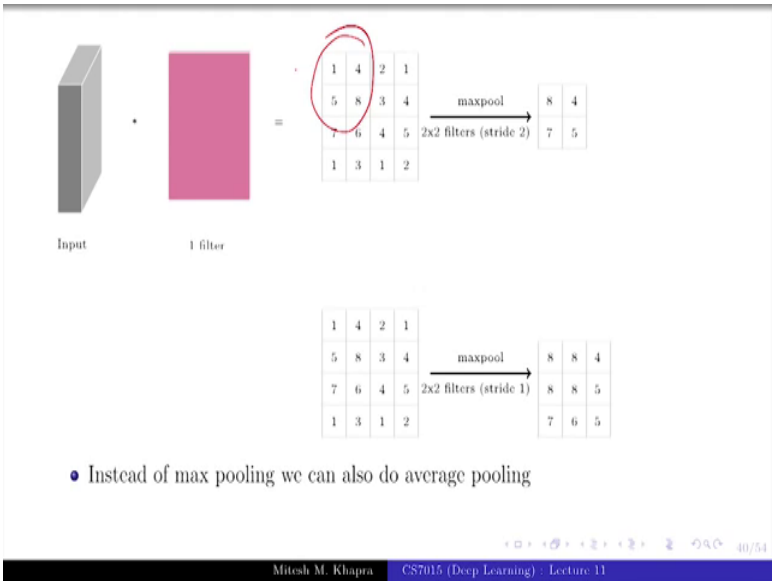
Because we are doing a 2D convolution, we are taking a 3D filter, but we are doing a 2D convolution , right? And then after that again this becomes my input and then do a max pooling on top of that and then something else happens after that. So, there as which we will come to later.

So, right now I just want to say that there is this input, then you come out will come with some output after applying convolutions, now this becomes the input for your next stage, where you have done pooling. Now the output of pooling becomes the input for the next stage. So, you will take this as the input, apply some convolution on that get some output and continue in this way right and we will come back to what these are.

(Refer Slide Time: 03:19)



(Refer Slide Time: 03:23)



So, now let us dive deeper into what is pooling? What does the pooling layer actually do? So, here is your input again, it is a volume and now when I say input, you should not just think of the input as this. Remember that, all of these can be inputs right. So now, at every stage once, you have got an output for the next stage that becomes input, that is typically how it was even in the feed forward neural network right, once you compute a hidden representation that is the input to the next layer.

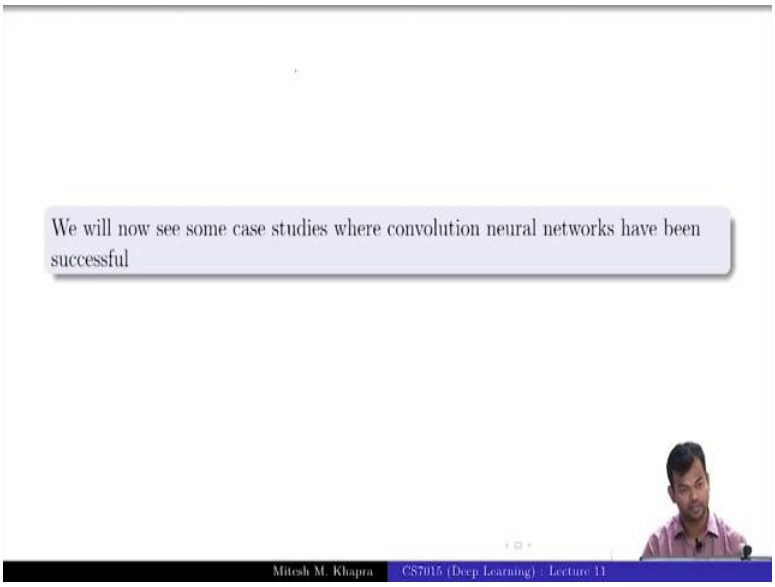
So, I have some input at one of these layers either, the input layer or any of the intermediate layers and I apply a filter on that and that filter gives me some 2D output, it gives me 1 feature map and let us say, this is what the feature map looks like.

Now, what does the pooling operation do? So, I would apply 2×2 pooling with a stride off 2. So, let us see what that will do? That means, I look at this 2×2 region, I will pick up the max value from there that is why, this is max pooling. Ok. So, the max value is 8, I will just keep that then I am going to do a stride of 2; that means, I am not going to place it on this block, I am just going to shift to the next block. Again, I will take the max from there, which is 4 right and I continue this and I get this is the output. So, you see why the shrinkage happens because, I am taking a 2×2 area and I am shrinking it by a picking up only one value from there right.

So, it actually kind of half the width and half the breadth so, total of 1 by 4 reduction is what you get. But you could also use a filter with stride off 1. So, this is what it would look like. So, you will place it here, take the max value then, the max value then, the max

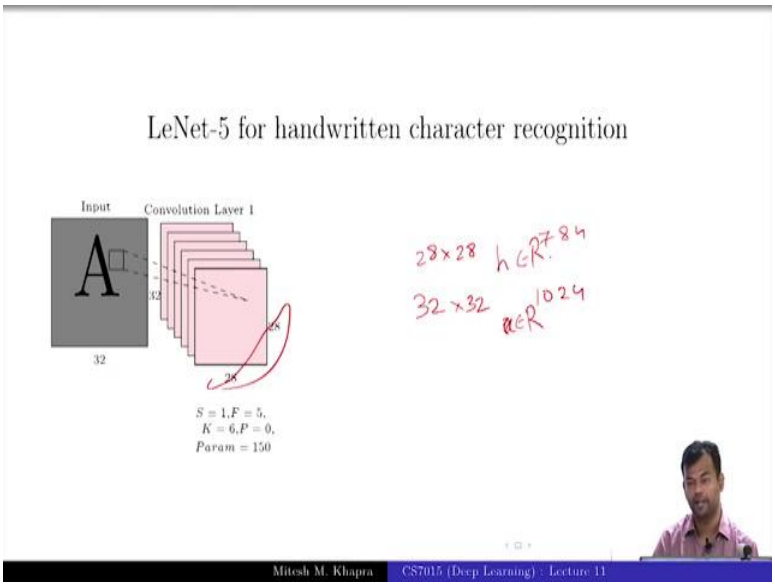
value and so on right. So, in that case you will get a lesser reduction and instead of max pooling, you could also do average pooling; that means, instead of taking the max of these 4, you could take the average of these 4, is it fine? So, is the pooling operation clear and how it results in the reduction of the size of your input? Ok.

(Refer Slide Time: 05:15)



So now, what we will do? Is now that we have some idea of what a full convolutional neural network looks like. So, it looks like alternating convolution and max pooling operations. We know what a convolution operation looks like, in particular we know that a 3D filter applied to a 3D input results in a 2D output, because we are not applying the convolution along the depth, we just applying the convolution along the width and the height right. So, that is what we know so far.

(Refer Slide Time: 05:47)



And based on this knowledge, now we are going to see some success stories of convolutional neural network right.

So, we will start with the first one, which is LeNet-5. So, this was already the fifth version. This was around 97 or 98 or something and I had mentioned this, when we were doing the history lecture.

So, this is the input, now you have decided to apply 6 filters, you have said that the stride is going to be 1, that means, you are going to place at every location. The spatial extent is going to be 5 and the padding is going to be 0. Now, the question that I have for you is, how many parameters does this convolution layer have? What are the parameters here?

Student: The weights.

The.

Student: (Refer Time: 06:25).

The filters, the weights in the filters, how many filters do have?

Student: 6.

6. How many ways does each filter have?

Student: 25.

20.

Student: 5.

25 right, 5 cross 5 is a 25. So, the total number of parameters is 150. Now, I want you to appreciate something here. So, the input was actually 32 cross 32, which I believe 1024 and the output was 28 cross 28 right, that is what the output you got that is, I guess 784 yeah. So, in a feed forward neural network, if you had x belonging to R^{1024} and your h belonging to R^{784}, how many parameters would you need? 1024×784, how many parameters do you have here?

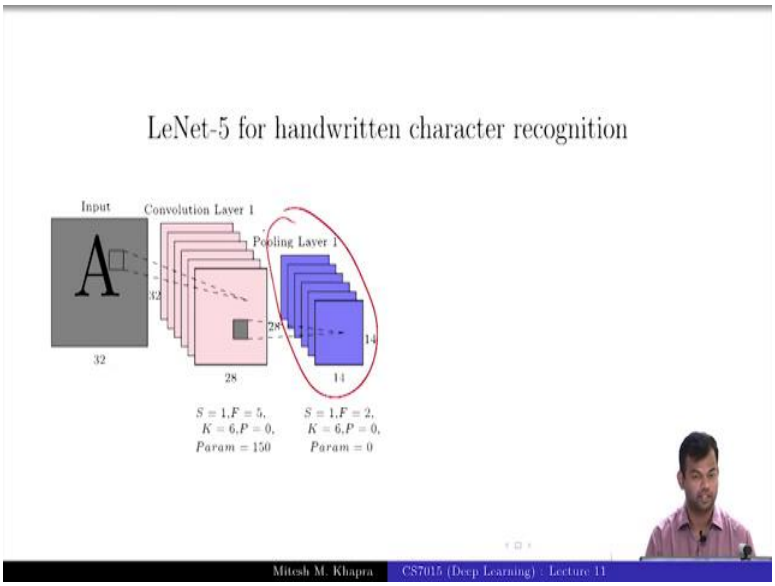
Student: 150.

150 much much smaller right, this is because of

Student: (Refer Time: 07:21).

Both sparse connectivity as well as weight sharing right, so now, you appreciate the difference between the two,ok. And thats one of the reasons that even, before this is for example, from 97, 98, 1997, 1998 right. So, even before the deep learning wave of 2006 or the revival after 2006 right, convolutional neural networks must still being trained for even deep networks 4 to 5 layers. Because they had much fewer parameters and that is why it was relatively easier to train them as compared to a very dense feed forward neural network.

(Refer Slide Time: 07:55)



So, I want you to appreciate that fact. Now, after this I have the pooling layer, when I am going to use a stride of 1 and F equal to 2; that means, I am going to pick up. Ok. So now, I am going to use a max pooling layer, where I am decided to use a stride of 1 and the max pooling will happen, in the region of 2 × 2 and again K 6, because there are just 6 filters. So, max pooling happens on every feature map independently, it does not happen across the depth; that means, I am not going to pick the max along these 6 layers, I am going to pick the max along each of these feature maps. So, it is a per feature map operation. Ok.

And this since it is a 2 × 2 filter, it will result in a size reduction and from 28 × 28, I will get to 14 × 14. How many parameters is the max pooling layer have?

Student: (Refer Time: 08:42).

Wow good, there is no parameters in the max pooling layer, because you are not having any weight matrices, just taking the input and applying a simple max operation on that, there is no w transpose x or any kind of a transformation happening there. Now, this becomes your input, what's the size of this volume?

Student: 32.

14 cross 14 cross.

Student: 6

6. So, all the filters that I am going to use from now on, what is the depth of those filters going to be?

Student: (Refer Time: 09:10).

What is the depth going to be?

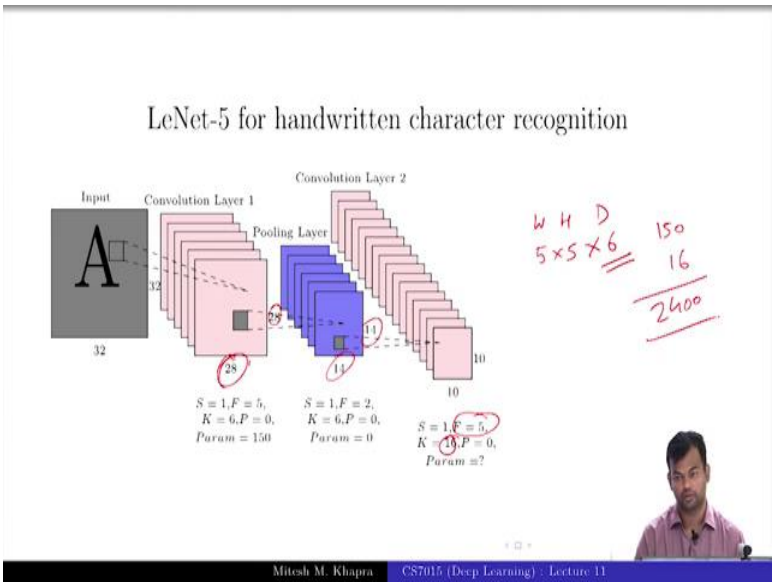
Student: 6.

I want to everyone to answer.

Student: 6.

6 right, because we are always going to assume that the depth is equal depth of the filter is equal to the depth of the input.

(Refer Slide Time: 09:19)



Now, here they decided to use 16 filters and by the way, you did hopefully notice that, this 28, how did you get it, from which formula?

Student: W n minus (Refer Time: 09:34).

Wn minus F plus 2P plus 1 right. So, that is the formula.

So, now we have a 14 cross 14 input and you have 16 filters. So, what is the depth of the output volume going to be?

Student: (Refer Time: 09:46).

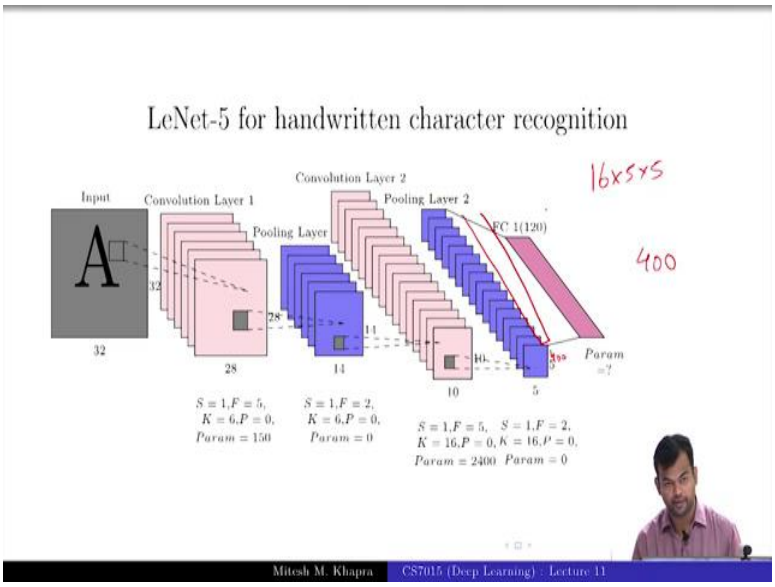
What is the depth of the output volume going to be?

Student: 16.

16, right? And a, ok so, you have 16 and you have a spatial extent of 5×5, just a minute spatial extent of 5×5, how many parameters does this layer have? I want everyone to say it.

Student: 400.

(Refer Slide Time: 10:11)



400 ok, fine so, that is.

Student: (Refer Time: 10:10).

There are 16 of these each is 5×5.

Student: (Refer Time: 10:13).

Oh into 6 into D ok ok good, then we made a mistake here also, no there the depth was 1 are you, do you get it how you got 2400 right, we forgot about the depth. So, each of

these filters is 5 × 5 × depth right, what is the depth? 6 the same as the input, right. So, each of these filters is 25 × 6, which is 150 × 16 yeah fine, is that ok? Did I confuse you or everyone back on track? Pooling layer, edge should have been 2, because of that half reduction using.

Student: (Refer Time: 10:49).

Yeah maybe, can we just check this? I think it should be 2. Ok, yeah there was a question.

Student: (Refer Time: 11:00).

Student: (Refer Time: 11:02).

Go from the pooling layer to the next layer from here to the next layer. Ok.

So now, what is the depth of your input volume? 6 and what is the width and height? 14 cross 14. So now, every filter that I going to apply at the next layer is going to have a depth of 6. So, I have decided to apply 16 such filters. So, what is the depth of this layer going to be? The depth of the output is equal to the number of filters. So, the depth is going to be 16 and all my filters are 5 × 5, but what we forgot is that, when I say the filter is 5 × 5, it is actually 5 × 5 × 6, because the depth is also there right. So, it is width into height into depth. So, that is this number of parameters in each of my filters right and that is 150 and I have 16 such filters. So, that gives me a total of 2400 parameters, is it fine? Ok? Ok.

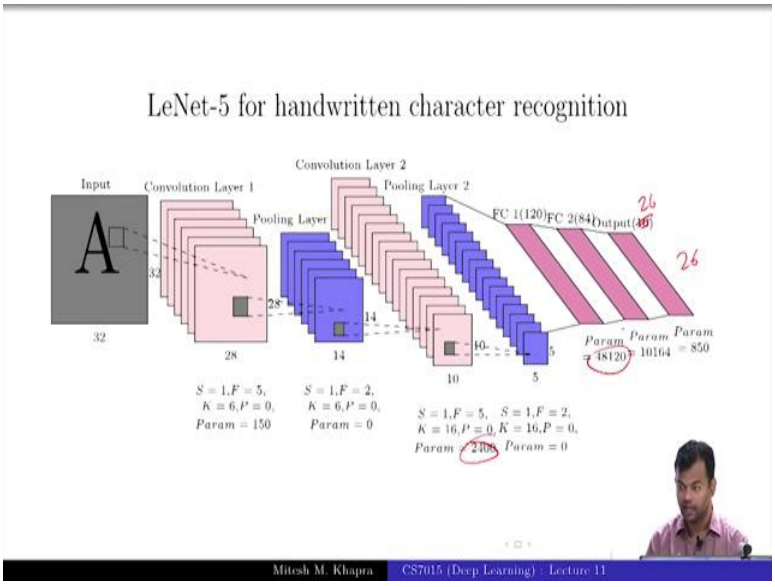
So now, we have a volume of size 16 cross 10 cross 10, now I am going to do max pooling on that maybe, again this should be 2, there was the same doubt you had fine. So, it will result in a reduction in the output and now what is the volume? What is the size of this volume? 5 × 5 × 16 and the parameters is 0, max pooling layer does not have any pooling.

Now, after this what we have is something known as the fully connected layer. Ok? So, now, as I said the size of this volume is 16 × 5 × 5. Its arranged in these feature maps, but I can always flatten it to get one single vector, do you get that? So, from these 16 feature maps each of 5 × 5 size, I can flatten it out and get the single vector of size 400, do you get that. Ok. So, thats what, I do in the fully connected layer.

So now, I am going to flatten this treated as a single vector and then fully connect it to the next layer, what do I mean by fully connected? Dense connections, no more sparse connection. So now, we have a feed forward network from this point of view. So, you have 400 and that connects to a layer of size 120. So what are the number of parameters?

Student: 400 into 120.

(Refer Slide Time: 13:13)



400 into 120, plus we will have 120 biases. So, that is what this number is, fine? So, this is one fully connected layer of size 120, after that I have another fully connected layer of size 84. So, the number of parameters would be 120 into 84 plus 84 and after that I have the output layer. So, the output layer this was 26 or whats the output?

Student: (Refer Time: 13:31).

Oh but the, this is digit, this is alphabet recognition right. Ok. So, probably they have done the computation using 10, but it should have been using 26 as the output layer, because you want to predict one of the 26 alphabets. So, you can assume this is 26. So, it would be $84 \times 26 + 26$ right that is the size of the output layer right.

Now, do you observe something immediately is a something very striking immediately, in terms of the number of parameters?

Student: (Refer Time: 14:00).

The fully connected layers clearly dominated right, here, we were dealing of order 2400 and max and here we just start with 48000 itself right. So, just keep this in mind that the fully connected layers have the largest number of parameters, that you have. And we will try to come back to this and see if we can solve this problem. Ok?

So now, when you see a convolutional neural network, you should be able to reason about the following things. At each layer, what is the size of the input volume? What is the size of the output volume? What are the number of filters being used? And what are the number of parameters in that layer? Right, if you can reason these things and you have really understood, what is actually happened and unless you can reason these things, I do not see how you can efficiently code it up right. So, you should be able to know that, this is the size of the input, this is the size of the output and so on and I guess all of us are comfortable with others.

(Refer Slide Time: 14:51)

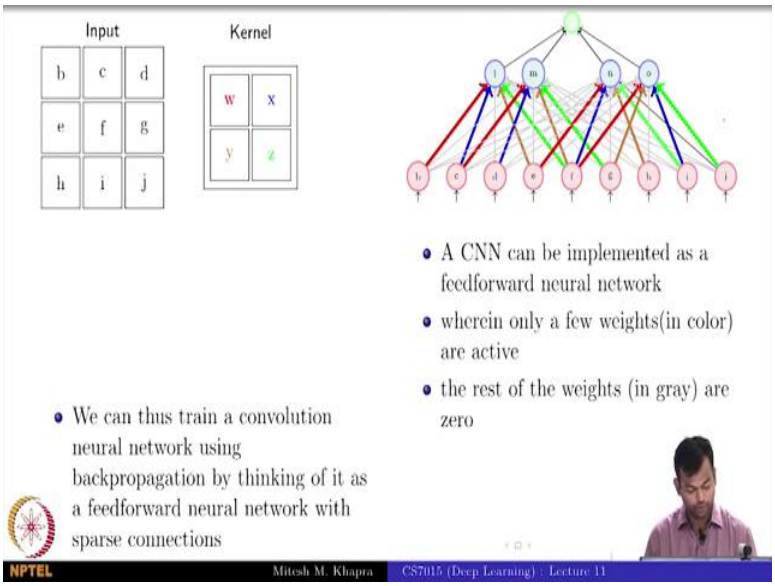


So now, how do we train a convolutional neural network? What is the answer? your nodding; that means, you do not know or you know it is too trivial to even ask this question, how will you train it?

Student: (Refer Time: 15:00).

But how do you back propagate through a convolution operation? That is a very nasty looking operation.

(Refer Slide Time: 15:07)



A CNN can be implemented as a feed forward neural network, with what being the difference?

Student: (Refer Time: 15:15).

Only some of these weights would be active, all the gray weights will not exist only the colored weights will exist right. Now can you back propagate to this network have you seen something similar before?

Student: Dropout.

Dropout right so, if you could do that you can do this also. So now, if you take this view of a convolutional neural network where so, this is just to give you an intuition or make you feel confident, that once you know the backpropagation algorithm, there is no nothing much different from training a convolutional neural network operation. Ok?

So, everyone is fine with that, how many of you agree with that? That you can actually train using the same algorithm with some smart coding required to make sure that these weights are not active and so on. But in practice of course, you will not do this. In practice, you will define the convolution operation, you will also define the derivative of the convolution operation and then use that right, because that would be much more efficient, this is very inefficient right because, you are assuming that there is a fully

connected network and then some things do not exist there. So, that is not thats the whole point, I mean you wanted to avoid such dense connections right.

But in principle you could have just used this and trained the convolutional neural network. In practice, you do not need to worry, because you just need to define your forward convolution operations and people like Google and all who release tensor flow, torch and all will do the hard work of doing the back propagation for you right. So, you never have to write back propagation in your life, apart from what you have already written in the assignment right that is why I make you go through that torture once.

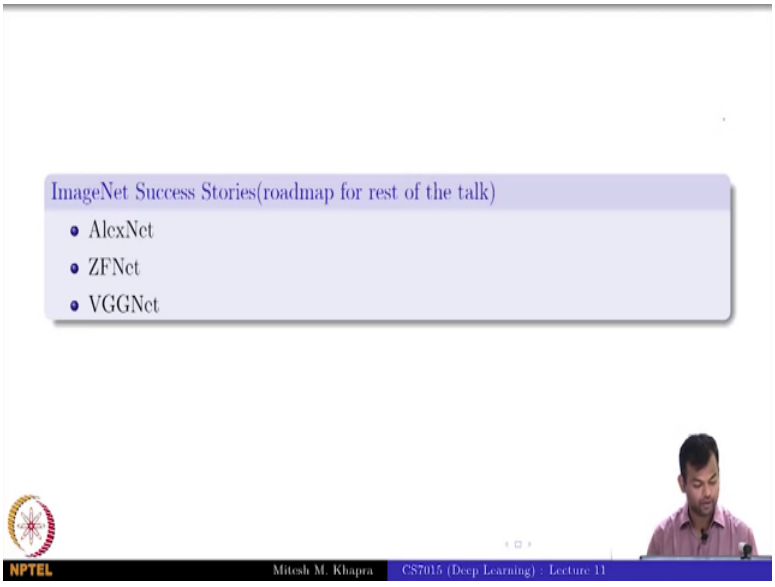
So now afterwards afterwards whenever you use any of these platforms or pytorch or torch or tensor flow, the back propagation comes for free, you just need to write the forward progression; that means, you just need to write convolution operations and you dont need to worry about how the derivatives will be computed. But I what I want you to understand is that conceptually, it is the same, you can still use or in fact, you still use the same back propagation algorithm, to train a convolutional neural network also, everyone is fine with this? Pk? Ok. So now, we know how to trained a convolutional neural network also.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 11
CNNs (success stories on ImageNet)

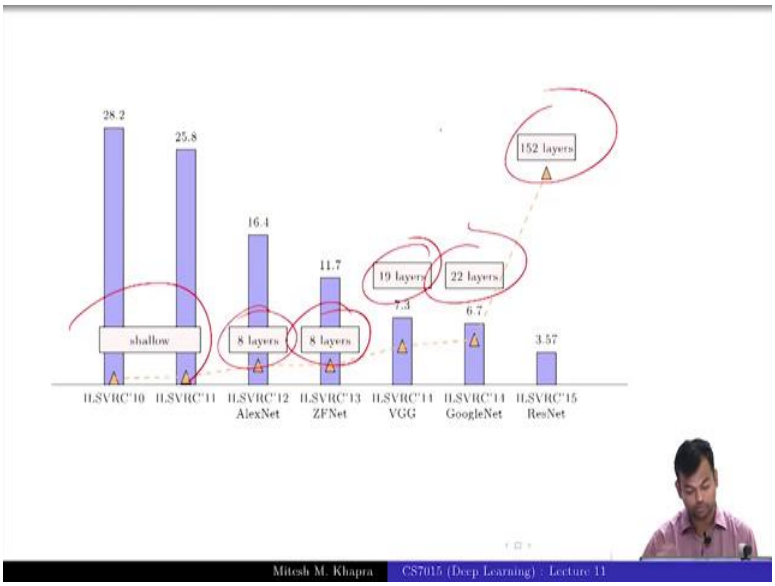
So, now, we will go to the next module, we will talk about some success stories on ImageNet right. So, this is the challenge which actually made convolutional neural networks very famous back in 2012.

(Refer Slide Time: 00:23)



So, they are going to look at some algorithms in fact, 2 more hopefully today.

(Refer Slide Time: 00:29)



So, this is the story right. So, there is this challenge or competition called ImageNet Large Scale Visual Recognition Competition right that is what ILSVRC stands for. And this was a data set created which had 1000 categories, it actually has 10000 categories, but in the competition we use only thousand of those categories yes.

So, 1000 plus 1000 I think, the roughly the data set size is 1 million. And so, that is what was used for training a classifier. And I am talking about 2010, the pre deep era right I mean so, of course, deep era networks existed at that time, but the participants and these challenges and that time were relying on the classical machine learning approaches. So, what was that approach? Take the image.

Student: (Refer Time: 01:16) feature.

Extract features, which features?

Student: (Refer Time: 01:20).

Predominantly.

Student: Sift and hawk.

Sift and hawk features were the predominant features at that time and then you train a classifier on top of that. And then, you use things like on symbols or better handcrafted features and things like that certain more tricks on top of that. So, that, with that on this

data in 2010 the error was 28.2 percent; that means, if I give you a test set of 1000 images you will make 282 errors on that right, that is what this means. Then, in 11 there was still some progress, this was again pre deep era and there was this error came down to 25.8. And then, in 2012 there was this AlexNet, which was a deep convolutional neural network applied to the task of image classification and it gave a dramatic reduction right from 25.8 to 16.4.

And was I think absolute; in absolute terms 8 to 9, 8 to 9 percentage better than the best system in that competition that year, ok, so that was in 2012. In 2013, there was further improvement on a different architecture for doing this and that give a further error reduction of 11.7. Then in 2014 there was VGG net. So, these are all 3 that we are going to see today, which give a further error reduction of 7.3. Then, Google decided to join the party and they make it 6.7 and as I have said before then afterwards Microsoft got crazy and they brought it out in 3.57 and this is when we started making claims that a convolutional neural network has become better at this task than humans right.

Because, if you show these 1000 images to a human, even a human is bound to make a 3.5 percent more than 3.5 percent error; that means, because some of these images would be blurred. So, I would not be very sure whether, this is a bulldog or a different type of dog or something like that right. So, I even a human cannot really recognize it correctly and that is the whole hype around, how convolutional neural networks have beating human level performance on this particular task right.

And let us see so, this was all the shallow pre deep era. The first architecture was 8 layers and I think this was called a varied no this probably not this yeah. The second architecture was also 8 layers, then, we had 19 then, 22 and then 152 right. Thats how the progress has happened. So, these are all the architectures that we are going to look at today or at least some of them today and the rest maybe tomorrow.

Ok. So, we will start with AlexNet and I am going to tell you the exact architecture of AlexNet, what was refused, what did it actually use.

(Refer Slide Time: 04:02)



So, the input was an RGB image. So, it had a depth of 3 and it was 227 × 227 that is what the data set input was, all the images in the data set were to 227 × 227 cross 3. So, the first thing that they did was, they decided to use 96 filters, can you read that? Anyways I will say it outright.

So, they resided to use 96 filters with a spatial extent of 11 cross 11, a size of 4, and padding of 0. Ok? So, the moment you see size, a stride of 4, what do you know is going to happen? There is going to be some shrinkage, roughly by how much? One-fourth, Right? So, now, can you compute these 3 things, what was W2, what was H2 and what was the number of parameters in this layer? We will do it for a few of these layers and then I will just rush through that. So, whats W2 going to be? You have already done this computation right, the exercise that we did was exactly this computation. So, there was 55 cross 55 and what is the depth going to be? I want everyone to say that.

Student: (Refer Time: 05:15).

Ok. And what is the number of parameters.

Student: (Refer Time: 05:20).

96 into.

Student: 11.

11 into.

Student: 11.

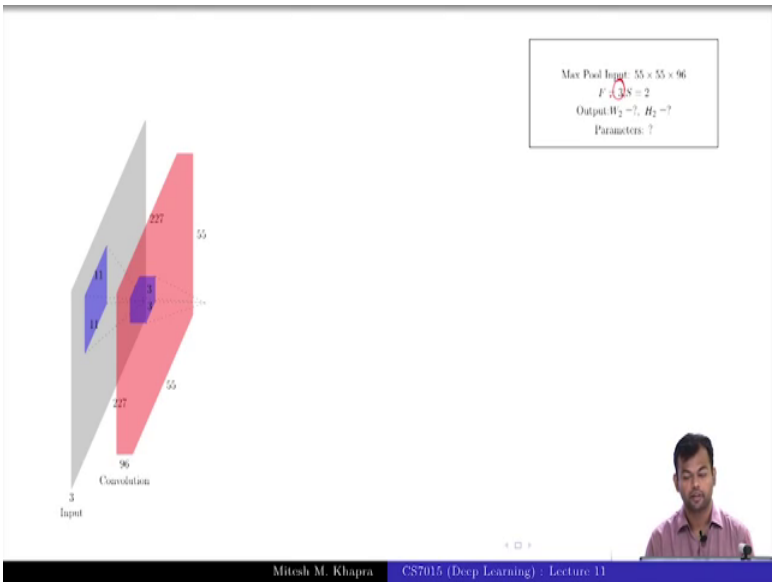
11 into 3. Don't forget the depth, the depth is 3 here.

(Refer Slide Time: 05:31)



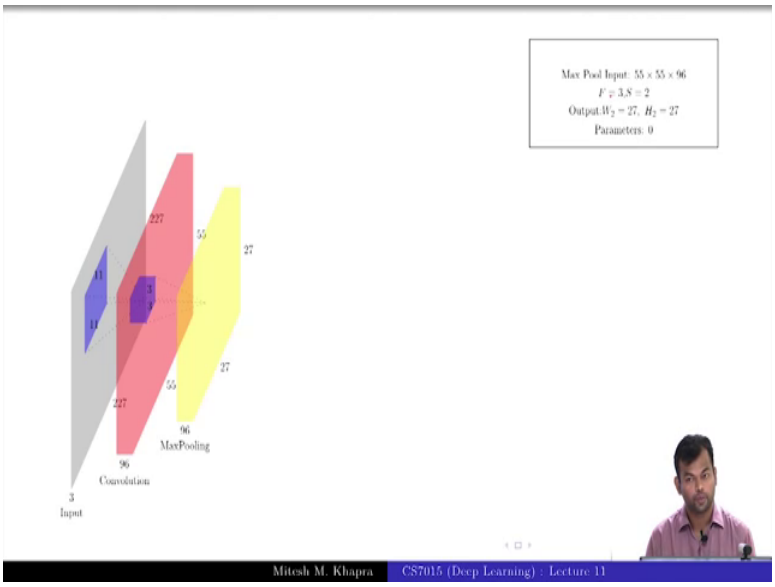
So, that is the number of parameters that they had in this layer. Ok, 11 into 11 into 3 into 96. Now, what is the next layer going to be? A max pooling layer.

(Refer Slide Time: 05:42)



Ok, so they had a max pooling layer, where they used a 3 cross 3 max pooling; that means, you are going to pick up max from a 3 cross 3 grid. And the stride was 2; that means we are going to get half the output. And now can you tell me, what W2 H2 would be roughly? Half of 55 55 right so, 27 27.

(Refer Slide Time: 06:02)



And what is the number of parameter is going to be? Don't be lazy, everyone be say it.

Student: 0.

0 right so, that is the max pooling layer.

Now, what is the size of your input volume at this point?

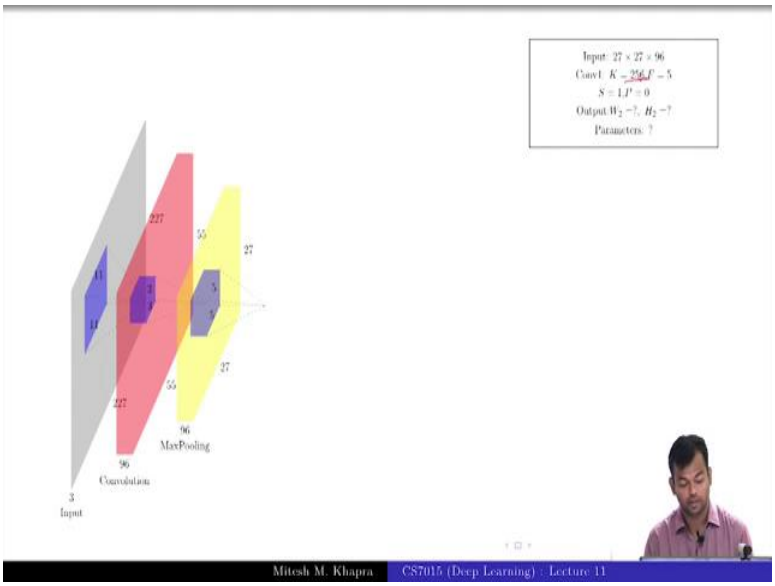
Student: 27.

27 cross 27 cross.

Student: 96.

96 as opposed to the original input which was 227 cross 227 cross 3, so as you keep progressing, your width and height is decreasing, but your depth is increasing because, you are using more and more filters to capture more and more patterns in the images.

(Refer Slide Time: 06:37)



Now, so you have 27 × 27 × 96, then they decided to use 256 filters, each of size 5 × 5 with a stride of 1 and padding of 0, ok, is it right? So how many parameters do you have now?

Student: (Refer Time: 06:53).

256 into.

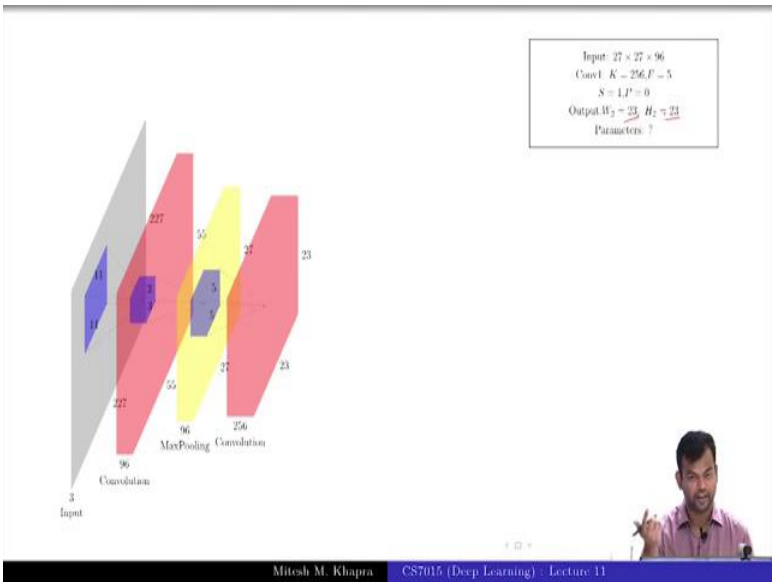
Student: 5 into 5.

5 into 5 into.

Student: 96.

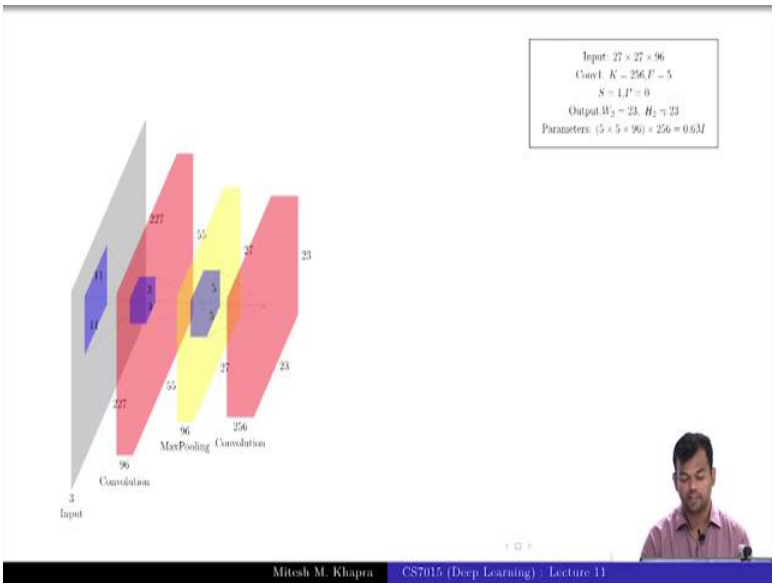
96.

(Refer Slide Time: 07:00)



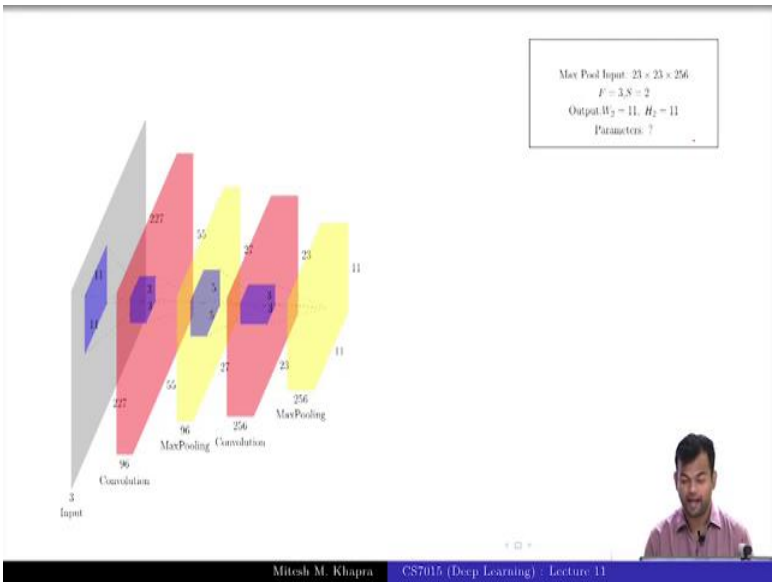
So, thats the number of parameters that you will have and the size since would decrease only by 1 right because, you have a stride of, it will decrease by 2, because a filter size is 5 and you have a stride of 5. Ok.

(Refer Slide Time: 07:12)



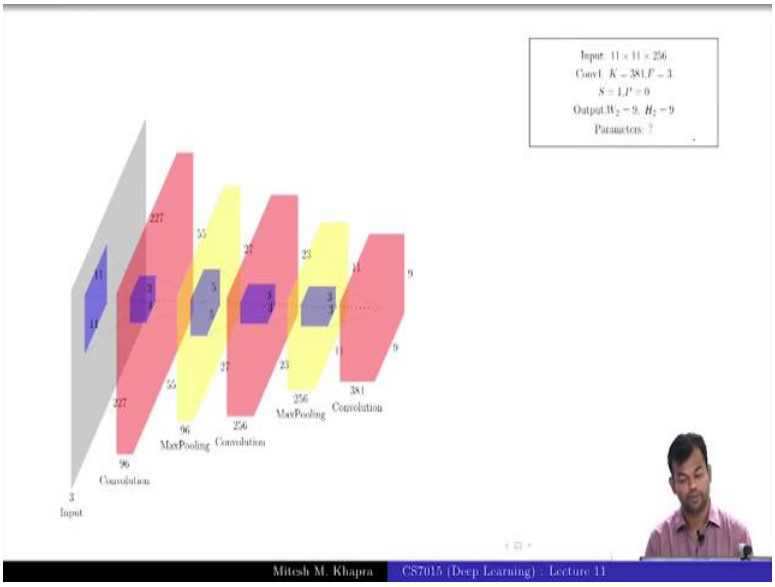
So, these are the number of parameters, we had 0.6 million parameters in this layer. What is the next layer going to be? Pooling.

(Refer Slide Time: 07:20)



So, you do a max pooling, again you do a 3 × 3, you do a stride of 2, so your width in height is going to decrease, the depth does not change. Remember in max pooling, the depth does not change because the max pooling operation is per feature map, it is not across the depth fine. Then use a 3 × 3 filter and 384 of those.

(Refer Slide Time: 07:40)



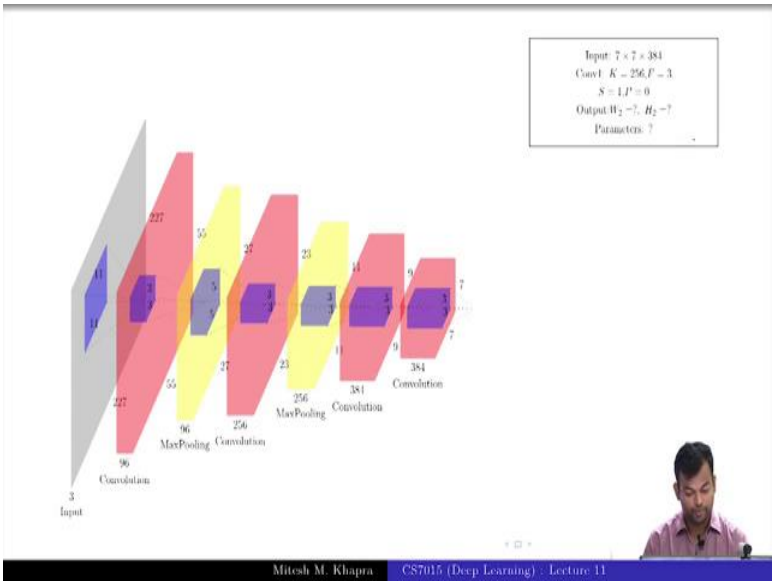
So, how many parameters would you have?

Student: (Refer Time: 07:43).

384 into 3 cross 3 into 256, the depth.

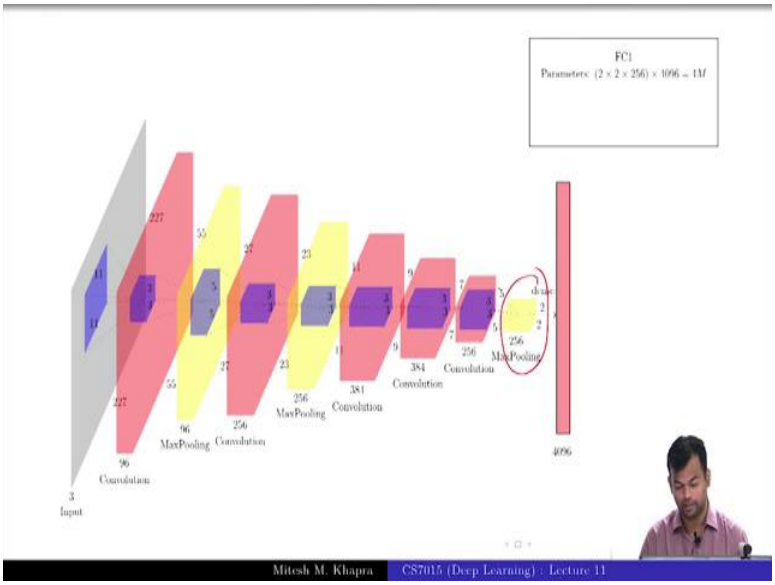
So, now you guys get it. So, I will not bore you anymore.

(Refer Slide Time: 07:51)



And then you have a convolution operation again, which is a 384 convolutions each of size 3 × 3 and so many parameters, followed by a convolution operation again, followed by a max pooling operation, then, followed by a fully connected layer.

(Refer Slide Time: 08:05)

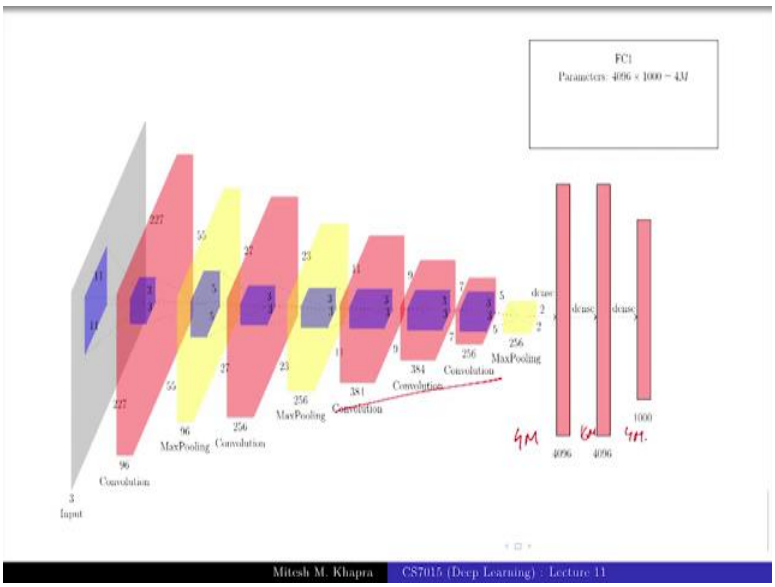


So, what would I do to this 256 × 2 × 2? I will fatten it. So, I will get what dimensional output?

Student: 1024.

1024. 256 into 2 into 2, so this 1024 dimensional vector I am going to fully connect it to a 4096 dimensional vector. How many parameters? 4 million right, 4 into 10 raise to 6 right so, roughly 4 million.

(Refer Slide Time: 08:30)



Then you have another 4 million, another 4096 vector fully connected, how many parameters?

Student: 16 million.

16 million. Then you have the 1000 classes that you are interested in. Right? So again fully connected. So, you get the full architecture. Anyone has any questions? No one wants to know why this particular configuration among all the possible configurations? Why not 10 layers? Why not first 8 cross 8 filters? Why not 9 cross 9 filters? Unfortunately, no one knows. [laughing]

Student: (Refer Time: 08:59)

So, this I mean see this what this is what would happen right. Now we get into something known as hyper parameter tuning right. So, what are the hyper parameters in this

network? The kernel size is and the number of filters right? So, you would have tried a lot of these things, evaluated on the validation set, seen which one gives the best accuracy and then chosen right. So, that is probably what would have happened, but there is not enough insight into how this particular architecture came up.

Apart from some things right that 3 curves 3 neighborhood sounds reasonable. Initially, when you have the full image, you use larger filter sizes because, you want to capture a lot of things there but, once the image has shrunken you use smaller filter sizes. So, those are some rational decisions which look reasonable. But why these 3 convolutional filter layers back to back instead of convolution max pooling convolution max pooling and so on right.

So, the some of those things are not clear so, just in case you are wondering, do not wonder, this is just the architecture, this is known as modestly named as AlexNet. So, that is [laughing] yeah and so I said that this has 8 layers? But you clearly see more than 8 layers here. So, why did I say that has 8 layers? Which are the layers we are not counting?

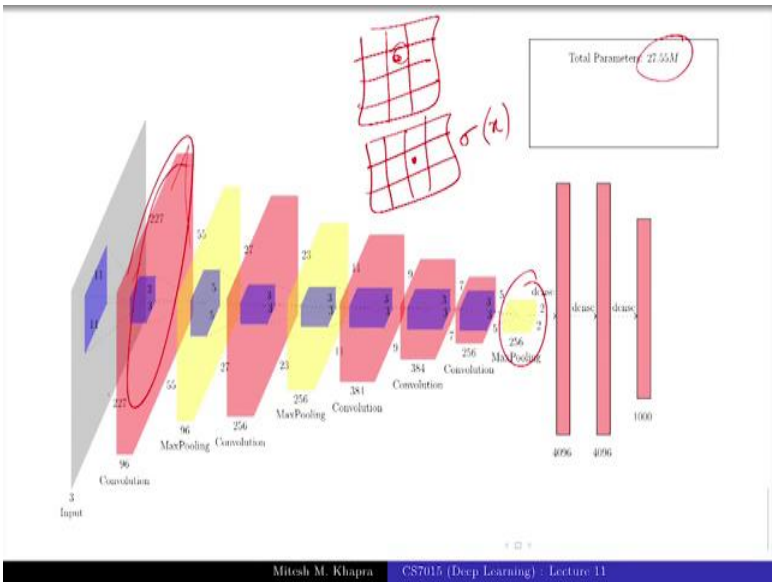
Student: (Refer Time: 10:10).

Why?

Student: (Refer Time: 10:12).

Because, they have no parameter right, so when you count the number of layers you only count those layers which have parameters. So, you have 5 convolutions and 3 fully connected layers.

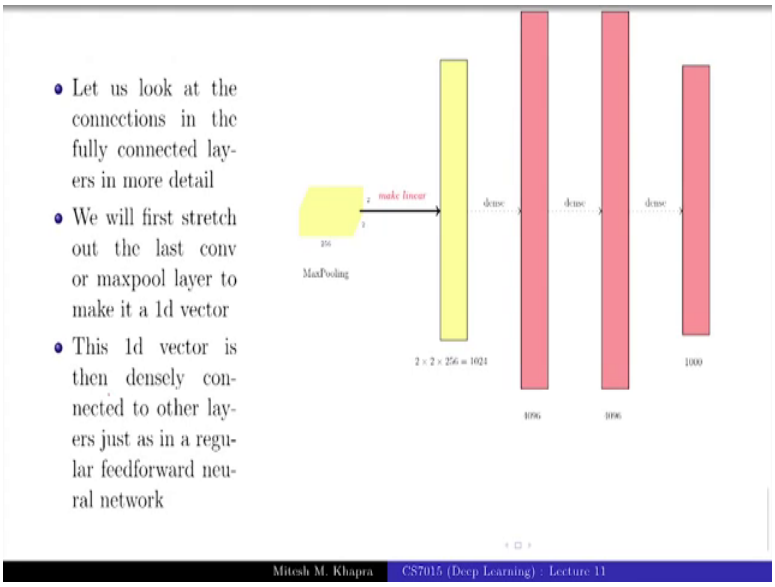
(Refer Slide Time: 10:21)



Then, so the total number of parameters in this network is 27.55 million parameters and, ok at this point I will and; obviously, you notice that most of these parameters were there in the fully connected layer. So, you had 4 million here, then 16 million here and then again 4 million here right.

So, roughly 24 million of the 27 million parameters were there in the fully connected layer, you see that skew in the number of parameters. Ok.

(Refer Slide Time: 10:50)



And I will just look at the fully connected layer again. So, the last max pooling layer actually gave you a 256 cross 2 cross 2 output. You just flatten it to get a 1024 dimensional vector and then you connected fully to the 4096 vector right. So, that is what I mean by a fully connected layer. Why do you move max fully?

So, the reason for that is basically, to shrink the size of the image right because, after that if, if you keep working with this size right then, the number of parameters is going to really blow up, a by using a larger stripe yeah both of them are feasible right. So, now, see from here remember that we had the original image sizes 227 cross 227 and by the end we were just left with 2 cross 2.

And then adding a fully connected layer on that makes sense right. If I had not done this shrinkage throughout either by increasing the stride of the convolution layer or by doing max pooling right then, you would have left with something of the order of 200 cross 200 here and then, you have to do a fully connected on top of that is just infeasible right. It just throws away all the hard work that you have done by doing weight sharing and sparse connectivity right. So, that is not feasible.

There are also papers with say, which I think it is titled fully convolutional neural network which does not have any max pooling layers and they show that that also works fine. In fact, when we see VGG net, we will see that it has back to back convolution layers and very few max fully layer right. So, these are all things which people have trained.

Not so many years, 2 years the challenge came out in 2010 and in 2012 this was used right. So, it is like not really a large gap right. And if, you read the original paper they had to do a lot of tricks to actually make this work, it was not as simple as I am showing it. Of course, now with all the stability which comes from these platforms, tensor flow pytorch, you can probably just go and implement this as it is and you should be able to reproduce the results. But six years back that was not the case. There was a lot of hard work involved in getting this too work. And they this was also the paper, which introduced the reLU non-linearity in the context of convolutional neural networks right. So, they had to change from sigmoid or tan edge to reLU.

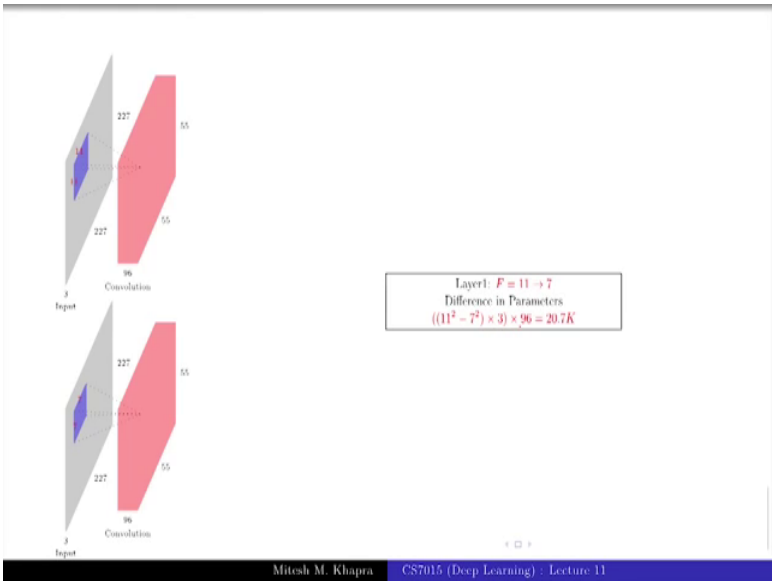
A lot of these small small things which they had done And at that time it is also not possible with the existing hardware to train this on the given GPUs that you had at that

time. So, they had to do some splitting across GPUs and so on. So, it was not as simple as it is today with all the hardware, as well as API developments or platform developments around this right. So, probably that is why it took 2 years to yeah sure.

So, each of these things, so after you do the convolution operation, you pass it through the reLU non-linearity. Ok so what does that mean is that? The convolution operation gave you a feature map, every entry here was just a weighted average of the neighbors right, you take this entry or rather you take this feature map and create a new feature map where every entry here is the sigmoid of every entry here. Do you get that or not sorry sigmoid, some non-linearity and they use the reLU has the non-linearity. So, you do get everyone gets this? So, all the convolution layers are followed by a reLU non-linearity layer.

So, you get this volume, pass it through the reLU and get a new volume, but, I have just shown that as a single operation. It is before pulling so this was the fully connected layer. So, now, we look at the next architecture which is ZFNet.

(Refer Slide Time: 14:05)



Now, I am going to compare ZFNet with AlexNet. So, on the top you will see AlexNet, on the bottom you will see ZFNet ok. So, again the input was the same, $227 \times 227 \times 3$. Now instead of 11 cross 11 filters, ZFNet decided to use 7×7 filters. And their rationale was that you do not need such large neighborhoods, you do not need as small as 3×3,

but probably you need at least as much as 7×7, you do not need 11×11. So, that is the first change that they did and that would also result in some parameter pruning right.

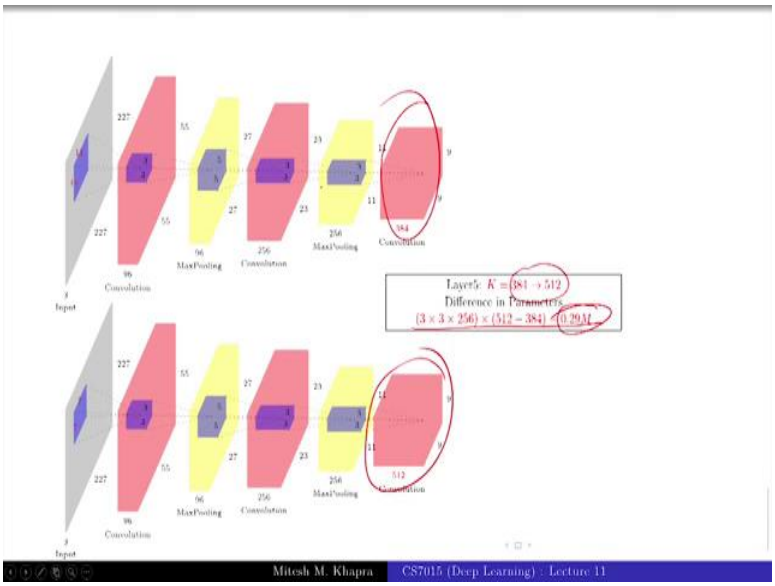
Because, the number of parameters now would be 7 cross 7 into 3. So, the difference in the number of parameters at this layer for ZFNet which is at the bottom and AlexNet which is at the top, would be this. How many of you get this? Ok. So, thats in the difference in the number of parameters. So now the output volume still remains the same. its $55 \times 55 \times 96$.

(Refer Slide Time: 15:03)



Then again they had the same max pooling operation, this layer there was no difference between ZFNet and AlexNet. And then after that you had again layer 3 which was exactly the same as AlexNet.

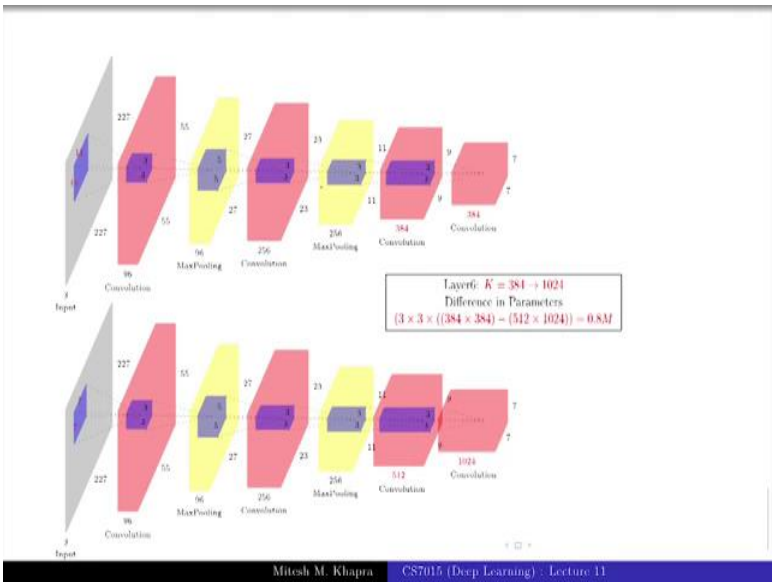
(Refer Slide Time: 15:20)



Then layer 4 again the same as ZFNet. Afterwards layer 5 instead of 384 filters, they decided to use 512 filters, the rest of the thing remains the same; that means, the size or the spatial extent of the filter remains the same. That again results in some difference in the parameters. So, thats the number of parameters that got added in ZFNet as opposed to Alex net.

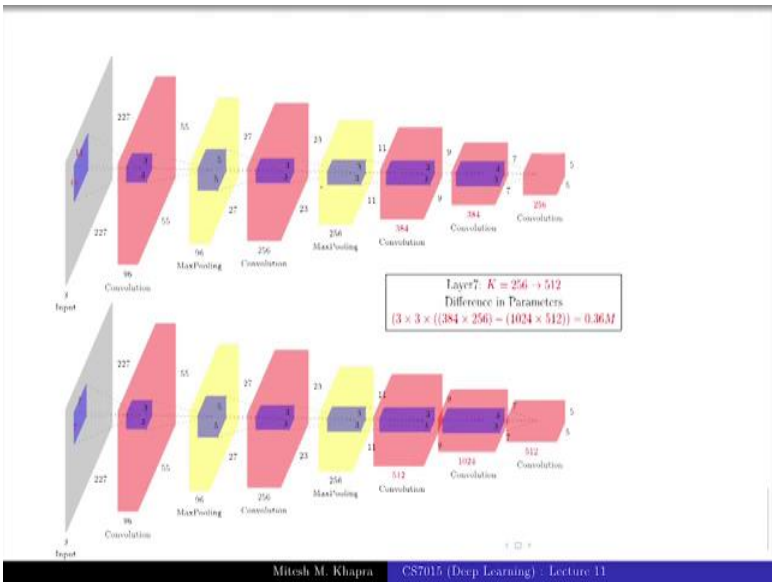
And of course the , oh sorry sorry, oh sorry, the bottom one is a ZFNet yeah that is correct sorry. So, in ZFNet you had 512 filters as opposed to 384 filters in Alex Net. Ok? Is it fine?

(Refer Slide Time: 15:52)



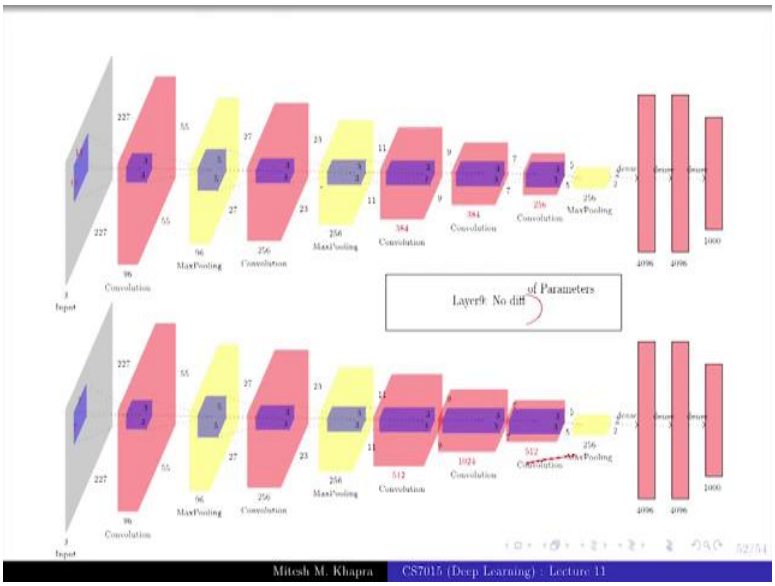
And then the next layer again instead of 384 filters, they had 1024 filters.

(Refer Slide Time: 15:57)



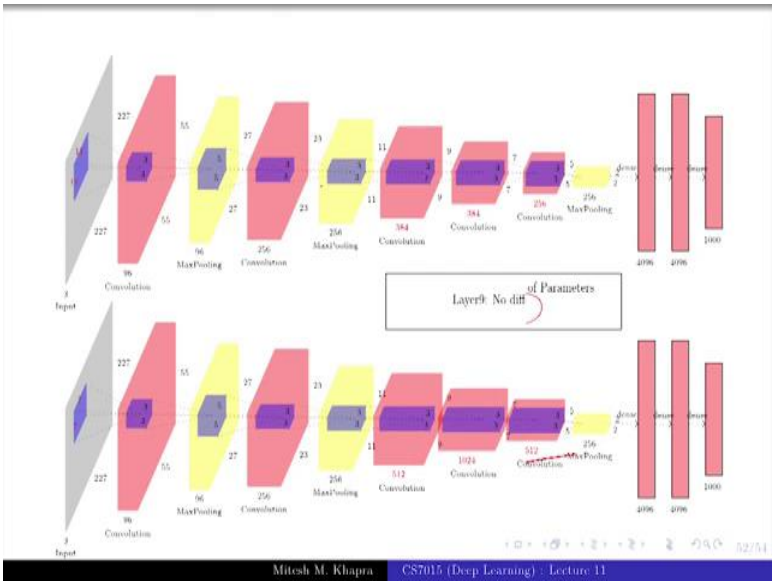
Then, again instead of 256, they had 512 filters and then, a max pooling layer then the same dense fully connected layers. Ok?

(Refer Slide Time: 16:02)



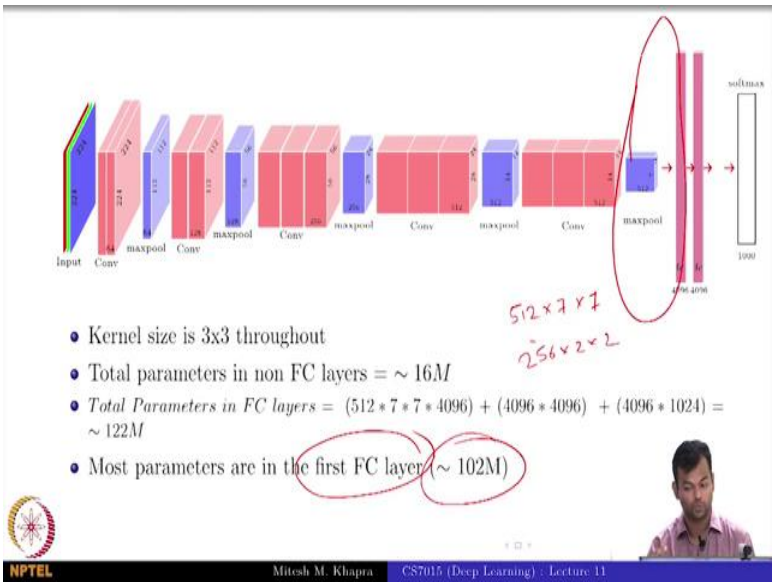
So, everyone gets this? This is the difference between the two architectures and this led to that difference in the error of around 3 to 4 percent is that we have seen earlier.

(Refer Slide Time: 16:15)



So, difference the total number of parameters was 1.45 million and of course, ZFNet had more parameters because, is that, it has these more filters in the deeper layers ok. So, we go to the last point which is may be more in that VGG net.

(Refer Slide Time: 16:26)



So, again in the case of VGG Net, the input was, ok, so I just want to I will not see it (Refer Time: 16:41) in. So, the input was again the same, it was RGB cross 227 cross 227.

So, this is what the VGG architecture looks like. They have so in VGG network throughout ok, wait. So, how many layers this ZFNet have, 8. So, you only count the pink boxes because, the those has ones which have two parameters. Now, VGG net has slightly more number of layers but, in all the convolution layers they use 3 cross 3 filters. Right, from the beginning they use 3 cross 3 filters. Ok. So, you have the first convolution layer, then another convolution layer, another convolution, another max pooling layer followed by 2 convolution layers, then a max pooling layer, followed by 3 convolution layers, max pooling just keep adding box is writing just because, you can and then, you have the fully connected layers.

So, again there is not much intuition for why 16, in fact, later on someone came if this is the VGG 16 architecture because, it says 16 layers. Later on some of someone came up with the VGG 19 architecture, which has 19 layers right. So, a lot of this is data center even right, so you try your best to get the best possible accuracy on the ImageNet data and that is the architecture you came up with right.

But as long as how many of few feel comfortable with what is happening right and I mean, when I say comfortable, I mean that, you really understand the gory details of

what is happening at each layer in terms of input volumes, output volumes, number of parameters, how are you going to train this network end to end. Can you see how are you going to train this. So, you will get some loss here that is going to propagate all the way back to the first layer, right? And this propagation is going to happen over some sparse connections that fine. Now, this is one very important point that I have skipped and which none of you is questioning. Is everything that is happening here differentiable?

Student: (Refer Time: 18:38).

What happens to max pooling? Is max pooling or differentiable operation? So, I am going to ask you this, how are you just note this down, how are you going to back propagate to the max pooling layer because, you need to see whether the max pooling layer is actually a differentiable layer or not . So, here I just some statistics about VGG net; everyone is writing that down [laughing] this perhaps, means I will not ask it

The kernel size is 3 cross 3 throughout; the total number of parameters in non fully connected layers is 16 million. The total number of parameters in fully connected layers is 122 million. So, you see that this fully connected layer is really a problem it like really hogs all the lime that it has the maximum number of parameters there right. And so and the most number of parameters are there in the first fully connected layer, because you have this $512 \times 7 \times 7$, you remember then Alex Net and ZFNet, the last layer was $256 \times 2 \times 2$, which has definitely more manageable than this layer which has grown almost 8 times in size, but not even 8 actually 4 into 4 into 2 right, 16 32 times in size right.

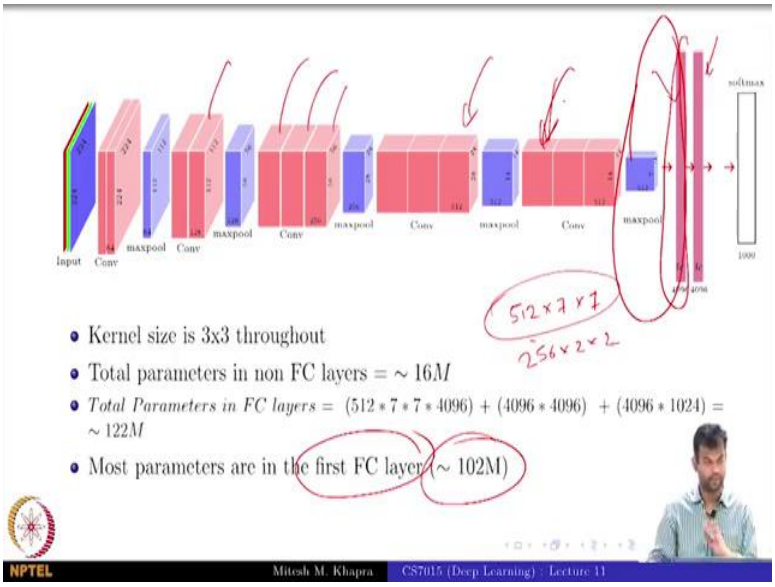
So, that is really blown up the number of parameters in the first fully connected layer. So, you just imagine the I mean, you have such a deep layer and then you realize that all the main number of parameters are there in this one particular layer, everything else is much fewer parameters or orders of parameters less number of parameter is less then, this one fully connected layer

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 91
CNNs (success stories on ImageNet) (Contd.)

This is where we left off in the last class so we look at three networks for image classification starting; with Alex net, then ZF Net, and then VGG Net. VGG net in particular had 16 layers including convolutions and fully connected layers. And one thing that we saw that a large number of parameters are there in the first fully connected layer.

(Refer Slide Time: 00:36)



Because you are connecting a 512 × 7 + 7 volume to a 4096 dimensional vector right so, that is one thing. The other thing that I would like to kind of mention right now so, that it becomes useful for the later part of the lecture is that. If I look at any of these pink boxes here right or even these things which are known as the fully connected layers. And if I just flatten them out and view them as a vector? what does that vector actually capture? It captures a, it captures an abstract representation of the image right.

So, now imagine what would happen is, suppose you have trained one of these networks; Alex Net, VGG Net or any of your favorite networks. And by what I mean by training is that you have been tracking the cross entropy laws and you have run it for several epochs

with some patience and so on. And I was satisfied with whatever training error you are getting and you have stopped training now right.

Now, after this if I pass images through these net through this network and I take the representation from any of these boxes or from the fully connected layer; what is it that I have essentially got now? I have got an abstract representation of the image that I have been feeding it right. So, just remember this and this is something that we will use.

So, this is very common to do. So, you have a trained image net many people have released different models for image net the ones which we have covered being included them. And now for him any image task if you want to do some processing then it's common to take the strain network pass your image through that.

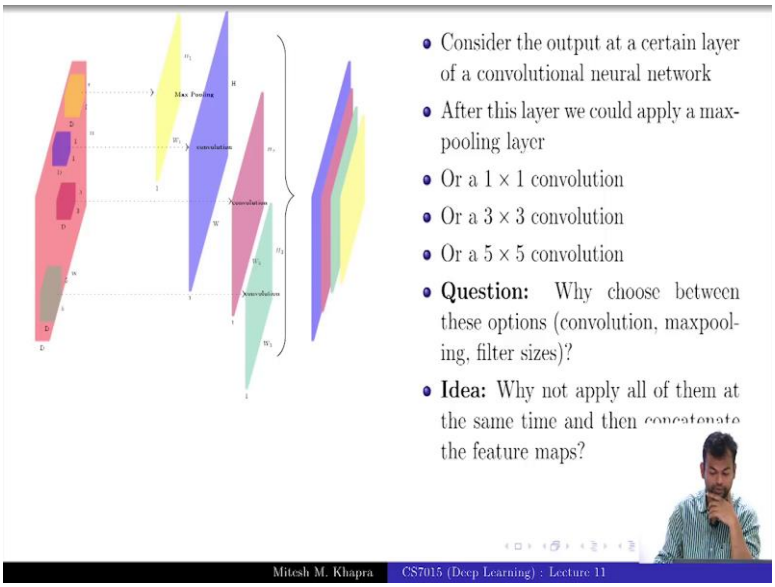
So, you can train any you can use any image trained image net and pass that image through it, or sorry, any trained convolutional network trained on image net and pass the image through that and you can get a representation for that image. And these are known as the fc representations and these are as the convolution representations ok, any of the convolution layers fine.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 11
Image Classification continued
(GoogLeNet and ResNet)

So, we will go to the next module where you wanted to look at two more architectures for image classification, these are GoogLeNet and ResNet.

(Refer Slide Time: 00:22)



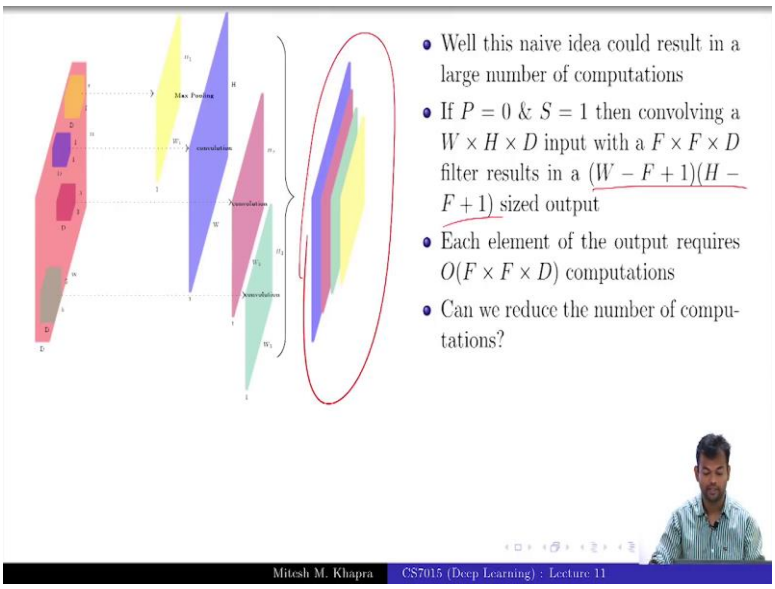
So, here is a question right, so, consider the output at a certain layer of a convolutional neural network. So, you have this layer after layer of convolutions and max pooling and so on and, you are at somewhere in the middle and this is what your volume looks, like this is what your output volume looks like. Now, after that you could apply a max pooling layer, you could apply a 1×1 convolution, you could apply a 3×3 convolution or you could apply a 5×5 convolution right.

And so, far we saw that all these architectures they do one of these things. They either do a max pooling or they do a 3×3 convolution or they do a 5×5 convolution or a 7×7, 11×11 right any convolution. But, they are all uniform they are all either 3×3 all either 5×5 or either 7×7 right. So, why choose between these options? Why not do all of this at every layer? Do you get the question that I am trying to ask right?

So, far what we were doing is that you have this volume, this volume at a certain layer of the convolutional neural network and after that you are either using all 3 cross 3 filters. So, you are using 256 3 × 3 filters or 5 into 3 × 3 filters or using 7 × 7 or using 5 × 5, you are never using a mix of all these right. So, why not use a mix of all these? Why the why take a decision on that I only want 3 × 3? Because, it is possible that you want to capture interactions at different levels so, you should have varied size filters at every layer.

So, how many of you get the question and the intuition that I am trying to ask?. Ok. So, the idea here in GoogLeNet or in the inception net is that why not apply all of them at the same time and then concatenate the feature maps right. So, I will also do max pooling, I will also do 3 × 3 feature maps, I will also create 5 × 5 11 × 11 and then just concatenate all of these together so, let us see how to do that right.

(Refer Slide Time: 02:14)



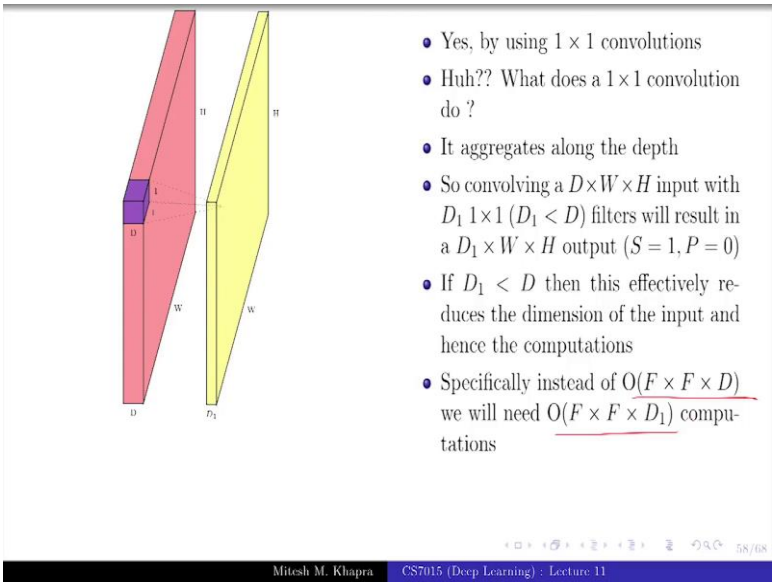
Now, one problem with this naive idea is that it could result in a large number of computations so, let us see what I mean by that. So, suppose the padding is 0, the stride is 1 then, if you have a W × H × D input as the volume and if you have an F × F × D filter, then the output would be of this size we all agree with this, this is the formula that we have been looking at. So, this is the size of the output volume. Now, every entry in this output volume requires how many computations? To get a single entry in this output volume how many computations do I have to do?

Student: F × F × D.

F × F × D so, how do I get a single value? I apply a convolution at that value and then I do those many computations. And, here the number of computations is that I am going over this block of F × F × D, doing a weighted multiplication and then adding them up right. So, you need that many computations, everyone is clear with this? Ok? Fine.

So, each element of the output requires these many computations and we have so many elements in the output right. So, you are doing really those many number of computations right. So, can we do something to reduce the number of computations right so, that is the key idea that we need to focus on. So, all of us buy the idea that doing this multi granular or multi sized filters is a good idea because, you are capturing interactions are different layer, but I showed you that this is a problem with this; you guys just apply multiple filters so, let us see what we do.

(Refer Slide Time: 03:53)



So, we what we do is 1 × 1 convolutions. What is the 1 × 1 convolution do? What does it make sense?

Student: (Refer Time: 04:02).

How does the 1 × 1 convolution make sense? You have a pixel, I fit a 1 × 1 convolution on that what will I get? I will get back the pixel.

Student: (Refer Time: 04:12).

Along the depth right so, remember it is not 1 × 1, it is 1 × 1 × depth.

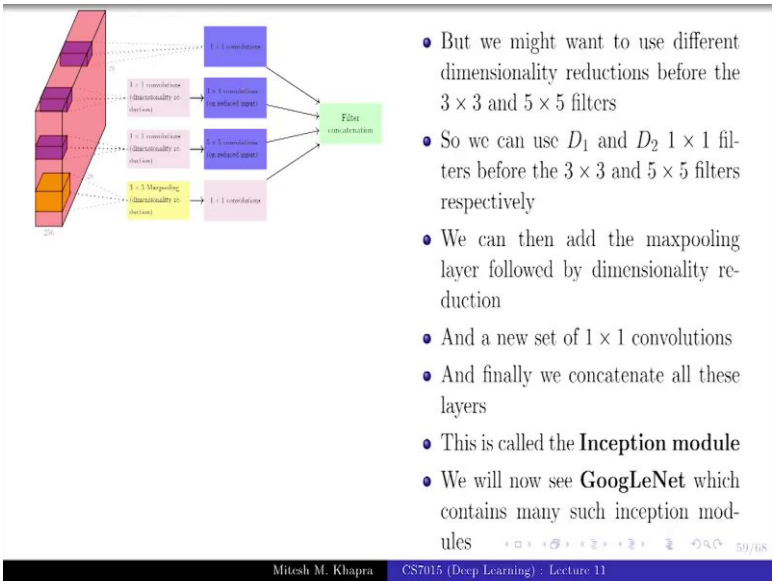
Student: Depth.

Right so, what does a 1 × 1 convolution do? It actually aggregates along the depth. So, this is what my 1 × 1 convolution looks like it is 1 × 1 × D. So, I just fit that block on that pixel and do everything along the depth and get a single value right. So, from a 3D output using a 1 × 1 convolution, I can go to a 2 × 2D feature map, everyone gets this ok.

Now, I could use several of these 1 cross 1 operations 1 cross 1 convolutions. In fact, I could use D1 of these such that D1 is less than D. So, what effectively happens? The depth of the output reduces. So, I take a certain output volume whose original depth was D, now I take D1 1 × 1 convolutions right. So, I get an output whose depth is smaller than the depth of the original output, is that fine everyone gets this ok. .

And, you see how this will save computations right because, remember that this was F × F × D and now I have reduced D to D1 ok. So, it is going to reduce the number of computations. So, that is what the idea is you reduce it from F × F × D to F cross of cross D1 right. So, thats this particular network or this paper introduced the idea of this 1 cross 1, actually it did not introduce it used it, but it made it popular probably.

(Refer Slide Time: 05:41)



Now, once you have done this so, this is how I am going to proceed now. I have a certain volume I have applied 1×1 convolutions to it, using that I have reduced the number of dimensions. Now, I am going to apply 3×3 convolutions as well as apply 5×5 convolutions on top of that right because, that is the motivation that I had started with that I want to apply kernels of multiple granularity.

Now, can you think of some refinement to this? You see this branching over here, why use the same 1×1 convolutions before feeding to 3×3 as well as 5×5? I could use a different set of 1×1 convolutions and feed it to a 5×5 and use a different set of 1 cross 1 convolutions and feed it to 3×3, is that fine. What is the problem with this?

Student: Again, increasing the number of computations.

Again, increasing the number of computations right, but they found out that the tradeoff between this is fine. Even if you are doing more 1 cross 1 operations it still is ok, the number of computations are still manageable ok. And, then you could also do a max pooling because, we were choosing between these things right 5×5, 3×3, 7×7 and max pooling. So, we will do all of these in parallel and we also do some 1×1 convolutions. So, how many different types of operations we have done? We have done 1×1, 3×3, 5×5 and max pooling followed by 1×1 convolutions.

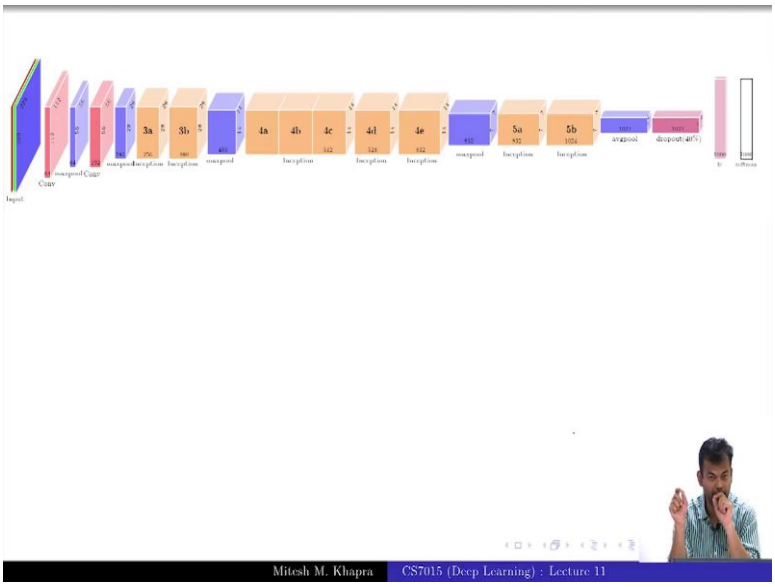
Now, all of these outputs that we have got, we have got a bunch of feature maps now. This is one set of feature maps, this is another feature maps, this is another and this is another. All of these 4, we are going to concat together to get a single output volume. Do you see what is happening, right? It is not very mechanical there is nothing really profound about what is being done. The only two profound ideas are, one is apply multiple kernels of different sizes and the other is to use 1×1 convolutions to make the whole computation manageable; that these are the only to main ideas. The rest of it is not very different from what we have been doing. How many if you get this operation completely?

So, this block is called the Inception module ok, this entire thing is called an inception module. So, in subsequent slides when I put an inception module then you know that these parallel operations are happening right. So, far whenever we had seen a convolutional neural network, it was all serial right so, you started with one operation then another operation, then another operation and so on. Now, you have an output or an

input volume, you apply multiple operations in parallel and get one single output right so, it is a parallel serial combination ok.

So, you will now see the full GoogLeNet architecture so, his question was basically 3 cross 3 would result in a different sized feature map right because of an 5 cross 5 would result in a different sized feature map. So, I will use appropriate padding so, that all of this becomes equal ok.

(Refer Slide Time: 08:26)



So, this is how GoogLeNet looks like so, you have the input again RGB and same 227 × 227 or 229 × 229. Then you apply a convolution layer followed by a max pooling layer, convolution, max pooling then you have this inception module with a very specific configuration. So, they have 96 1 × 1 convolutions before feeding to 128 3 × 3 convolutions, 16 1 cross 1 convolutions before feeding to 32 5×5 convolutions and so on. And, I do not really see much point in going into the details of these numbers, there is hardly any intuition behind them.

I again guess that it's you try a bunch of things and this is the one which probably gave the best output. So, the key idea is that of course, you have this inception module which is a parallel module which does a lot of operations in parallel. This is again followed by another inception module which has a different configuration followed by max pooling, then again a few inception modules. In fact, 5 of them again max pooling then inception and this is the other interesting idea that they came up with.

So, at this point remember in VGG net at the final layer you had an output of size 512 $\times$ 7 $\times$ 7 right. And, we said that this was a problem how many of you remember this? Why was this a problem?

Student: (Refer Time: 09:54).

Because, I need to connect this to a.

Student: Fully connected.

fully connected layer right and that fully connected layer was of size 4096 right. So, what they said is that what you could do is instead of taking 512 cross 7 cross 7, for each of these 512 feature maps that you have take the 7 cross 7 and just do an average pooling from there. What does what do I mean by that?

Student: Average.

Take these 7 cross 7 values, take an average of that. So, now instead of 512 cross 7 cross 7, how many values will you end up with?

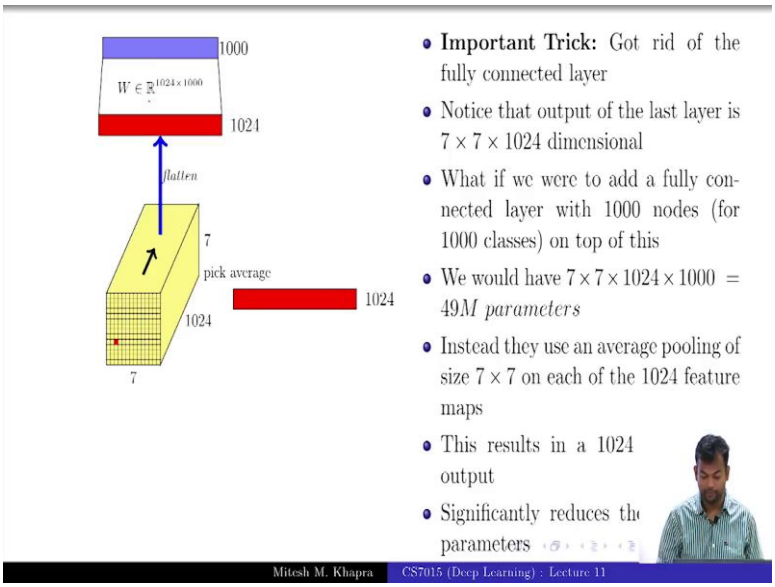
Student: 512.

Just 512 and in their case instead of 1024 cross 7 cross 7 you will just end up with 1024 values right. So, instead of looking at these dense connections with every pixel in the output volume, you just take the average of those pixels and then do a dense connection from there. So, from this volume you just go to a vector of size 1024 which is exactly this vector shown here and from there on life becomes easier right because, you have done a 50 percent sorry 50 times reduction in the volume. So, this was 1024 cross 49, now we just have 1024 cross 1 so, you have a 49 times reduction in the size and that is a huge parameter reduction.

And, that actually worked very well in practice they of course, add these dropouts and other things and then you have your fully connected layers. And finally, the soft max layer at the output to predict one of the thousand classes right. So, this is the full structure of GoogLeNet or inception net or with multiple inception modules right. So, just remember that key takeaways are three: one is half filters at multiple granularity applied in parallel, the other is use 1 cross 1 convolutions to reduce the number of

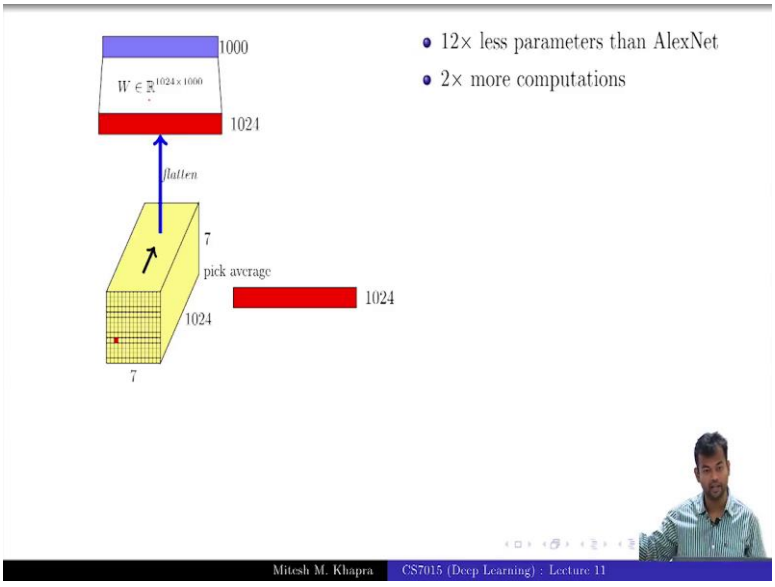
computations. And, the final one is to use this average pooling to make sure that you do not have this blow up of parameters at the output ok. So, these are the three main ideas that you need to do right ok.

(Refer Slide Time: 11:45)



So, this is exactly what I explained so, instead of having this nasty looking connection which would have been 50176 cross 1000, you just take the average from this grid. And, just get a 1024 dimensional vector which results in a much smaller weight matrix at the output; everyone gets this so? Ok? Yeah, so this is fine.

(Refer Slide Time: 12:08)

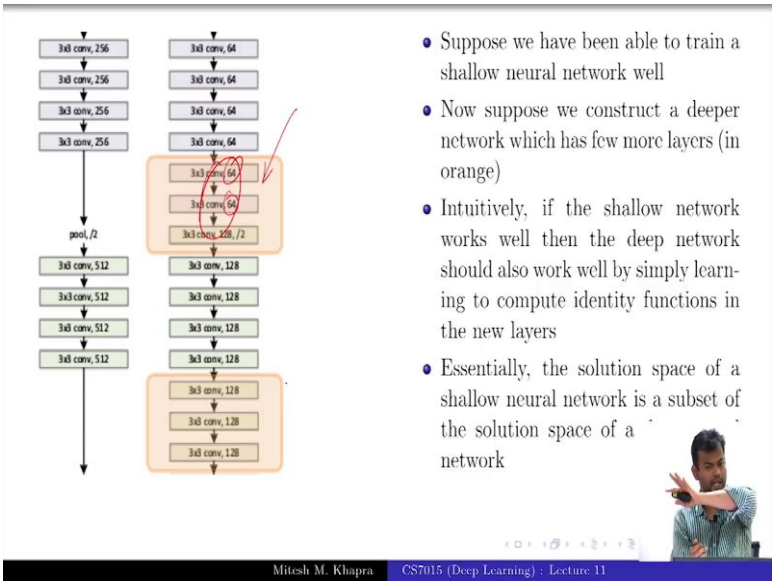


So, this has 12 times less parameters than AlexNet, it has 2 times more computations right. So, that is what I meant by the tradeoff so, the number of parameters has reduced significantly of course, a large amount of this savings happen in the fully connected layer. Its not the ingenuity in the inception module which led to the fewer number of parameters that actually leads to more number of parameters right.

But, the reason they could afford more number of parameters in the convolution layers is because, they reduced a lot of parameters in the fully connected layer. Do you get that? So, they did this tradeoff and it has 2 times more computations then AlexNet, but it is still acceptable because you see that there are many many layers as compared to AlexNet right. So, let us actually count the number of layers that we had here so, 1 2 3 4 5 6 7 8 9 10 11 12 13 14 right? So, it has 14 layers and each of these inception modules is again like split layer right it has this parallel components there.

So, having 2 times more computations was still an acceptable tradeoff. And, it of course, led to much better accuracy as compared to AlexNet or ZF Net or VGG Net right that we had seen in the original trend graph ok. So, now we will go on to the last architecture that we will discuss for image classification which is ResNet.

(Refer Slide Time: 13:24)



So, here is the idea behind ResNet or here is the motivation right. Now, suppose we have been able to train a shallow neural network well, now again my definitions of shallow

are relative this is by no means shallow; there are many layers here right. So, you have some 8 layers here.

Now, if I have been able to do this properly; that means, what I mean by that is that using this network at least I was able to reduce the training error to 0 or close to 0 some acceptable value. And, I was able to get some reasonable generalization performance; that means, on the test that I was able to get some reasonable accuracy that is what I mean by I was able to make this network work well.

Now, suppose I add a few layers to this network and I have carefully added some layers in between here and in between here or over there right. Now, intuitively I could argue that if the shallow network was working well, right then for the deep network this is exist at least one solution which can directly come from the shallow network. Can you tell me what that solution is?

Student: (Refer Time: 14:29).

I want all of you to kind of digest that idea. What the deep network could have done is, I know that this shallow network works why not I just behave like that and I learn these parameters in such a way that I just end up copying from here to here. How many if you get this? Right so, there is a case for the deep network to do at least as well as the shallow network and it could do the same thing at this point. All of you get this idea right?

So, in other words the solution space of the deep network or rather the solution space of the shallow network is actually a subset of the solution space of the deep network. There was one solution for the shallow network which could have been used as it is for the deep network. Of course, for the deep network there are several other solutions because, instead of the identity here you could learn different things there, but at least that one solution exists. So, I should at least if I do use this in practice ex I should expect that this would work as well as the shallow network right. Is that argument fine with everyone?

Student: (Refer Time: 15:29).

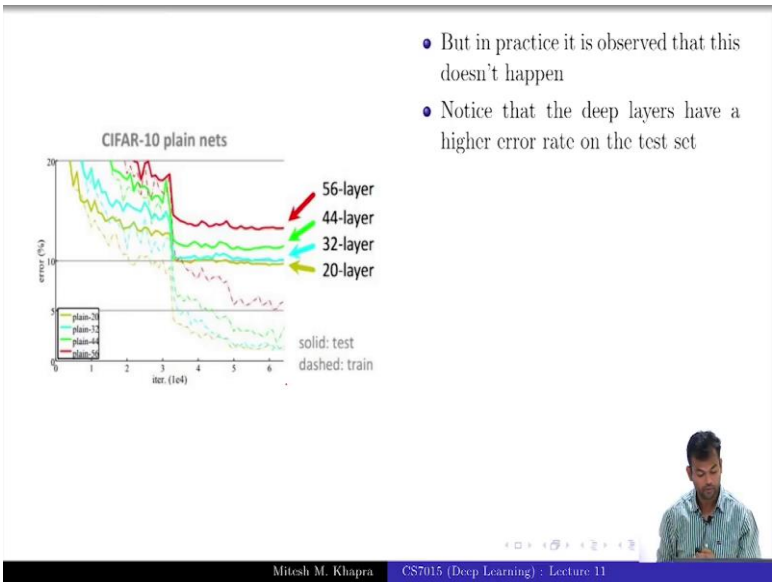
Or which has only one, yeah yeah of course.

Student: Yes.

I mean so it those arbitrary things would not work, but here what there it is the for the explanation intuition right you are using some reasonable things. And, you are just trying to make it compatible with whatever you have so far. So, the argument is valid right so, I cannot expect that I had a volume whose depth was 128 and then I suddenly decide to use only 1 filter in the next layer right.

That means, I have compressed everything and now I expect it to be able to recover from there that is not going to happen right. So, that is a fair argument, but the argument which I was trying to make or at least for the illustration purpose is that, if you do reasonable things and that is what people were trying out right. These this is the exact network that someone was trying out and this did not work with well; I will tell you what it is. So, do you get his doubt and my clarification on that? Is that fine? Ok.

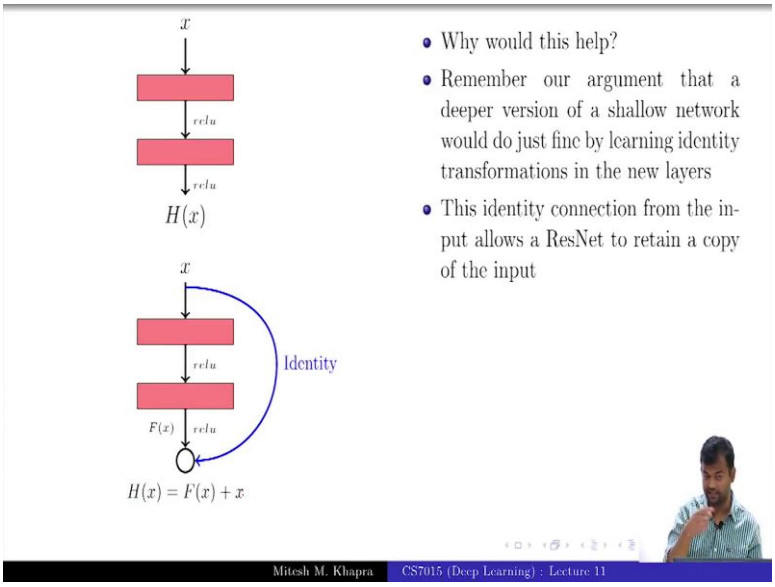
(Refer Slide Time: 16:16)



So, this is what was happening in practice right. So, you have a 20 layer network or 32 layer network or 44 layer network and a 56 layer network. And, you see that the training error of deeper networks is much higher than the training error of the shallow networks. That means, this argument which I was trying to make that the deep network should at least do as well as the shallow network was not working well in practice right. And, it is if you think about it is not very surprising because, this argument hinged on the fact that it should be able to learn this identity mapping. But, this identity mapping is one of many solutions right.

So, for it to be able to narrow down on that solution, it is easy for you and me to think about it, but for the network it does not have this intuition right, that I can just copy it from there to here. Do you get that? The solution space is really really large and like finding that needle in a haystack right, you have these many solutions possible. And, I am trying you to arrive at a solution where you end up with the identity solution. Is that clear and it is not so, easy for the network to do that. Everyone gets this? How many of you get this idea?

(Refer Slide Time: 17:25)



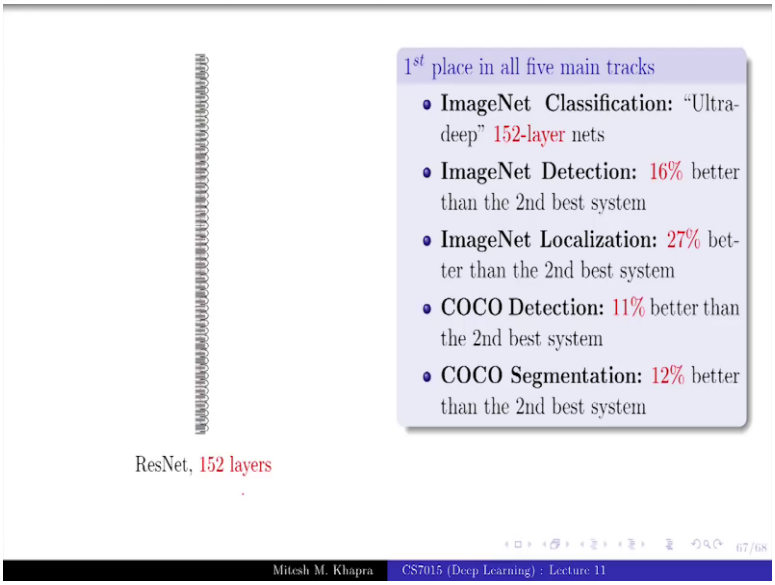
So, why not explicitly try to do something of this sort, where the network can actually learn some kind of an identity function. So, now consider any 2 layers, you know by stack layers I mean this is a convolution layer and followed by a convolution layer right so, these are 2 convolution layers back to back. So, from I what do I actually end up doing here I had a certain input x and I am learning some transformation of x, through these convolution layers right. I am trying to learn x and then I sorry I was given x, I am passing it to convolution layer so, I will run some transformation of x.

And, my argument was that if it could learn to directly copy x here, the deep network should at least work as well as the shallow network. So, why not I explicitly ensure this? So, why not I do this, that in addition to these connections I also explicitly connect x to the output. Do you get this? So, now, what I am trying to do this, is Hx is equal to F of x which is the transformation that I learned for x and in addition I also add x. So, what am

I doing? I am explicitly feeding it the identity function right. How many if you get the intuition for this? So, what I am trying to do is I have a sense that if I could have transferred this x as it is across layers, then there is a chance that I should be able to do as well as the shallow network right.

So, now I am going to explicitly ensure that that you learn these transformations, but I will also feed you x at every stage at a reasonable time right. So, these are known as skip connections. So, after every 2 layers I will feed back the x or you could try after every 3 layers I will feed back the x. So, I am trying to maintain the original copy of x after every interval. Ok? Fine. So, why would this help so, this follows back from our argument and it is the same thing which I said before.

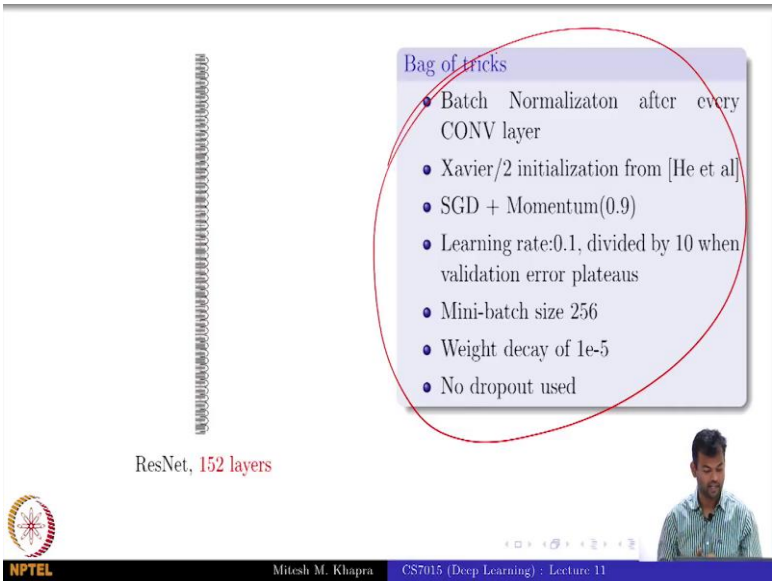
(Refer Slide Time: 19:17)



So, using this idea of using these skip connections, these authors were able to train a really deep network of 152 layers right. And, this gave on multiple vision challenges right one being ImageNet, it gave 16 percent better results and the best network. And, this is the one which reads that near human performance. Then ImageNet localization is another challenge, where you need to localize the object. So, there they get 27 percent better than the best results and there are these bunch of other vision tasks detection segmentation and all of them. And, in all of them this significantly outperformed the best system using a very very deep network. Of course, the downside is that you have a very

very deep network; it will take its own time to train and so on. But of course, if you have a Microsoft or Google you can afford to do that.

(Refer Slide Time: 20:08)



So, that is the current theme right I mean the one with the largest computational resources wins everything right. So, and they also are there is some other bag of tricks which is not I mean it is not very difficult to understand. So, they used batch normalization every after every conversation layer. Have you heard of batch normalization ever in your life? Ok good. They used Xavier by 2 initialization, ever heard of that? Xavier by 2 was the same as?

Student: He initialization.

He initialization, right? then they use SGD, not any of the fancy Adam or Adagrad or anything with a momentum of 0.9. Learning rate was set to 0.1 and divided by 10 whenever the validation error plateaus. The mini batch size was 256; they use a weight decay of 1 ht. What is weight decay? Weight decay is in the context of which regularization?

Student: L2.

L2 and what does this mean, weight decay of 1 e raised to minus 5?

Student: Lambda (Refer Time: 21:05).

The lambda was set to 1 e raised to minus 5; all of you remember these things right. We did it in some previous course, in some previous life and no drop out was used right so, since I have this here I will just say something more on this. So, in your reading papers on deep learning right focus on the experimental section, where all these hyper parameters are described so, these are known as the hyper parameters. These are not related to the parameters of the model, these are related to hyper parameters which is what the batch size is whether you used l 2 regularization. What was the learning rate? What was the optimization and so on?

So, turns out in many cases if you do not stick to this you will not be able to reproduce the results of the paper right. So, you might be wondering that this network I understand, this is just 152 layers and I can just keep adding skip connections; I can easily code this up. But, I am not getting the same results as the original authors of the paper.

So, this is where you need to dig up them right, you need to look at the experimental section where most good authors provide these details of how they have trained the network, how many epoch that they use, what was the patients set and all that. If you follow those the chances of reproducing are much higher still not guaranteed, but definitely much higher ok. So, that is where we end the lecture on convolutional neural networks and ImageNet classification.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 12
Visualizing Convolutional Neural Networks, Guided Backpropagation, Deep Dream, Deep Art, Fooling Convolutional Neural Networks

So in this lecture we will look at various ways of Visualizing Convolutional Neural Networks and although it is not very obvious at this point as we go along we will see what I mean by that. So, let us start this lecture.

(Refer Slide Time: 00:29)

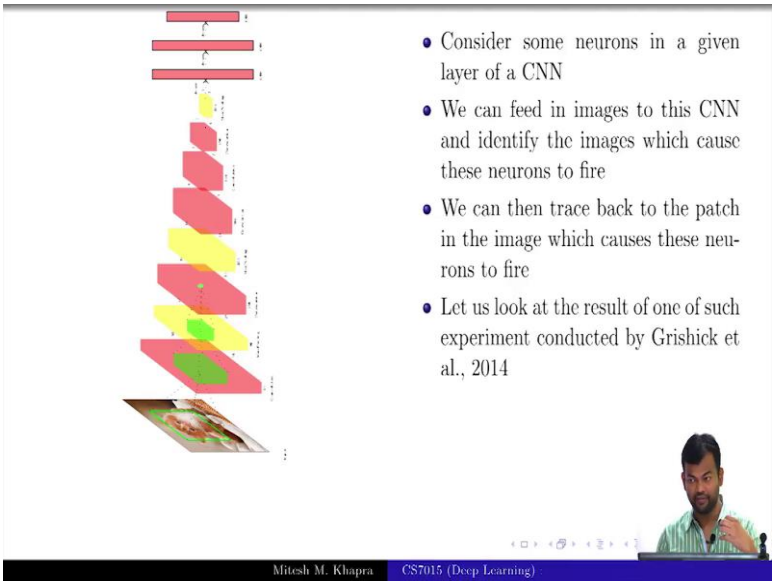


So, I forgot to add the acknowledgments slide. So, a lot of the material that I am going to cover today is based on some content by Andrey Karpaty in his online course, Stanford course. We will add the appropriate acknowledgments and a link to the course ok.

So, with that I will start module 1, which is visualizing patches which maximally activate a neuron ok. So, what are we trying to do here is we are trying to, the quest today largely is going to be able to understand what a CNN has actually learned right and what I mean by that is we said that there are these filters, which try to detect edges, which try to detect blurs and so on and then there are these neurons which fire for certain things and so, on.

So, we want to see different ways of finding out what a convolution neural network has actually learned or what have the filters actually learned or what are the different neurons in the convolutional neural network actually capturing? What do they fire for what are the kind of images that make them trigger and so, on right. So, that is the first thing that we are going to look at how do you visualize patches which are causing a neuron to fire.

(Refer Slide Time: 01:32)



So, this is again our VGG network just put it vertically. Say have passed an image to that, and then at every layer you are applying convolutions and then match pooling and so on, right up to the last layer right. Now we consider some neurons in one of these layers. So, I am considering this neuron and I want to find out what exactly is this neuron trying to do right and which is the same as asking what kind of images does this neuron fire for.

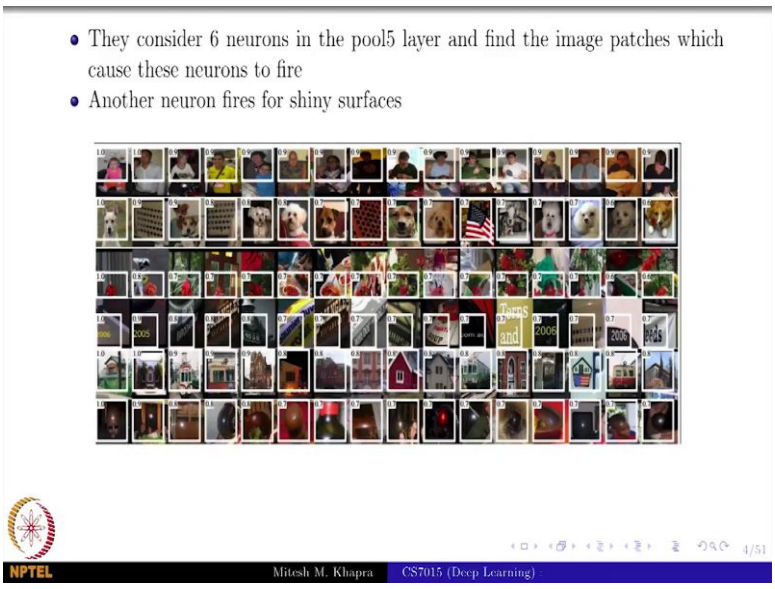
So, I have thousand different classes I have cats, dogs, cars, trucks and so on. I am interested in figuring out what are the different kinds of classes that this neuron fires. And this is more from say I am already getting some output accuracy and I am either happy with it or not happy with it in either case I just want to see what is it that my network is learning is there any scope for improving. Is that that there are no neurons in the network which actually fire for the dog class, did not should I do something differently was it that most of the neurons fire for all classes; that means, they do not have any discriminative power. So, what exactly is going on right?

So, that is why we are that is why this study is interesting and you will do something of this sort in your CNN assignment ok. So, and by now we are clear that if I am focusing on any neuron and any layer, I can always go back and trace the patch to which it corresponds in the input image. Everyone is fine with that? Right. So, we saw that if I am somewhere here, then every neuron here corresponds to some 16 cross 16 patch in the original image and the same is true for every layer right. I can always this is a deterministic process, I can just find out which are the original image pixels which contributed to the computation of this particular neuron in any layer ok.

So, I can do that. So, now, what I am going to do is, I will send as many images as possible, whatever images are there in my training data, test data whatever images I have, I will pass these images through the convolutional neural network ok, and for the neuron of interest I will note down, which when does it fire and where ever it fires and by fire I mean it is a output is close to one or it is a output is high because these are ReLU neurons, I look for high output they do not saturate at one right. So, this I look which images for which this neuron had an high output and for those cases I will go back and trace the image and see which patch of the image actually caused this to fire.

So, I want to see whether my neurons are actually learning things like noise detector or eye detector or something right.

(Refer Slide Time: 04:01)



So, let us look at the results of one such experiment done by a group of researchers. So, they considered some neurons in the pool 5 layer and they did this experiment that they pass a lot of images and whenever this neuron fired they went back and saw what was the patch in the image, which was causing this neuron to fire.

So, that they found that one set of neurons is actually fires for people places. So, if you go back and trace which is the image, which caused is to fire or which is the patch, then it is largely centered around a persons face or which is something which is very clearly a person ok. Another set of neurons fires for dogs, another set of neurons fires for flowers all sorts of flowers and different orientations different maybe colors are same here, but they are all different thing right somewhere inside a bouquet somewhere inside a flower pot some somewhere on a table and so, on, but expected of that these neurons are firing for any flowers that appear in your input image and the fire only for that patch nothing around it.

So, it is very is actually able to localize and fire. There are some images which fire for this images the digits and alphabets written in the image. So, these are some addresses or dates or billboard signs or something like that and whenever there are these characters or numerals there and this neurons fire.

And some neurons fire for houses and then some neurons fire for shiny surfaces. So, there is this different sets of neurons which fire for different sets of things right. So, also; that means, your convolutional neural network is trying to learn specific characters of the input characteristics of the input and this is one way of visualizing. So, this is not like anything tricky here it is just that its good you can think of this as debugging tools for your convolutional neural network right, because in your you I guys are used to programming where you give different inputs and see what is the output and then try to debug it.

So, this is one way of trying to figure out whether your network has learned does it really need more training is there a certain class of images for which it is not firing at all or is it confusing between two classes and so, right. So, that is one way of visualizing.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 94
Visualizing filters of a CNN

So, now, this was Visualizing the Neurons inside the Convolutional Neural Network. So, neurons remember are the outputs right, these are not these are the feature maps. What about the weights itself, What are the weights in a convolutional neural network?

Student: (Refer Time: 00:25).

The filters; the filters themselves are weight. Have you ever tried to visualize weights before, when?

Student: Auto encoders.

Auto encoders and what was a trick there, how did we?

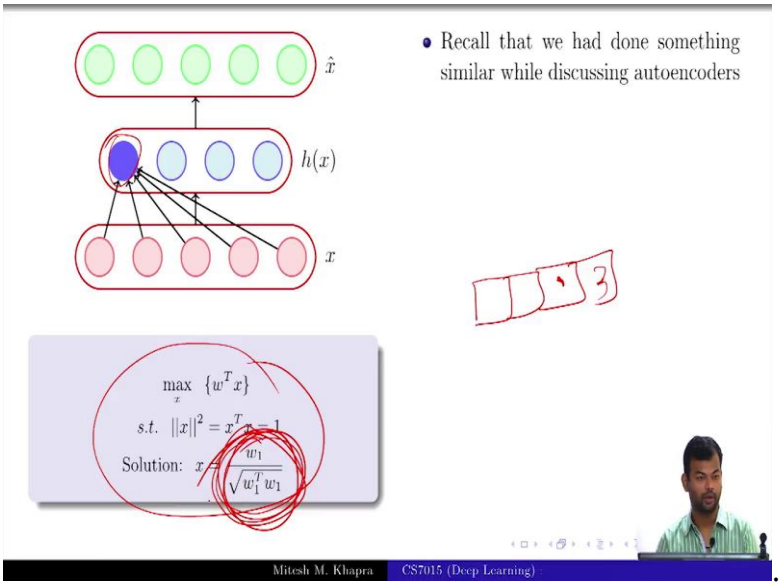
Student: (Refer Time: 00:36).

Visualize what was the optimization problem that we solved?

Student: (Refer Time: 00:40).

How many of you went and looked at the prerequisites? How many of you looked at the prerequisites? ok.

(Refer Slide Time: 00:46)



So, we had done something similar while discussing auto encoders. So, because that we had done something similar while discussing auto encoders right. So, we were interested in knowing that there is a particular hidden neuron inside the auto encoder and we wanted to see that; what does this neuron capture? So, if I give it emnist digits then what kind of patterns does it fire for and if you remember we had solved this optimization problem and realize that, this neuron will fire for an input, which looks like this where w1 or all the weights which are connecting to this neuron ok. What was the dimension of the input if you are dealing with emnist digits?

Student: (Refer Time: 01:22).

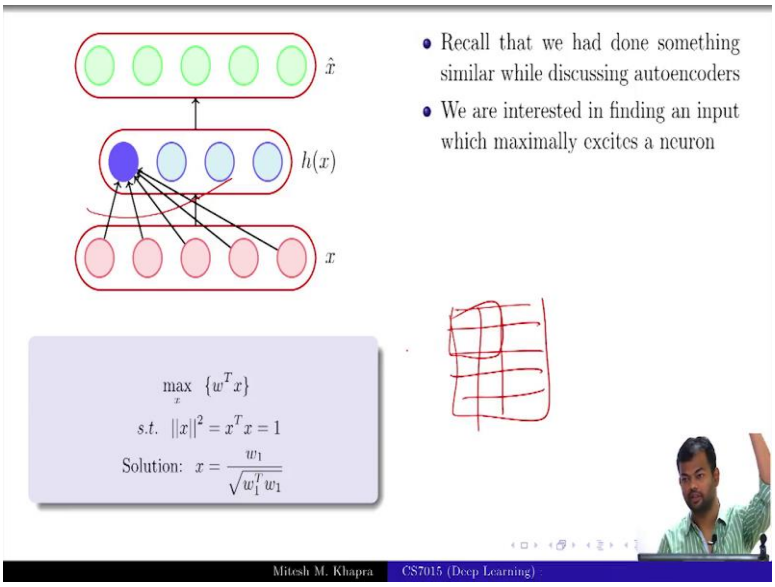
784; what is the dimension of this a one thing which I have circled here?

Student: (Refer Time: 01:27).

784 right; it is written x equal to so, it has to be 784. Why is it 784 because there are 784 weights connecting each of the input pixels to that neuron right so; that means, this weight matrix itself we can visualize it as an image and thats exactly what we had done if you remember we had this grid of images that we were analyzing and in some images we saw that some dark element fires here and each we were arguing that this is the curve which exists in 2 or 9 or 8 and that is the one which is capturing.

And in some cases there was a cusp here which was firing and we were arguing that this could be for the 3 or for a 9 or for a 8 or something like that right. So, we were trying to visualize these things and the way we had plotted it was just treating this weight matrix or weight vector as an image and seeing what causes the neuron to fire right.

(Refer Slide Time: 02:21)



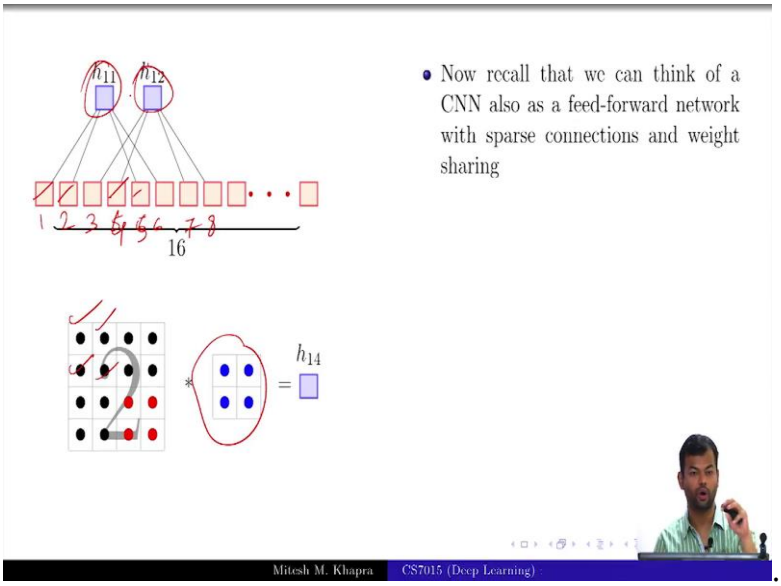
So, we can do something similar for convolutional neural networks. I want you to think how would you do that I will give you some hints; the answer is there on the next slide, but I just want you to think about it right. So, remember here you have dense connections ok; that means your weight vector was the same dimension as the input vector. What about filters in the case of CNN? They are smaller they are 3×3, 5×5 or 7×7 much smaller than your original image.

So, then what do these filters correspond to? Just think of the animation that we had seen right we had this image and we were taking a filter and applying it at different places. So, what does the filter correspond to? What is the filter overlap with; patches in the image right. So, now, what kind of analysis can you do?

Student: Dense.

What kind of patches does this filter fire for or what kind of patches does the neuron connected to this filter fire for? Does that make sense everyone gets the intuition? How many if you get the intuition please raise your hands; thank you.

(Refer Slide Time: 03:23)

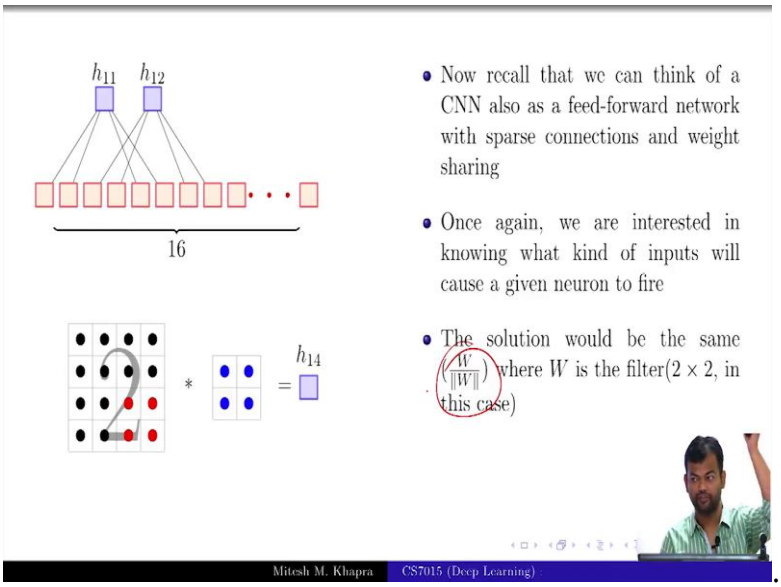


So, now, recall that we can think of a CNN as a feed forward neural network and in particular when you have a filter it actually interacts only with few pixels right. So, interacts with say pixel 1 2 5 and 6. So, that is the patch that it interacts with.

And now I want to see when does this neuron fire. So, that is the same as asking what do I put in 1, 2, 5, 6 for this neuron to fire or similarly what do I put in 3, I do not know this was 1, 2, 5, 6 I guess. So, 3, 4, 7, 8 for the same different neuron to fire right, but all these neurons fire because they are connected to the same filter.

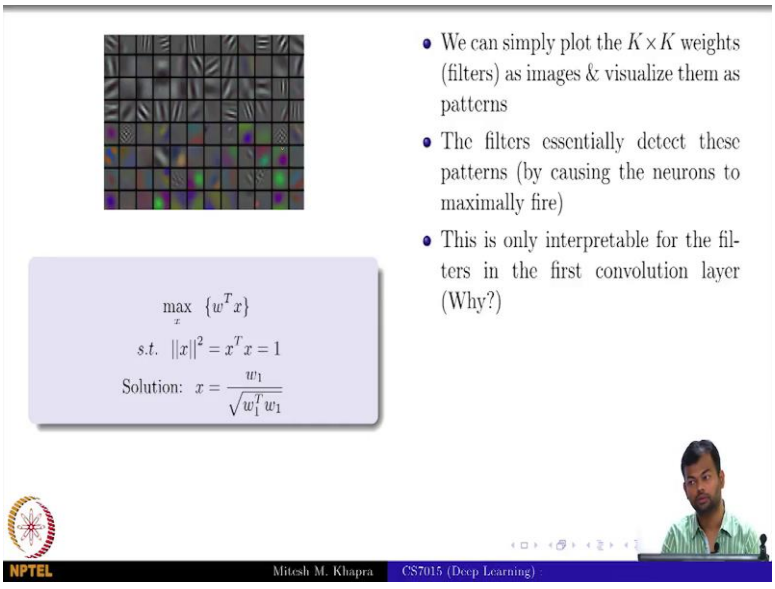
So; that means, I am interested in these patches, which will cause the neuron to fire and those patches can appear anywhere in their image, is that fine that is the whole point of convolution neural networks; wherever there is a nose whether it is at the top corner of the image or the center or the bottom it should be able to detect right that is the whole point of weight sharing and sparse connectivity ok.

(Refer Slide Time: 04:23)



So, we are going to do exactly the same thing. We will have a 3 × 3 filters or 5 × 5 filters or 7 × 7 filters were just going to visualize as them as images, but unlike the earlier case where the image actually correspond to the full mnist image here these images are just corresponding to those 3 cross 3 or 5 cross 5 patches and you want to see what kind of patches causes the neurons to fire ok, and the solution is still the same we will have this W by W the normalized weight filter weight, which is causing the input to fire. How many if you are fine with everything at this point please raise your hands high up.

(Refer Slide Time: 04:55)



So, this is what we get right and we observe certain things which like we had earlier made a case for that these filters try to detect certain types of patterns or textures or edges. So, you can see that right this is capturing these slanting edges this is trying to capture some horizontal sorry vertical edges then some edges oriented differently and also some colored, patterns some texture right. So, you see something like a checkbox here or chess box here and so on.

So, these filters are actually firing for different kinds of patches. So, they are trying to detect different things from the images. So, you could visualize this and unless you see a lot of variety in this; that means, something is wrong right because your filters are not being trained to be discriminative with terms of different patterns that they can detect and so, on right. So, you want these variety of patterns to occur ok, and I am going to make a claim that this is only interpretable for the first layers in the convolutional neural network why is it so, I am seeing some half complete answers. So, I will ask this as a quiz question.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

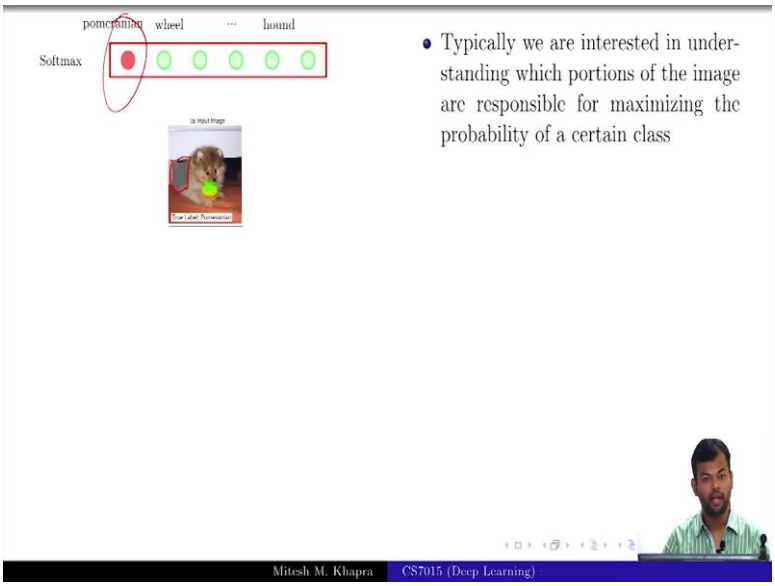
Lecture – 95
Occlusion experiments

Ok, the another thing that you can do is to figure out whether things are working properly or not. So, you can do something known as an Occlusion Experiment. So, these are all your debugging tools sort of to say if you are working in Vision or computer Vision where you are using a convolutional neural network and this is to gain more insight said, most of you will get away by just taking an off the shelf convolution neural network training it on your data getting some accuracy and reporting it.

But, for those of you who want to really understand what is happening and how can $\times$improve things further. So, this could actually tell you for example, if you want to compare whether a 5×5 filter would have been better than a 7×7 filter. Then you could have observed what these filters are actually learning and in your data does it make sense to have a 5×5 filter versus 7×7 filter. Because maybe the 5×5 filters are not being able to distinguish enough. But, if you had used a smaller or a larger filter things would have been different, right.

So, this is for people who really want to get into the know how of how things are working otherwise most of you I do not really expect you to do this is, but this is an important set of tools to have and I would strongly encourage everyone to experiment with them and some of this you will do in the assignment, ok.

(Refer Slide Time: 01:16)



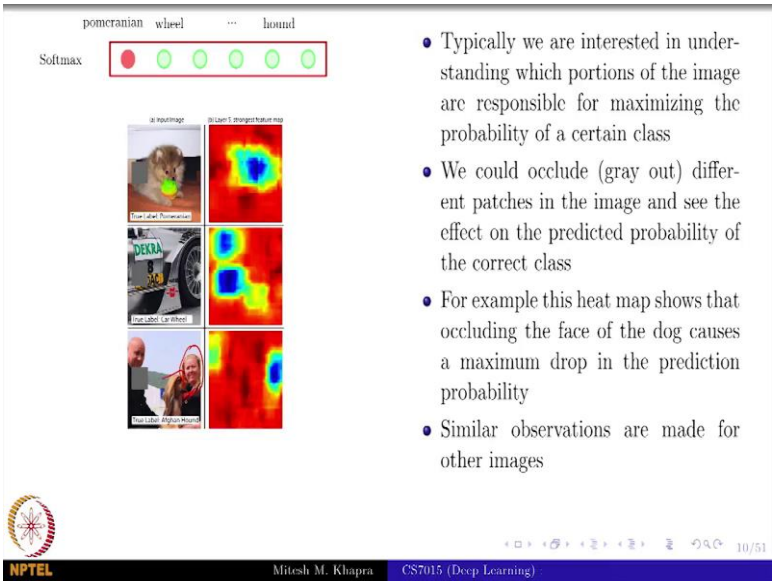
So, here is the idea of behind occlusion experiments. So, we are interesting knowing that what patches in the image are actually causing the output to belong to a particular class, right.

So, I have here the figure of a dog and the class being probably predicted is a Pomeranian and I want to know that what patch of the image actually resulted in this output, right. So, have you tried doing this in any other context; if you want to know if you have several features or several things several factors and you want to decide which actually influenced the output, how would you do it? So, what you could do is you could drop one factor, right and see whether your output would have drastically changed if it goes from positive negative then that maybe that was the factor which really mattered, right.

So, if for example, it is a movie review classification, right. So, when you drop certain words from your review. So, you drop the word amazing, great and so on and keep everything else the same. Now, it is quite likely that your probability of the review still being tagged as a positive review will at least drop earlier maybe with these words it was getting tagged as a positive review with 0.9 probability it would come down to 0.6, but now if you drop words like the, and, for and so on then you do not expect the output to change much because these words are not really important indicators of positive or negative, ok.

So, the similar thing that you do here is you occlude certain patch, patterns, a certain patches of the image. So, I have shown one occlusion here so, I have replaced that patch by a gray patch and I again feed the image to the convolutional neural network and I see what is the probability of the Pomeranian class right now, ok, and I do it for all such patches in the image. I can do for as many patches as I want.

(Refer Slide Time: 03:03)



And, I create something known as a heat map. So, the red portions here are the ones which do not cause a large drop in the output probability if you occlude them and the blue portions are the ones which cause a large drop in the output probability if you occlude them and it is pretty obvious. Because what is happening is when I cover the face of the dog the probability drops drastically and that is what you would expect, right. So, this is also an indicator that your network has actually learned something meaningful. It is being able to detect this based on the facial features and not just randomly guessing that this is a dog, right and see the similar experiment.

So, for example, if there is a car, sometimes these results are not very at least to me it does not look very intuitive. So, I would have expected that the wheel would have been one of the deciding factors, right. So, if I occlude the wheel the probability should drop drastically, but the other way of looking that it is that its really learning a lot of redundant features. So, it is not heavily relying on the wheel unlike in the dog case even

if the wheel is occluded it is relying on certain other features which look like cars and hence the probability is not dropping drastically, right.

So, this allows you to interpret what kind of things it is running. So, if its heavily for example, for face detection if its heavily relying on nose to detect the face to say that this is the face; the moment you block the nose it will drop its probability of detecting this as a face, but thats not good right because you want these redundant features. Remember we had discussed this at some other point where it should try to detect the face not only from the nose, but also from the ears, from the hair, from the eyes and so on.

So, if your occlusion is not drastically reducing your probability; that means, it has learned some redundant features which are still allowing it to operate well even though certain portions of the image are not there; that means, it is more robust noise in that case, right. And, here it looks like it is not so robust because it is probably heavy this is the rearview mirror of the car. So, it is probably heavily relying on that feature to detect a car, ok.

Then this is another thing where the true label is an Afghan Hound and for some reason if you occlude the face of the woman its probability decreases, now let us not comment on that, but if you go back and look at the image you might be able to make some observations, right. So, these are things so, this is an indicator that is probably not really learnt it well maybe all the Afghan Hound images that it saw maybe a woman was carrying the dog always, right.

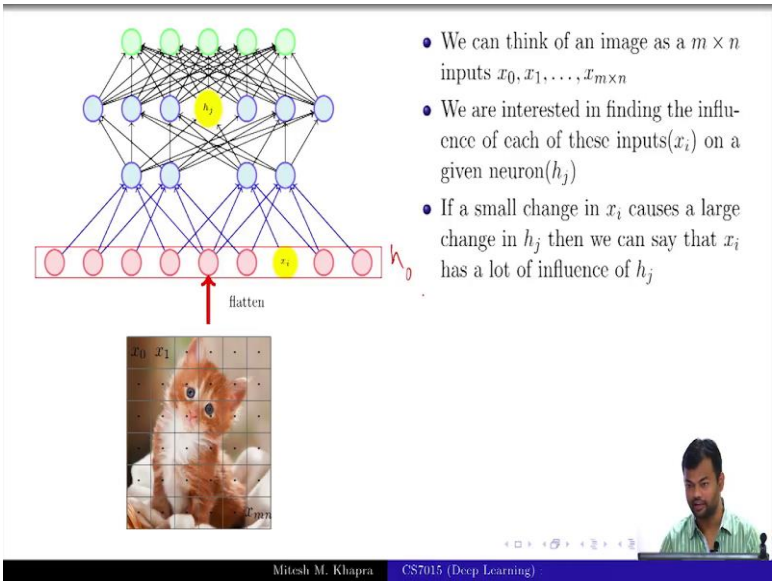
So, its learn this wrong association that when I see a woman with some object it that is the portion which is the dog which is bad, right. So, now, you can see that your network has not learned something interesting and you would want. So, if you look at one network which is predicting a dog based on this kind of a occlusion and another network which is predicting a dog based on this occlusion then you would prefer the other one and so, this is a very interesting experiment to do.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 96
Finding influence of input pixels using backpropagation

So, now, so far what we have done is, we have seen the influence of neurons on the or what image patches cause a neuron to fire. Then we have visualized the weights. So, neurons have been visualized, weights have been visualized then, we have done some occlusion experiments on the input image. Now, we will take this further and we are interested in seeing that, what pixels in the image actually help in the output or in any neuron in the intermediate layers and we will find out some principal way of finding this influence right and we are going to use back propagation; that means, we are going to use gradients ok.

(Refer Slide Time: 00:50)



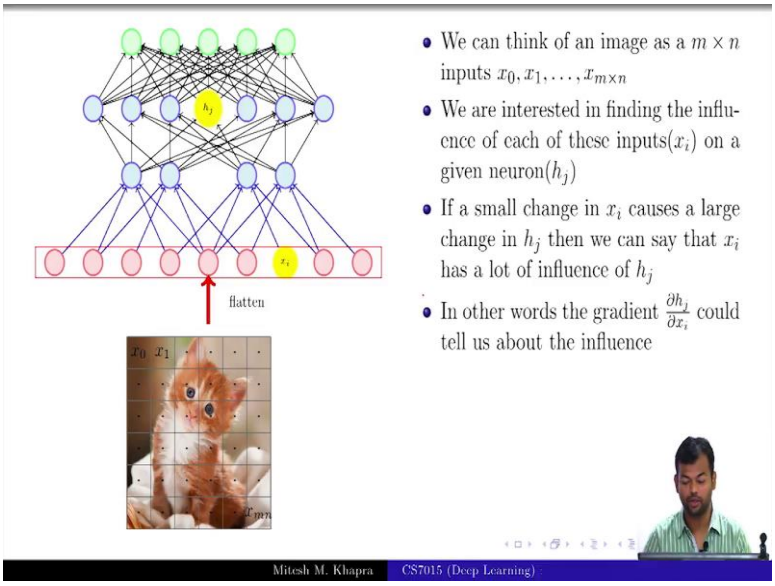
So, we can think of an image as an m cross n inputs, going from x_0 x_1 all the way up to x m × n nothing great about that. And we are interested in finding the influence of each of these inputs on a given neuron ok. Now, what is one way of computing influence that you have learned in this course, what is the hero of this course? Gradients, right. So, gradients tell you the influence. So, now, can you tell me if I want to compute what is the influence of this neuron or this input on this neuron, what would you do?

Student: (Refer Time: 01:27).

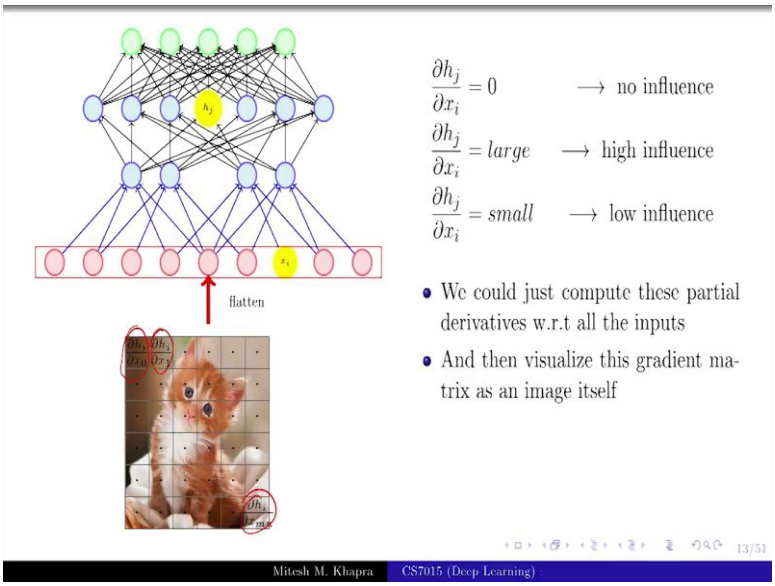
$\partial h_j / \partial x_i$, but can you compute that, how will you compute that, how do you compute the gradient with respect to the input? We have always stopped at the last hidden layer and the weights before that. So, how will we do that, how will you do this ok? This is a trick question just a hint. Is there a restriction on the chain rule or can you do it? You can just keep adding links to the chain right. So, what is so difficult about that? You already know how to compute the gradients till this point and in fact, you will also know how to compute the gradients till this point..

And what is stopping you from doing it up to this point. What if I just call this h instead of x then, you would not have a problem right. And actually, we call it h right we call it h_0, we can do it right. It is straightforward, so let us see.

(Refer Slide Time: 02:11)



(Refer Slide Time: 02:15)



If, I want to compute $\partial h_j / \partial x_i$, I can see that if the if $\partial h_j / \partial x_i$ is 0; that means, this pixel has 0 input on the neuron. If it is large then, it has a high influence if it is small then it has a low influence. So, this is how I will see whether, a pixel has an influence of the certain neurons in the in some of the hidden layers. And this is not restricted to convolutional neural networks as you can see, I am just actually treating it like a feed forward neural network with parse connections ok.

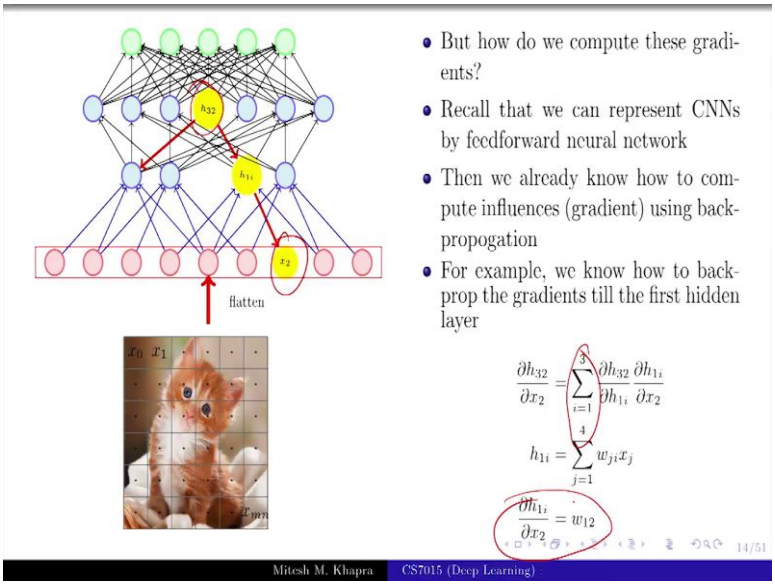
So, we could just compute these partial derivatives and visualize this gradient itself as an image. So, what do I mean by that is, I am going to compute $\partial h_i / \partial x_0$, $\partial h_i / \partial x_1$ all the way up to $\partial h_i / \partial x_{mn}$ right. So, I am going to compute this m cross n entities and I can just visualize this as an image. Now what do you expect this image to look like? If 0 represents gray colour, what do you expect this image to largely look like, what would you actually hope for? This image should be largely gray because, most of the input pixels should not be influencing a given pixel in the hidden layer right, that pixel should influence by only a small number of pixels.

So, that we can say that, this is the patch which causes it to fire and not that every pixel in the input is causing it to fire because, that is meaningless. So, that does not that is not something that we care about. How many if you get this please raise your hands. So, I will just repeat it, if a pixel fires for every pixel if a neuron in the hidden layer is influenced by all the pixels in the input; that means, it is not really discriminating, it is

not really specialized right. We want neurons which fire only for certain patches in the input, so that we know that this neuron is responsible for this kind of a pattern ok.

So, if I plot this as a image, I would want most of these entries to be close to 0 right because, I want the influence to be 0 ok.

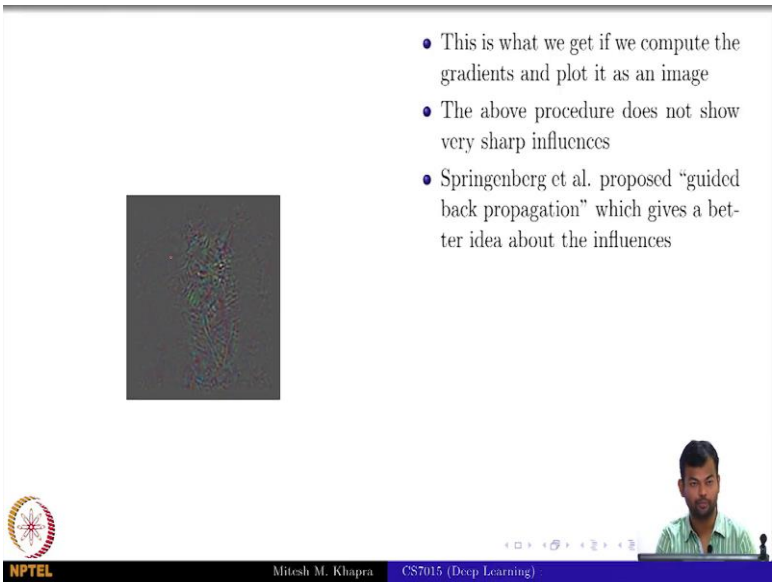
(Refer Slide Time: 04:04)



Now, the question is how do we compute these gradients. So, we will just treat them as a free forward neural network. We already know how to do back propagation across these roots and we just need to add one more term to the chain right. So, I will just show you what we will do here. So, I am interested in $\partial h_{32}/\partial x_2$. So, I will observe that there are 4 paths which go from h_{32} to x_2 or rather from x_2 to h_{32}. So, I will just sum up the gradients along these 4 paths right and if I solve it I will just be left with this ok. So, that is how I will visualize. So, this is very simple we have done a lot of gradients in class, so you can just go back and check this and it should work out well ok.

So, you can just see this and this way we can just compute the gradients for all the input pixels.

(Refer Slide Time: 04:53)



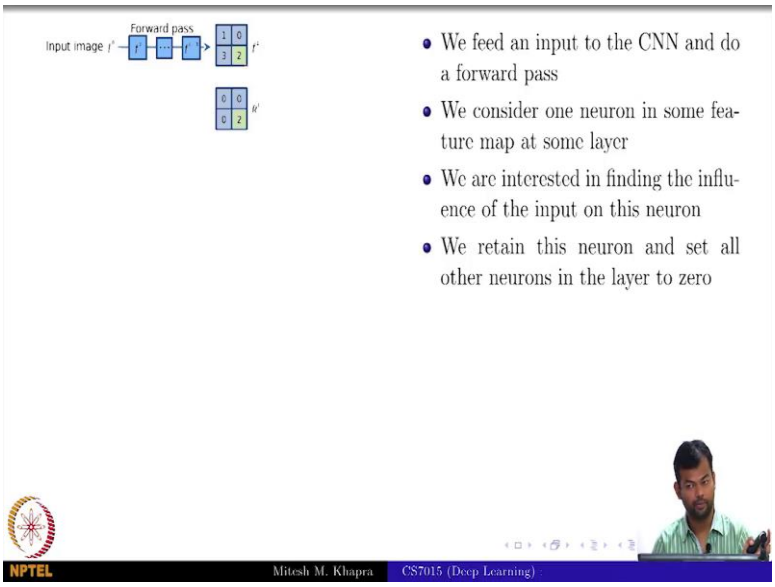
And now I am going to plot it as a image and this is what my image looks like. Do you see what is happening here? Its all very murky right, most of it is great that is fine we expected it. But there is nothing really standing out right even in this patch where you have some non gray pixels, it is almost like the entire cat region is appearing as non gray, the influences are not coming out to be very sharp. We would have wanted something like, only the eye pixels cause some neuron to fire or only the ear pixels cause some neuron to fire and that is not really happening ok. So, it does not produce very sharp influences. So, someone proposed, something known as guided back propagation, which we are going to see next and that helps you to better understand the influence of the input pixels.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 97
Guided Backpropagation

So, we will see what guided Backpropagation is. So, idea here is a bit hacky a bist heuristically, but it still works very well so, let us see what it is right.

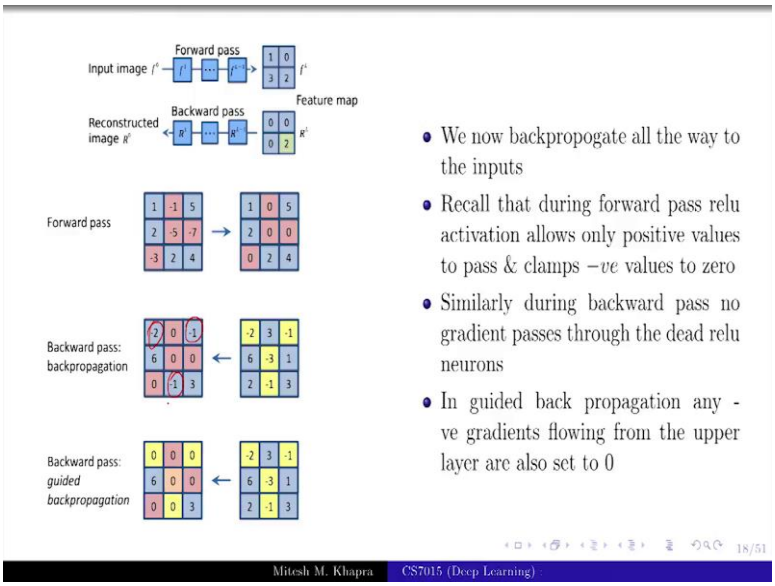
(Refer Slide Time: 00:20)



So, suppose you feed an input image to a convolutional neural network, that image will go through all the convolution layers and say it one convolution layer this is what your feature map looks like. I am operating at a very small scale, I am just considering a 2×2 feature map ok.

Now, we consider 1 neuron in some feature map at some layer. Ok so, we will consider this particular neuron. And we are finding interested in finding the influence of the input on this neutron. So, this is what I will do is, I will set all the other neurons in this layer to 0 because, I do not care about them, I only care about this particular neurons, I just focus on that.

(Refer Slide Time: 01:00)



And we now back propagate all the way back to the image right; that means, I will compute if I call this as h_2 then, I will compute $\partial h_2 \times \partial i_1 i_2 i_3$ and so on ok.

Now, recall that during forward pass what happens is because you have relu neurons, any output which was negative that was clamped to 0. In the forward pass any output which was negative was clamped to 0, so what would happen to the gradients when they flow back, through those neurons? You already did this if an relu neuron is dead, the gradients do not flow back right. So, the gradients will not flow back through these neurons; that means, that only the so, only these gradients will actually flow back, which correspond to non negative entries in the image before it or in the matrix above it right. Is that fine?

So, now these guys use this interesting idea that in the forward pass you dont allow negative things to go forward. So, the backward pass also do something similar, dont allow the negative influences to go back; that means, any gradient which is negative just clamp it to 0 ok. So, what I am going to do is, all these negative elements in the gradient, I am going to set them to 0, you see that? So, this is just taking the same idea which you apply that forward propagation that relu clamps the output to 0 if, the influence was negative and the backward pass also do the same, any gradients which are negative just clammed them to 0.

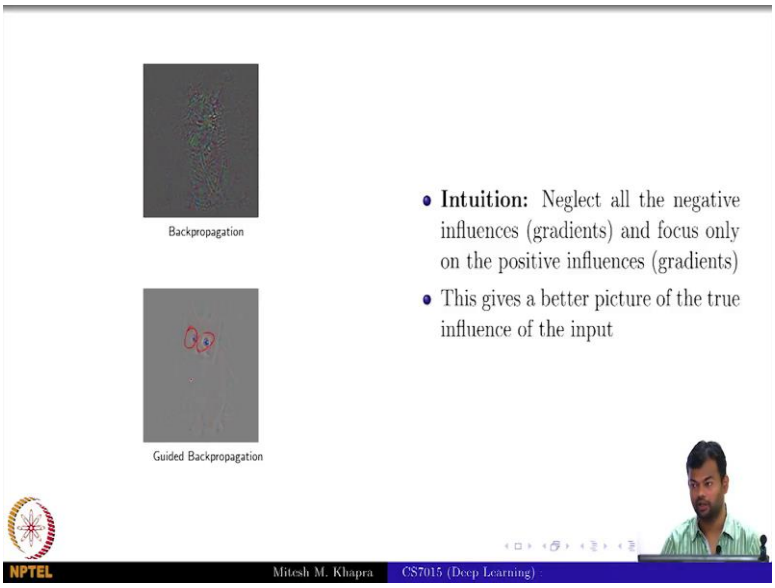
So, the intuition here was that maybe there was a pixel which is really influencing the particular neuron and it stands out, but because there are some positive and negative

gradients flowing back, they seem to cancel each other. And all these influences tend to be 0 because, thats what we observe that image was largely gray with very few non gray pixels.

So, this is very heuristically because, the reason I call it a heuristic is because, you are messing with the math right, the math tells you that the correct gradient has to go back irrespective of whether its positive or negative but they give this justification that on based on two things. And the forward pass you are not passing the negative gradients, a negative outputs, so in the backward pass also kill them and this should avoid this canceling of positive and negative output.

So, this is known as guided back propagation because, you are meddling with the actual back propagation, you are doing something different.

(Refer Slide Time: 03:24)



And, so the idea was to neglect all the negative influences and when they apply this guided back propagation, this is what the influence looks like. So, you see that it is much sharper now, it is actually very nice its focusing completely on the eyes and you can see the layout of the cat much more clearly as in the earlier picture earlier image right.

So, this is a popular technique to use to for various things it is also among other things, for in for understanding what your convolutional neural network is doing right. So, this lecture is entirely about understanding what are the neurons learning, what are the weight

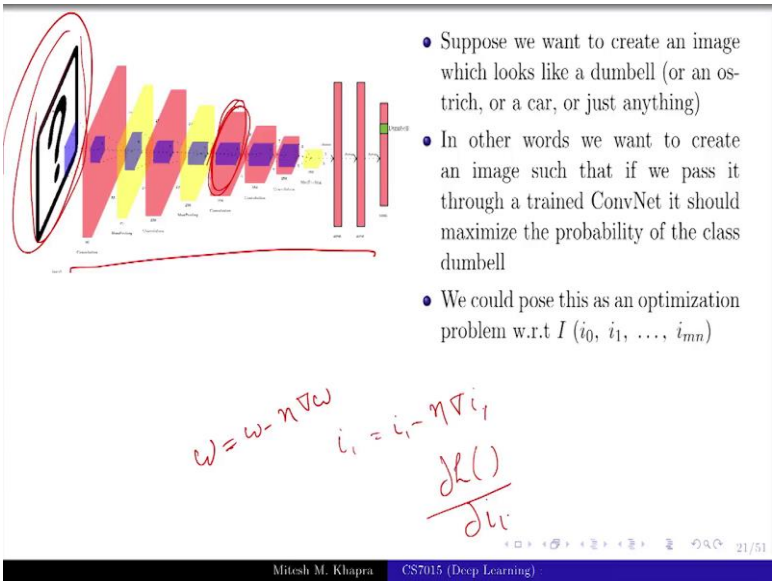
matrices learning, what are the kernels learning and so on. So, these are all again tricks that you need to have in your repository to be able to do something more than just reporting accuracy ok I will get 70 percent accuracy on this status (Refer Slide Time: 04:15) right. So, this guided back propagation is one algorithm that you will implement as a part of the assignment so.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture-98
Optimization over images

Ok. The next thing that we are going to do is Optimization over images. So, this is again interesting and it eventually led to this whole field of adversarial deep learning or adversarial machine learning in general right. So, we will see what this is.

(Refer Slide Time: 00:26)



Suppose I have a trained convolutional neural network ok and now I want to figure out what kind of image should I pass through this, so that, it gets recognized as a dumbbell. Why we want to do I would not want to have such a weird objective, can you think of a reason, why would want such a weird objective? I know there is a convolutional neural network, which can distinguish k classes these classes could be anything.

Now, I want to deliberately create images, which get passed as the dumbbell class. Why would I want to do this? Ok you are going into your details. So, I will give you a application right suppose, this network is supposed to do face detection and the k classes which are there are k people right. Now, you want to see what kind of image should I feed to this so that, I get recognized as Amitabh Bacchan right. So, now, that could have certain benefits and various high places and so, on its I would want to do that right.

So, thats the whole idea behind adversarial learning. So, now, I am asking this question that, I want and here its in of course, a toy setup there is no reason, I why I would want to generate dumbbells, but say if I am going to if its an automatic verification whether, my product looks like a dumbbell or not I might want to do this right. So, you could think of all sorts of reasons why you want to do this. So, what we will do is, the question that we are interested in is that, I have a blank slate with me, it just contains some pixels. I want to be able to modify this pixel so that, my class dumbbell class gets fired.

Now, we have done enough gradients, enough back propagation everything in this class. So, I will ask you to give me a solution for this. And the hint is, treat the image itself as a parameter matrix. The second hint is assume that all of this is going to remain constant, you are not going to change any of this. And you have initialized your parameters, which is the image pixels to 0s; that means, you are started with a gray image.

Now I will change the question a bit, only a bit and all of you will be able to answer this ok. Suppose my network is strained and now, I want to change the weights in this layer so that, my accuracy improves. So, that when its a dumbbell class, it predicts dumbbell. How will you do that? It will pass the same image, what will you do, how will you change the weights in this layer, back propagation what is the update rule, say the gradient descent update rule, say that the gradient descent update rule.

Student: (Refer Time: 03:01).

W is equal to ok, you guys actually unanimously said gradient is an update rule ok. So, w is equal to w minus oops oops oops, ok, minus eta into, ok,

Student: (Refer Time: 03:16).

thats what you will do. Now if I ask you the question for this you can answer it, but if I ask you the same question here, why cannot you answer it.

Student: (Refer Time: 03:27).

So, here what were you doing, computing the gradients of the loss with respect to the weights what will you do here?

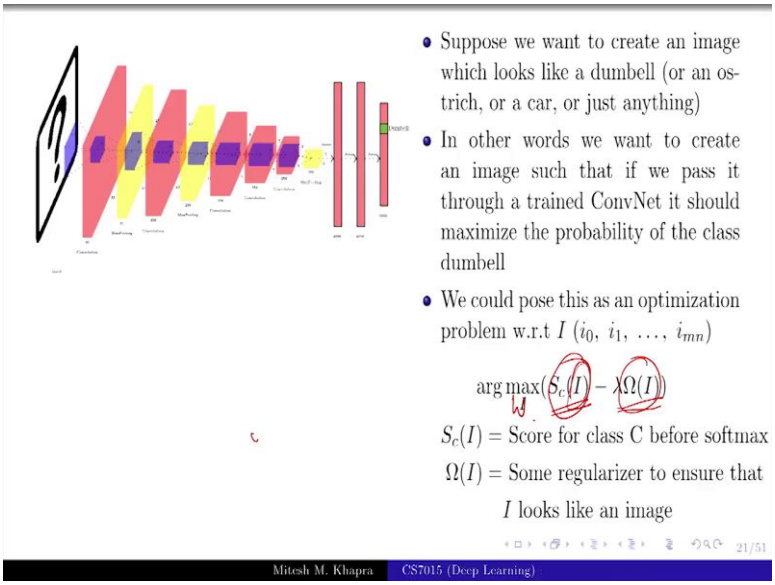
Student: (Refer Time: 03:33).

It put respect to each of these pixels and then, update this pixel by using what formula?

Student: (Refer Time: 03:39).

i1, thats the first pixel is equal to i1 minus eta gradient i1 where, what is gradient i1 actually? Everyone gets the intuition right you can do it now.

(Refer Slide Time: 03:59)

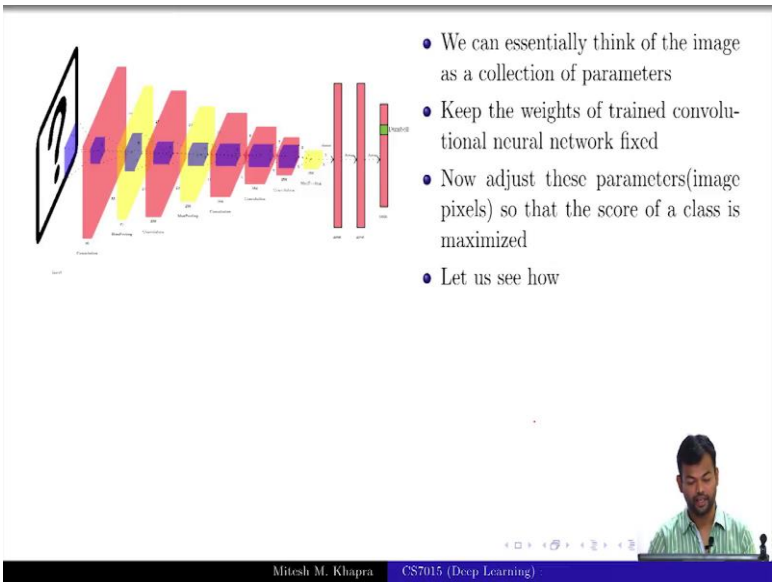


So, we could pose this as an optimization problem where what we want to do is, given an image, we want to maximize the score of the output class. And I also want some regularization because; whatever I get I want it to look like an image right.

So, we will see different types of regularization for doing this, some very simple regularizations but this is the overall idea right. So, any generic loss function is always the training loss plus the regularization So, I have just kept both, the training loss as well as the regularization. Whats my training loss? The score for the class that I am interested in. And what are the parameters of this object of this optimization problem, the input pixels right.

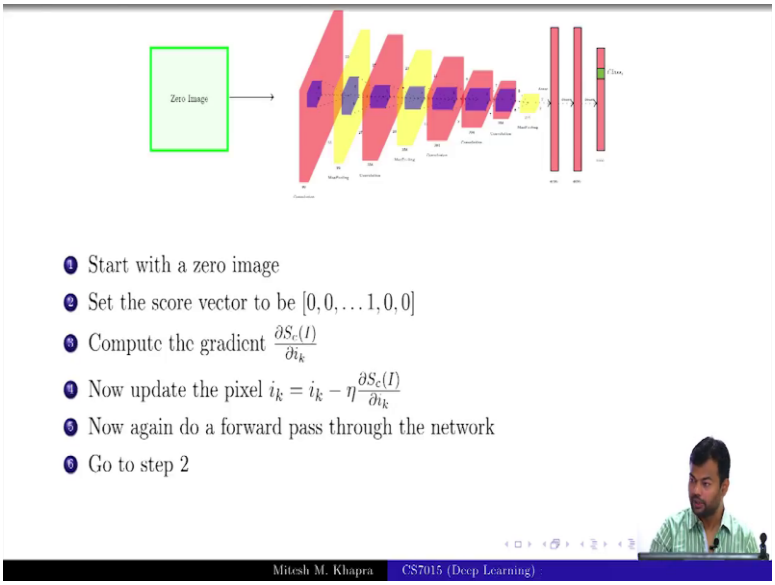
So, far we had already be always been doing w, b but instead of w, b you now have I as the parameters of your optimization problem, is that fine?

(Refer Slide Time: 00:46)



And now, we can just think of the entire image as a collection of parameters, and we can now update the weights of this matrix, which is the image matrix ok.

(Refer Slide Time: 04:58)



So, let us see how we will do it. So, we start with a 0 image as I said. Set the score vector to all zeros and 1 for the class that I am interested in ok.

Now, compute the gradient of this score vector with respect to i_k, its I want this quantity to be maximized everything else to be 0. So, thats what my loss function is. So, I am

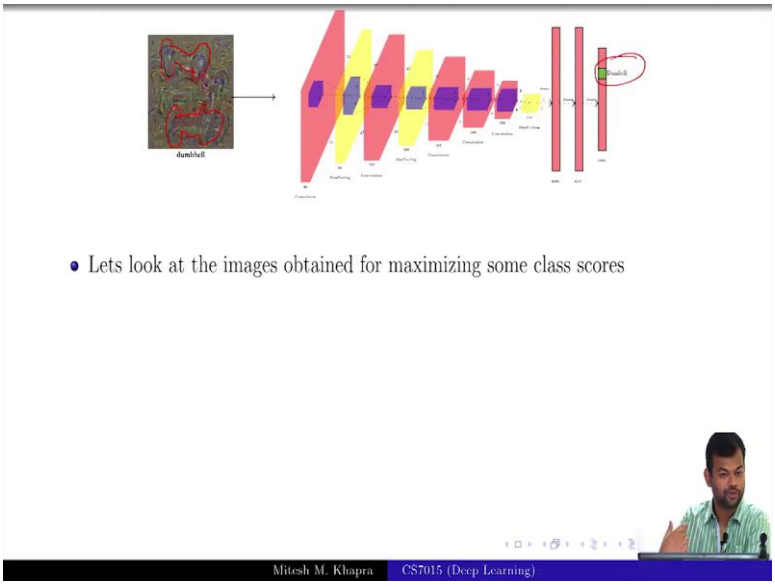
going to compute the gradient of each of the pixels with this. Now I am going to update the pixel using my gradient descent rule, which I just explained brief previously.

Now I again do a forward so, now instead of this 0 image I have a modified image, slightly modified image because, the pixels I have moved away from 0 update based on the gradients. Now this image I will pass back through the network and what will I do now? Again change, so this is the same as the weight matrix right, so you should be able to visualize it exactly the same way as you would have visualized this. You had certain weights here, you change them a bit, again did the forward pass, again did the backward pass change them a bit and keep doing this till.

Student: (Refer Time: 05:59).

Till convergence right, whatever is your definition for convergence till you are satisfied and instead of score of 1 you are at least getting a score of 0.9 or 0.95 or something like that right. So, we will keep doing this right till convergence. At the end you will you all of you can imagine that this image will keep getting modified ok.

(Refer Slide Time: 06:20)

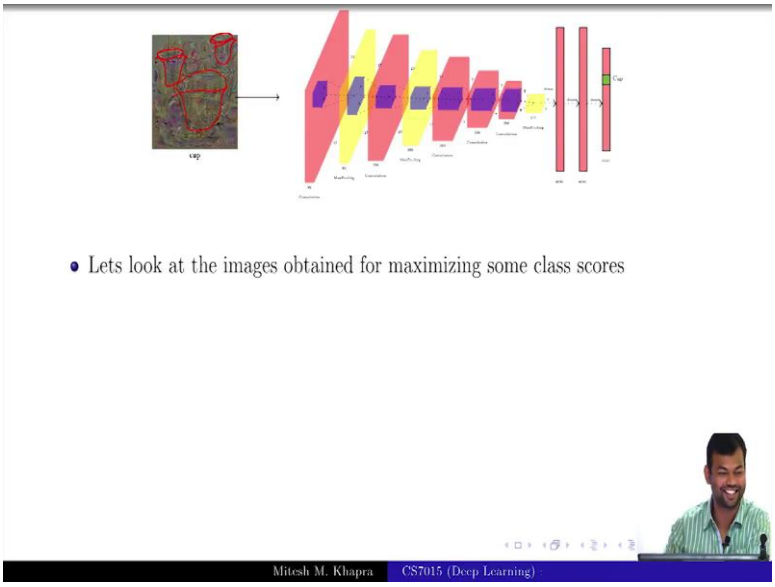


So, now let us see if, we learn run this score or the run this code for certain classes. So, I mean interested in the dumbbell class and I have ran that algorithm starting with the 0 image and this is the kind of image that I end up with. You see a dumbbell here, without me drawing it right, if you go back and look at it, you will see that there are a lot of these

dumbbell like shapes which have actually appeared here. The colour is of course, very much different I dont think dumbbells are of these colours ever, but you can see that its actually trying to produce that shapes, which will cause the dumbbell output to fire.

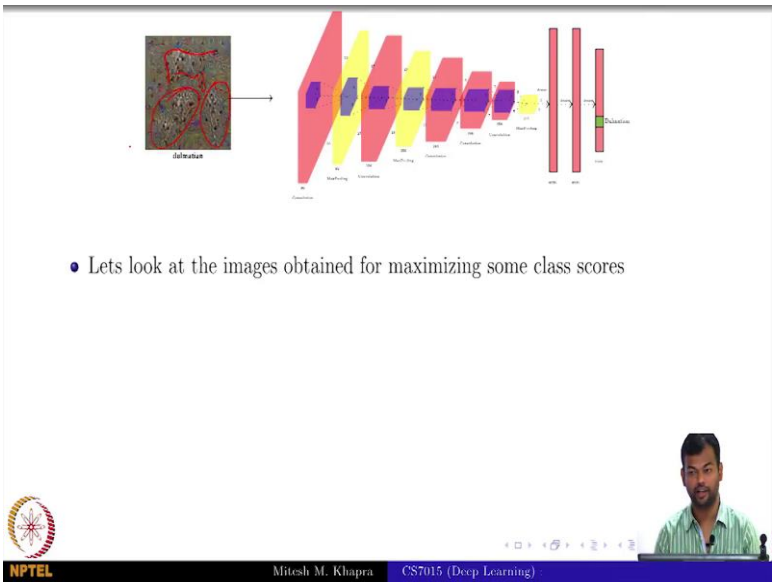
Now, what is interesting is that, its being very redundant. So, its not trying to generate a single dumbbell, a generating a lot of dumbbells of different orientations. So, i just keeping its basis covered so that, some of this should actually fire and cause a dumbbell output to be maximized ok.

(Refer Slide Time: 07:14)



Now, let us see if we take a cup and this is like the trophy cup I believe. So, this is what is appearing here, there is one more cup here and there is one more cup here. its a generating these cups so that, you cant be, you would not be able to see it, its different oh it really looks like I am manipulating it, but I am not, you can go back in check it, those cups are there ok.

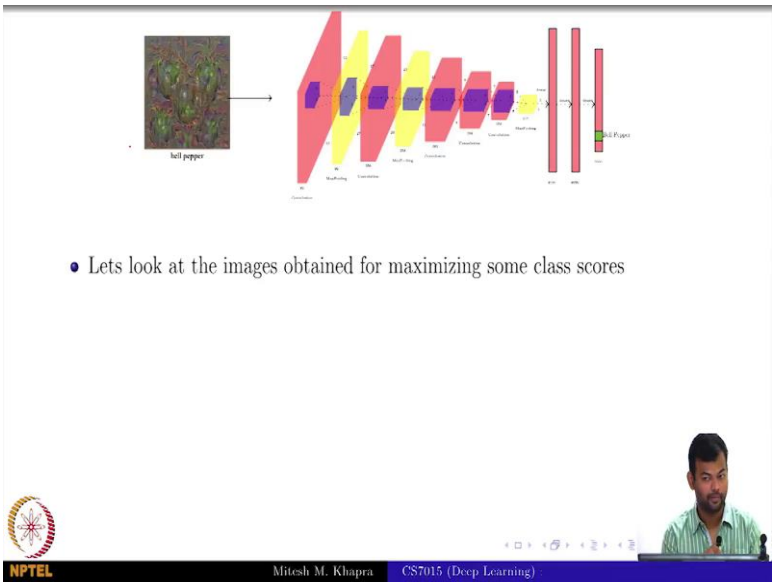
(Refer Slide Time: 07:41)



And then, for dalmatian actually, this at least you can see some white and black spots right at least thats fine.

So, dalmatians are these dog which have these white and black spots. So, and you can also see some kind of a shape here right, which with my drawing. So it is actually producing that doglike shape and its producing multiple of those .So, its being redundant I am trying to compute that right.

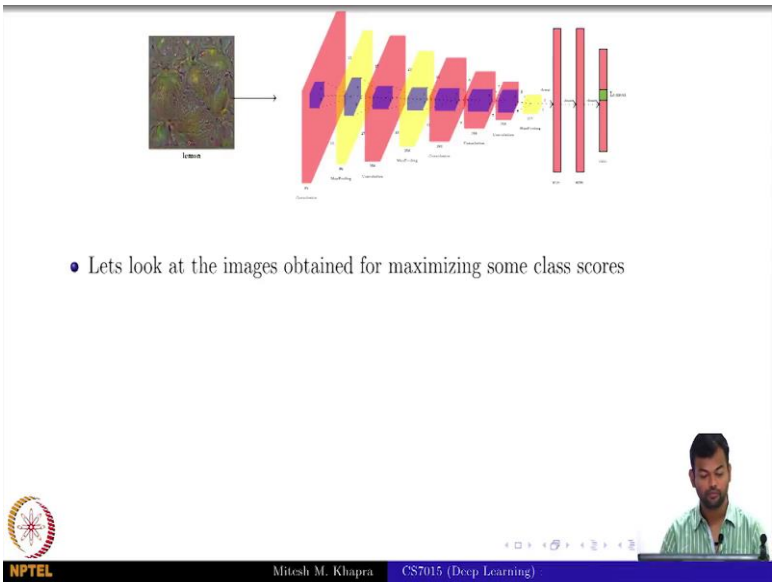
(Refer Slide Time: 08:04)



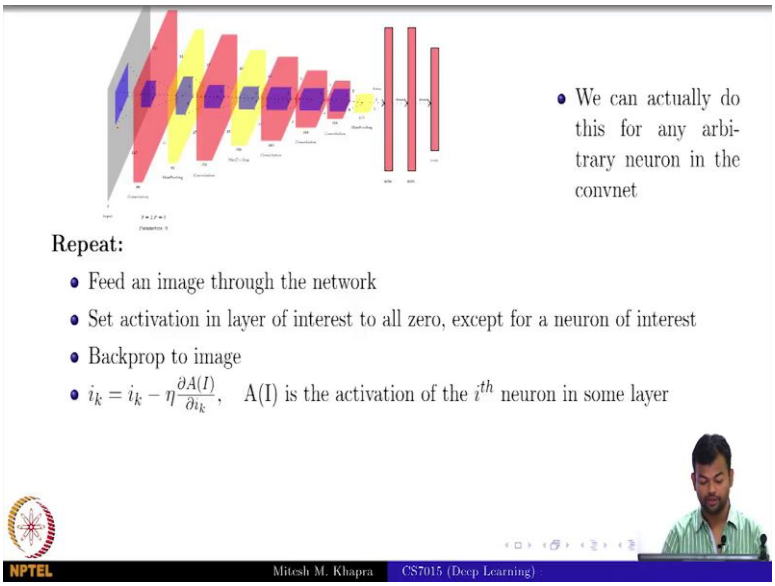
And now you see right with these very arbitrary images, which to you and me do not know nowhere close to we will fire will classify this as dalmatian. But for the machine and is classifying this as a dalmatian and this is bad right, this is not good, there is nothing to be impressed about this is actually bad because I can give it these horrible images and still get away by something called as a dalmatian.

So, if I want to sell some a dalmatian on OLX, this is what I can do right. I can upload this image and a machine would trigger it and some one would buy it ok. So and this is a bell paper so you can go back and see you see a lot of bell papers here and similar for lemon and so on right.

(Refer Slide Time: 08:41)

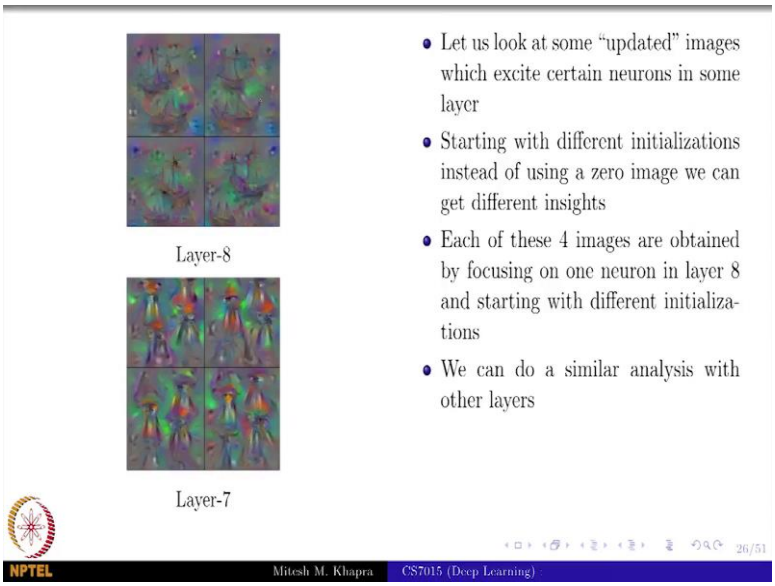


(Refer Slide Time: 08:45)



So, various classes you can see that, its actually trying to produce those shapes, but its nowhere actually producing a clear image, which is undoubtedly of that object right, is generating something which can later on be used to fool the network right, which is not a good thing ok. And we can actually do this for any arbitrary neuron, so I was trying to actually fire this neuron, which was the output layer, but maybe I want something else to fire here so, I want to actually see what is, it that causes this neuron to fire. So, I could repeat the same algorithm by setting something here as high and then again back propagating the gradients only from here, and reconstructing the image every time so that, this neuron then 5 is right.

(Refer Slide Time: 09:25)



So, these are what the updated images look like, which excite certain neurons and some layers. So, what does this look like? its actually like a pirates ship if its not very clear, you have these multiple layers of things and something like this ok. So, its some neurons are actually firing for this kind of a pattern; there are some other neurons which are firing for different kinds of patterns and so on right.

So, you can just create images which cause certain neurons to fire and all these are lot of fun to do. So, you should I would encourage you to do this, I will get more insights into what your network is.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture-99
Create images from embeddings

The next thing that we will see is how do you Create images from Embedding. So, let me see what that means.

(Refer Slide Time: 00:18)



So, remember that each of these things can be thought of as an embedding of the image right because, you had this original image which was 227×227 dimensional and now you have a 4096 representation for that or a $256 \times 7 \times 7$ representation for that, so you could just flatten it as an out as a vector and you could treat that as a embedding for the original image right.

Now, for any kind of embedding or hidden representation, what do we always want from that representation? Think auto encoders, it should capture all the important characteristics of the original image and in particular I should be able to dash the original image from it.

Student: (Refer Time: 01:03).

We construct the original image from it right. So, thats what I would want from a good embedding. So, let us see if we can do this right, so find an image, this is the optimization problem that I am interested in. Find an image such that, its embedding a similar to a given embedding. What do I mean by that is? Suppose, I take a monkey image and pass it through all these layers and compute all these embeddings right.

Now, again I start with a blank image and my optimization problem is such that, for this blank image I want to modify it, so again this blank image is my parameter matrix and I want to modify it such that, the embedding that it produces should be similar to the embeddings that the monkey image produced. So, how can you set this as a optimization problem, what would your loss function be. So, lets call the original monkey images I1 and let us call this as embedding of I1.

Now, can you tell me what the objective function would be? For the new image that you are trying to create, this entry the first entry in its output that let me call that e I 2 ok. So, e I 2 1 and ei 1 1; that means, the first dimension of the embedding they should both be

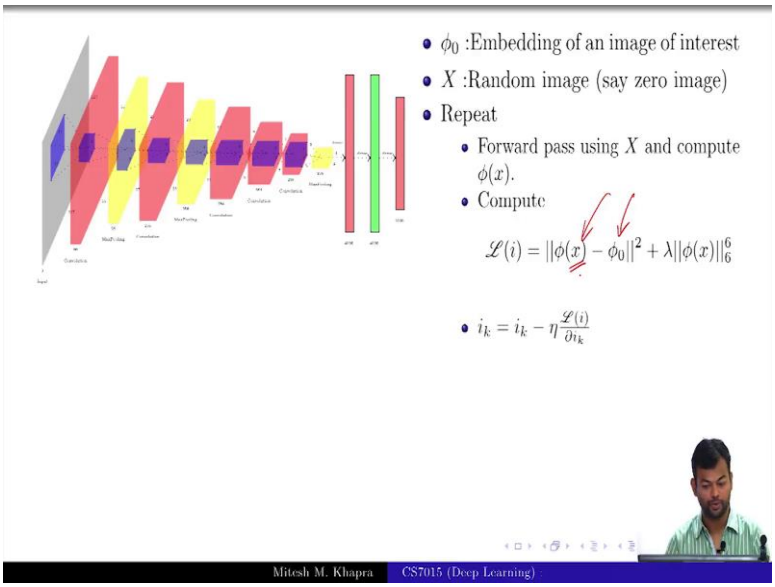
Student: very

Very close. So, in such cases what is the error function that we will choose?

Student: (Refer Time: 02:29).

(Refer Time: 02:30) right. So, you have to get comfortable with designing these loss functions right, so you have seen, you have seen this loss function before, we just have to be able to related to the problem that you are trying to work on

(Refer Slide Time: 02:40)

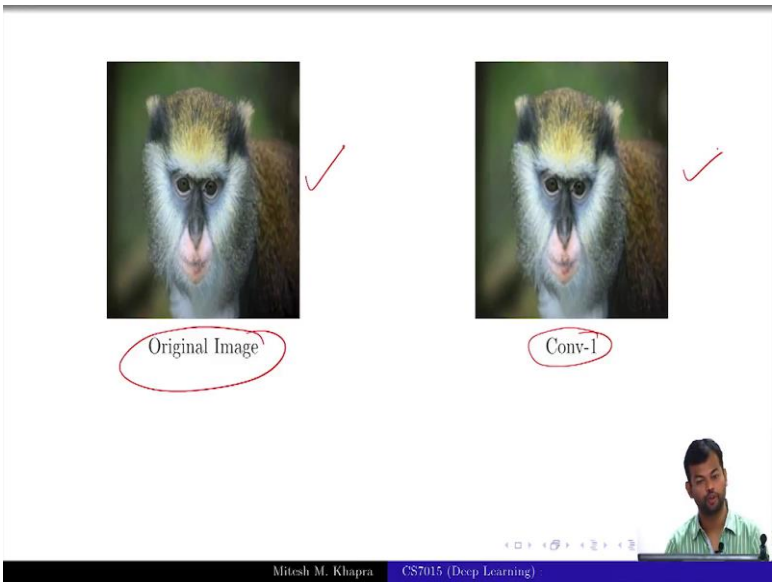


So, let phi 0 be the embedding of the image of interest, let X be a random image and we will report the repeat this. Forward pass using X and compute phi of x right; that means, were computing the embedding of this random image that we have started with. Then we compute this loss function and add appropriate regularization for that and that propagate and update what, what will you update?

Student: (Refer Time: 03:06).

Image right, you will update your x matrix right and you will keep doing this till convergence.

(Refer Slide Time: 03:13)



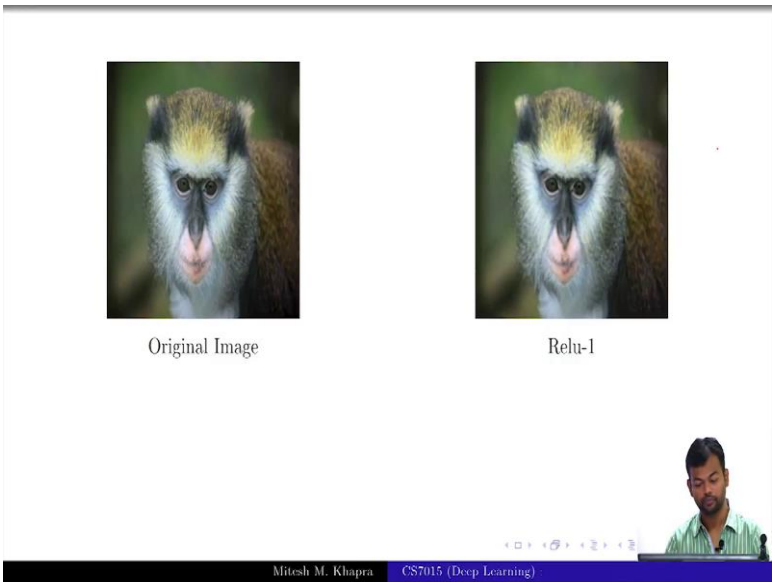
And let us see what happens, so its suppose so now, what I am trying to do is, this is my original image and I have the convolution one embedding of it. So, in this I am using convolution one as the embedding and then, I am trying to solve this optimization problem to recreate x such that, its very close to the original image.

So, let us see what are the different outputs that I get. So, this is the original image and on the right hand side you have the reconstructed image such that, the Conv 1 embedding of both the images is the same. So, you can see that when I am trying to do a reconstruction from the Conv 1 layer, I get almost the same image back. Now, if I keep doing it from different layers what do you expect it to be, if i do it from Conv 2, Conv 3 Conv 4 and so on.

Student: (Refer Time: 03:59).

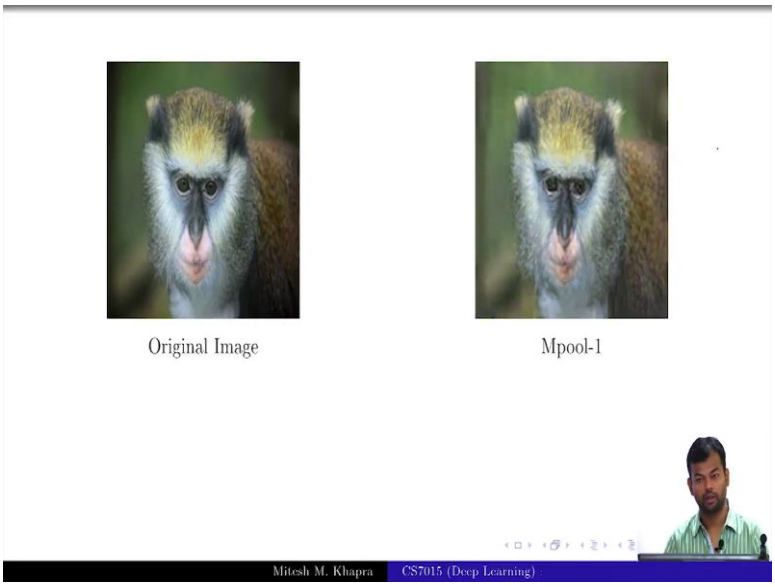
It wont be so accurate right. So, let us see what happens if, I try to reconstruct it from Conv 2

(Refer Slide Time: 04:05)



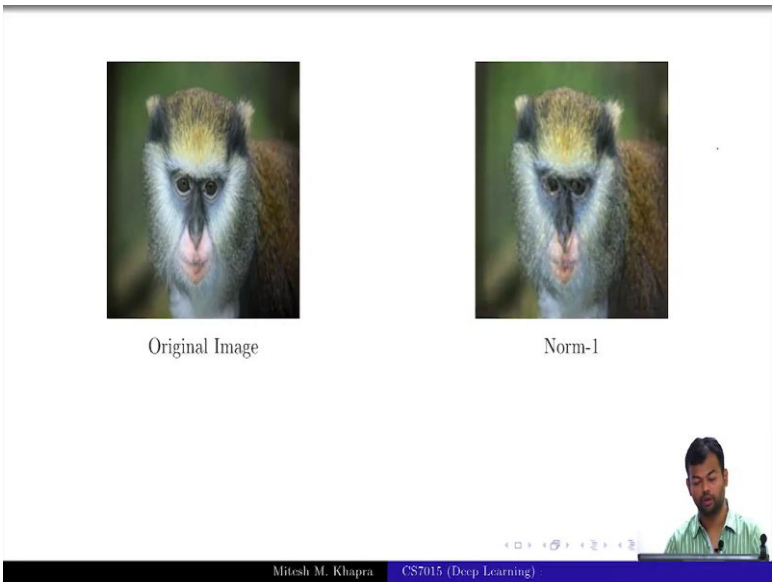
Ah relu 1

(Refer Slide Time: 04:07)



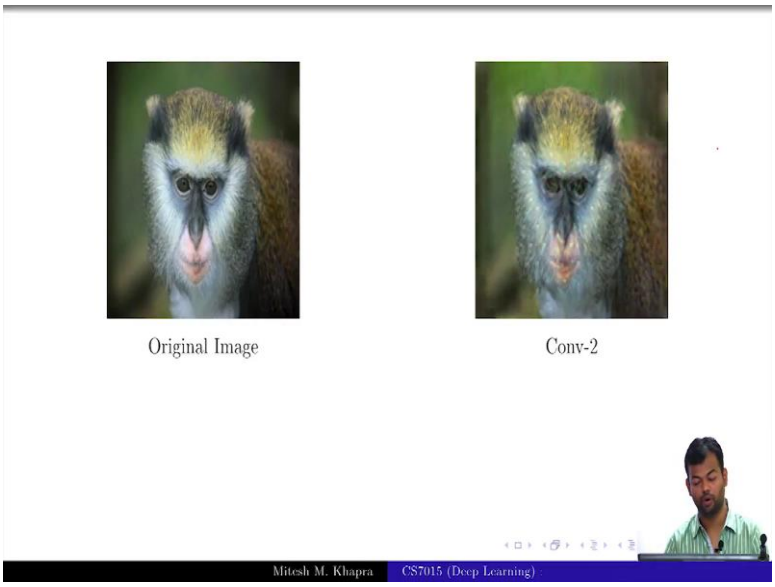
Max pooling

(Refer Slide Time: 04:09)



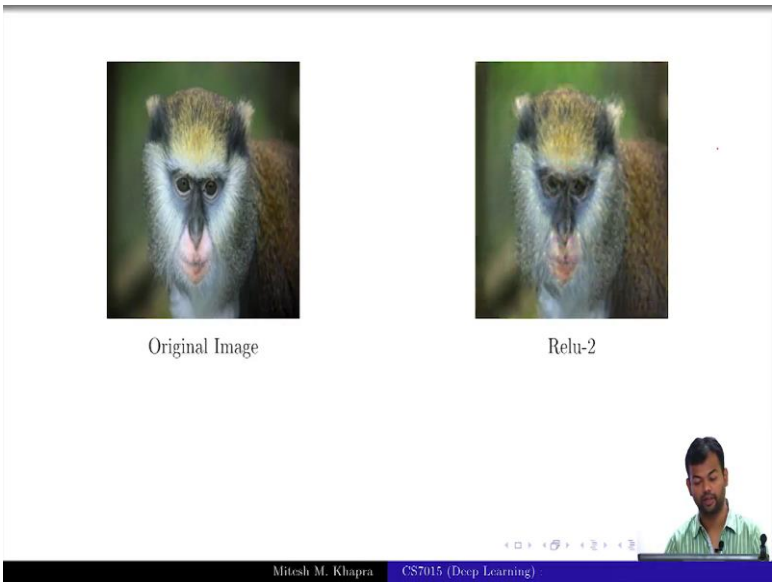
Norm 1.

(Refer Slide Time: 04:09)



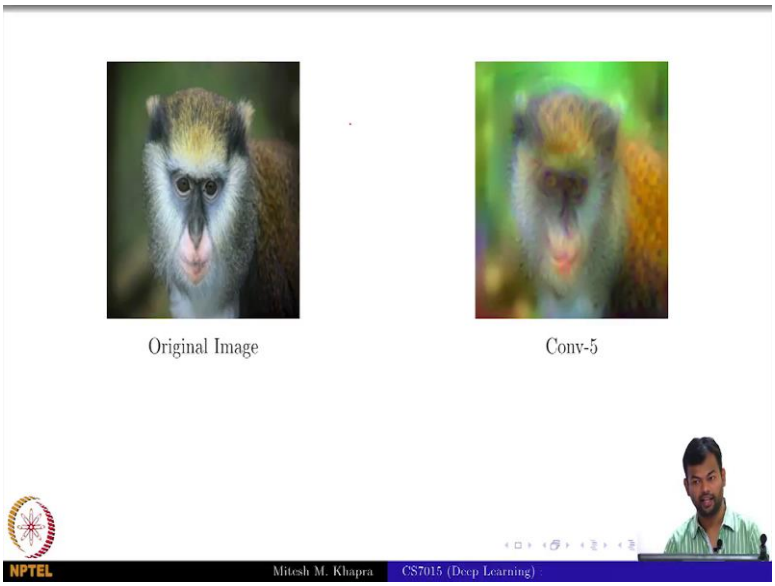
Conv 2.

(Refer Slide Time: 01:10)



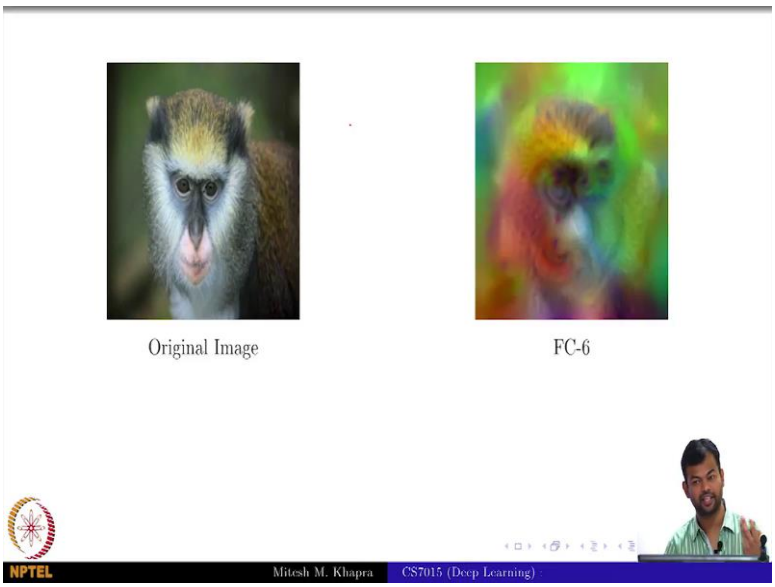
Relu 2, I am keep I am going deeper and deeper into the network. So, what I am trying to do here is remember that, I have different choices for these embeddings. So, the first thing which I showed you was, when I was trying to the first thing was when I was trying to set my objective function such that, I am trying to map this embedding. The second image that I showed you was, when I was trying to map this embedding and the last image that, I will show is when I was trying to map these to embedding. So, my objective function was to create an image such that this embedding of the created image is the same as this embedding of the original monkey image right. So, thats what I am progressively trying to do.

(Refer Slide Time: 04:52)



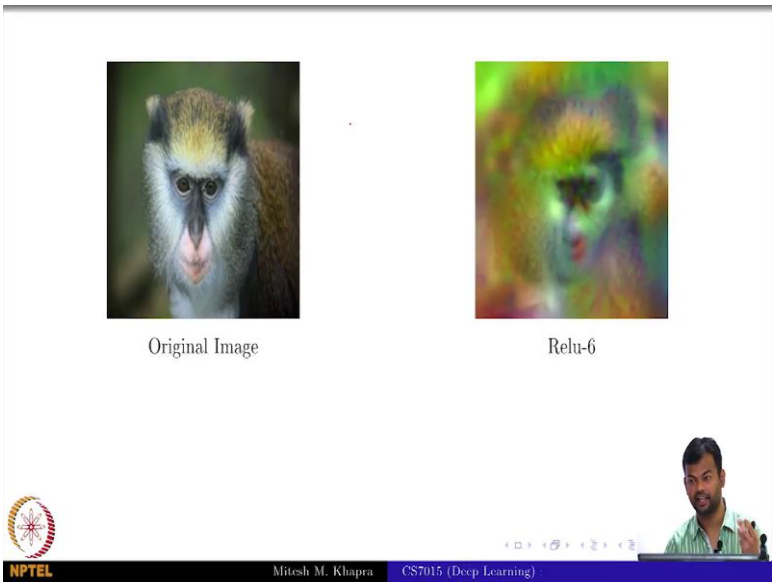
As you can see as I keep going ahead, I get more and more abstracter reconstructions and I dont really get the monkey back.

(Refer Slide Time: 04:57)



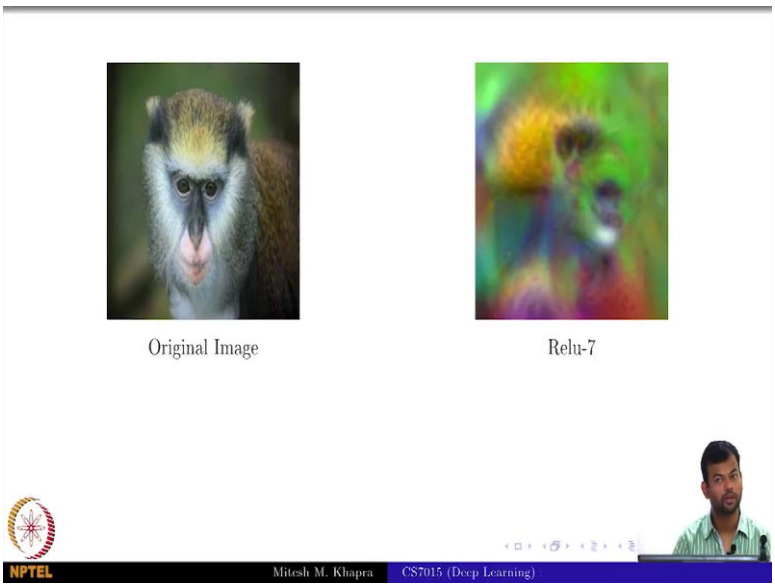
And once I go to the last FC 6 or F 7 layers, I get very weird looking reconstructions.

(Refer Slide Time: 05:00)



And thats expected right, because by that layer they have completely abstracted it out right, you have just probably captured there is something like a nose, something like eyes or some for here and there, but you have loss the entire shape and other characteristics of the original image right, from the deeper layer the construction would not be that good and thats kind of expected right.

(Refer Slide Time: 05:21)



In spite of having the maximum no you could right. A maximal operation is just another embedding, which is that the compression there is much more because, you have ignored

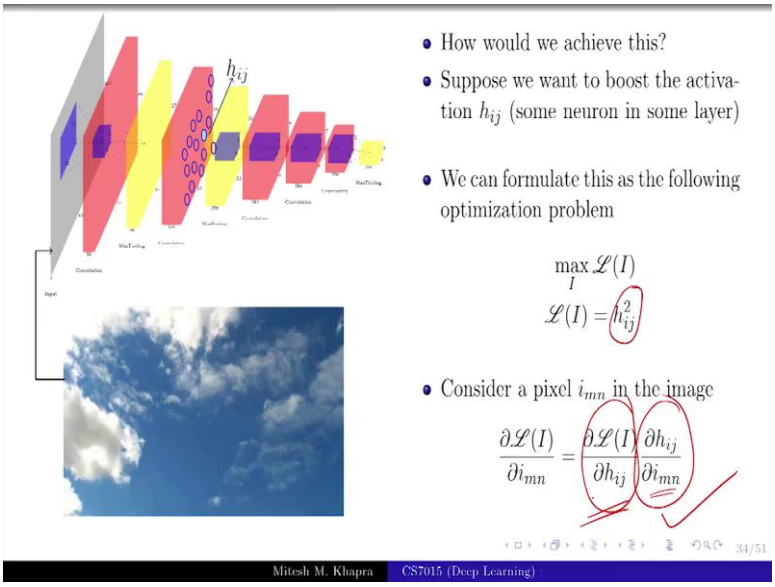
the 4 entries and just taken the max value. So, it becomes harder and harder to do that, but mean you I wouldnt call this as a reconstruction right, what you see here is not except for the Conv 1 and Conv 2 layers, the rest of the things were not really such accurate reconstruction. So, just says that you are losing a lot of information in that abstraction or maybe not, I will do it next time.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 100
Deep Dream

So we will start. So we were in the 3rd lecture on CNN's. Where we were looking at different visualization tools for understanding what your convolutional neural network is learning. And we did a bunch of things and now you move on to the next module where we talk about something known as Deep Dream very interestingly titled. But I am sure most of you have already seen this or read about this.

(Refer Slide Time: 00:34)



So, here is the idea right so, far we were seeing that if we start from a blank image, then we could suitably modify it by constructing an optimization problem whose parameters are the pixels of the image. And we can modify the image so that it starts looking like a certain class of interest right. But now suppose instead of starting with a blank image, I start with a natural image right, say a sky or any image that you have in your dataset.

I start with this and then I focus on neurons in some layer of the convolutional neural network. I am focusing on these neurons say any one of these neurons I am focusing on. And I want to change the image so that these neurons so when I say neurons I actually mean only a single neuron, but for illustration I will show multiple neurons. So, I want to

change the image so that this neuron fires even more. So, how would we achieve this? What will we do?

So, say this is the neuron which I want to fire even more. So, what is my optimization problem, first of all what are the parameters of the optimization problem?

Student: (Refer Time: 01:51).

The pixels of the image, that is clear. Now I want this to fire even more. So, what is the objective function? What you are going to maximize? Lets call this neuron as h_{ij}, what you are going to maximize.

Student: (Refer Time: 02:18).

Sorry.

Student: No (Refer Time: 02:21).

No I want this neuron to fire more.

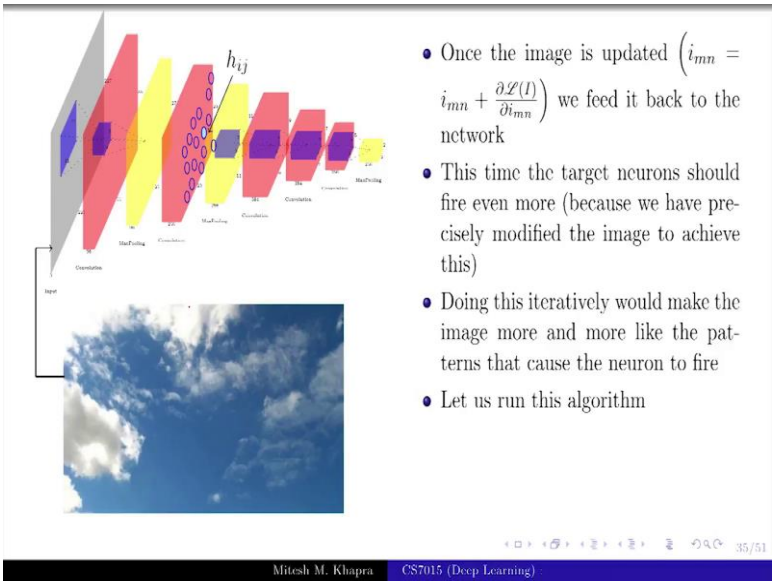
Student: (Refer Time: 02:27).

Maximize hij right, I mean that is I mean why so that sort of a thing. Ok.

But of course, we will do something so that it is a neat differentiable thing and so on. So, you want to maximize the activation of one such neuron h_{ij}. So, we could just formulate the following optimization problem that I want to maximize h_{ij}^2. Ok, and of course, the parameters of the optimization are the image pixels and if I consider one such pixel in the image then I essentially need to compute this gradient.

The gradient of the loss function, which is h_{ij}^2 with respect to this image pixel and I can do it in these two parts. The lead ability of the loss function with respect to h_{ij} and the derivative of the h_{ij} with respect to the image pixel. This we have seen a million times while doing back propagation of course, you are not gone all the way back to imn, but we saw last time that it is just one more term in the chain rule. And this again looks straightforward right the derivative of the loss function with respect to h_{ij} looks straight forward. So, I have a very simple way of computing the derivative of the loss function with respect to any pixel of the image.

(Refer Slide Time: 03:31)



So, now, I can apply gradient descent and I can update the image. So, I started and now remember that the my original i mn was not blank or random or 0 or anything. It is actually the sky image so maybe it was blue or cloudy or whatever pixel that I have in my original image and that pixel I am changing.

So, I have started with the sky image, I have changed a bit based on this gradient update rule; gradient descent update rule. And now I feed it back to the network, what will happen? What will happen to h_{ij}?

Student: Fire a bit.

It will fire a bit more because that is exactly how you have changed the image with exactly that objective function right.

And now if I keep doing this what will happen? So, remember what does h ij actually capture now this is where. So, if you understand this right you will really understand and appreciate everything about convolutional neural network. And I will be sure that you are actually understood the details and not just these boxed architectures right. So, what if this happens right then what does actually h_{ij} capture? It captures certain.

Student: Patterns.

Patterns in the image right. Now if h_{ij} is firing; that means, these patterns have started.

Student: (Refer Time: 04:57).

Appearing in the image we started with a sky image.

And now hij is firing more and more; that means, it is now the image is suddenly becoming more and more or containing more and more patterns for which h ij should fire; does that make sense? Ok, yeah. So, let us run this algorithm we will start with this image and we will run this algorithm. So, I will run it before that I want some guesses; what kind of patterns do you think will start appearing here and this is deep dream is the title right ok so fine.

So, let us see, so I will run this algorithm so what I am doing is I am starting with this image and running exactly what I showed you that I will compute the gradient with respect to one of the neurons. And I will keep updating the image so, that it becomes more and more like the patterns that I am trying to capture. So, lets run this and observe carefully it is almost a magic trick. I hope this does not disappoint, what do you see.

Student: (Refer Time: 05:51).

Most of them are what?

Student: (Refer Time: 05:53).

They are dreaming so they are literally building castles in the air right. So, what is happening, why is this happening? Everyone sees castles right that is the first thing otherwise.

Student: [laughter].

Ok. Good. why is this happening? Have you seen the Disney logo the castle what does it have in back background. How many of you find this interesting. How many think this is ok, expected. Ok, why is this happening? Think about training data, think about what would have happen or you missed the magic show. So, what is the convolutional neural network actually trying to do.

Student: (Refer Time: 06:32).

I will give you a hint its being over enthusiastic. How many of you get that? Ok, so here is what is happening right. Should I explain it or no, I am not going to ask you a quiz question I am just saying that I have some more images to show, ok I will explain it first. So, this is what is happening right.

So in the training data whenever the castle appears it is typically has the sky as the background ok. So now the convolutional neural network started drawing these correlations. So, whenever it sees a sky it is trying to find a castle somewhere, but because it knows that most of the times whenever I see a sky there is a castle in the foreground.

So, those neurons are firing a bit and then now you are trying to fire them even more and more. So, that keep trying to change the image till this castle actually appears on the image. How do you how many if you get this explanation please raise your hands? Ok. So, let us see some more examples right. So, now guess what will happen here, ships, ok again a generation which thinks of.

Student: (Refer Time: 07:38).

A ships is ok. I shouldnt comment on that.

Student: Birds.

Fishes (Refer Time: 07:43).

Student: Birds.

Birds. What else but there are also mountains.

Student: (Refer Time: 07:47).

Ice, ok interesting.

Student: (Refer Time: 07:50).

Now our expectations are increase. Let me just run this and see what happens oops oh no.

Student: (Refer Time: 07:56).

I have my final trick ok.

Student: [laughter].

The prestige is gone ok. Yeah so what do you see here? So, actually if you go back and look at it carefully right this is very interesting a lot of fish eyes actually start appearing here and some shapes like fishes actually start appearing here. Go back and look at it carefully and all on the mountains and the green regions a lot of birds and animals start appearing right which is again expected. Because in your data set you would have seen birds and animals with a green or this kind of a background right whatever you call it a mix of green and brown background right.

So, now it is trying to find those things even though they do not exist and as it try to force it more and more it starts creating those images as you start asking to dream more and more right and since this is about dreams I could not let this go it has to had inception in that. So, what will happen here now?

Student: (Refer Time: 08:51).

There is actually nothing interesting is this for my own sake that I put this. Unfortunately nothing interesting happened with this.

Student: Oscars.

Wow.

Student: [laughter].

If only, but thats the point right this is so data set specific that it cant really generalize it cannot dream beyond the data set. Actually nothing interesting happens. It is just a lot of these men are wearing brown suits and in the data set unfortunately all brown things were dogs. [laughter] So this is what will happen, we will start seeing dogs appear everywhere, you see one here.

Student: [laughter].

You see many here actually

Student: (Refer Time: 09:31).

It is like a few more and this would have turned into [laughter] something unpleasant right. So, that is what is happening actually see a lot of dogs here, here in many places right.

So, this is, its still running. So, what exactly is happening here the same thing that I had detected right the network has been trained to detect certain patterns dogs, cats, birds etcetera which appear frequently in the ImageNet data and with these backgrounds that I am trying to do or these textures that I have in my images. It starts seeing these patterns even when they hardly exist and now as I start focusing on these neurons which are firing and try to modify the image to make them fire even more. It will start producing these pixels or these images in the original image right.

So, you can read this explanation which is from the Google blog on this they have some really some code and something on this. So, you can just read this explanation if a cloud looks a little bit like a bird. So, that will make it look more like a bird, this in turn will make the network recognize the bird even more strongly on the next pass and so forth until out of nowhere a bird actually starts appearing in the image right. So, that is exactly what is happening so this is deep dream.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture-101

Deep Art

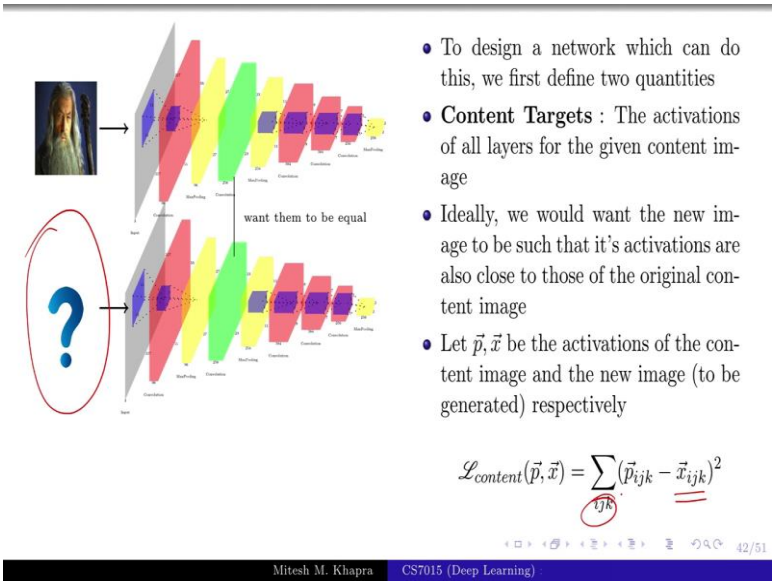
Ok now we will go to deep art. Now here any questions on that? Ok.

(Refer Slide Time: 00:19)



So, now here is what, here is a again an IQ test right, so what will happen ok. So, this is deep art ok, someone wanted to try this that if you take natural images or camera images and if you have art from various famous artists and I want to render this original image in this art form. And how can I do? So, I will explain this, the bit of a leap of faith in what is happening here, but just indulge me right. So let us see.

(Refer Slide Time: 00:47)



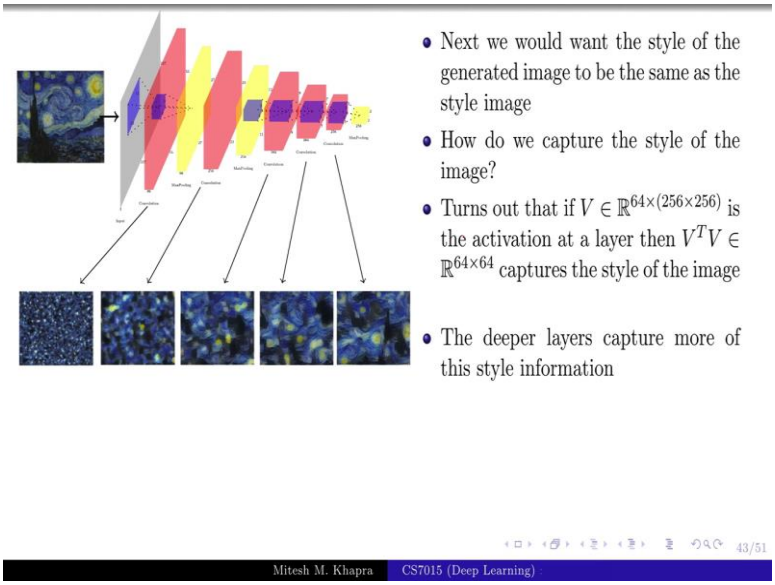
So, to design a network which can actually do this, we design we first define 2 quantities; one is the content targets. So, I call this image as the content image because, this is the content that you are interested in right, I want my final content to look like this. For the content, we would want the following thing that if, I am able to create a new image, when I pass it through the same convolutional neural network; we want these hidden representations to be equal. Right because, that is the assumption here is that, the hidden representation actually captured the essence of the image which is this face and it is various attributes right.

So, if I create a new image in a different style, still this content should be present in it. And my way of ensuring that or rather the way of the author's way of ensuring this was to make sure that, the embeddings that I learn for the new image and the original image are the same ok. So, I want these to be equal and I have just shown one for illustration, but you could have the same objective function for all the representations right, remember that we learn multiple representations and a convolutional neural network.

So, this is what my objective function would be for the content. I would want that this tensor which is the volume ijk, every pixel or every feature value in the tensor for the original image should be the same as the generated image ok. And again my optimization problem is with respect to what image, I am going to change the image and this is the loss function that I am interested in. Is that fine? Ok? Fine.

So, I think x is my original image and p is the new image which I am going to create right.

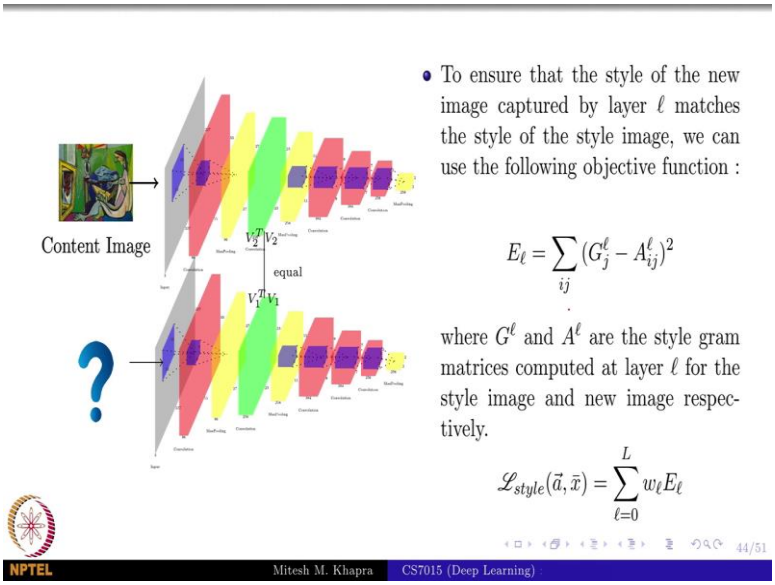
(Refer Slide Time: 02:27)



Now, next and here is where there is a bit of leap of faith. We want the style of the generated image, to be the same as the style image. So, I gave you 1 content image and 1 style image. So, for content the loss function is clear. Now, for style how do you capture the style of the image? So, the explanation given here and I am not very sure about this, but maybe it comes from some traditional computer vision literature, but I just take it on faith that if, you have this volume here, which is say 64 × 256 × 256 or any other dimension right then, $V^T V$ which is a 64 × 64 dimensional image or matrix, captures the style of the image. So, this is what has been written in the original paper, I am not really dug deep, but my feeling is it comes from some of the traditional literature from computer vision right. So, that is not important, we will just take it for granted that that gives the image and here is the illustration for that. As you go deeper and deeper, so this is if you plot the 64 cross 64 image that you got, then you get different styles as you go deeper and deeper you get a better representation of the style of the original image right. So, that is the argument made in the original paper.

Now, if you assume that this is correct then, can you design a loss function for the style part of it? I want the style of the created image to be the same as the style of the style image.

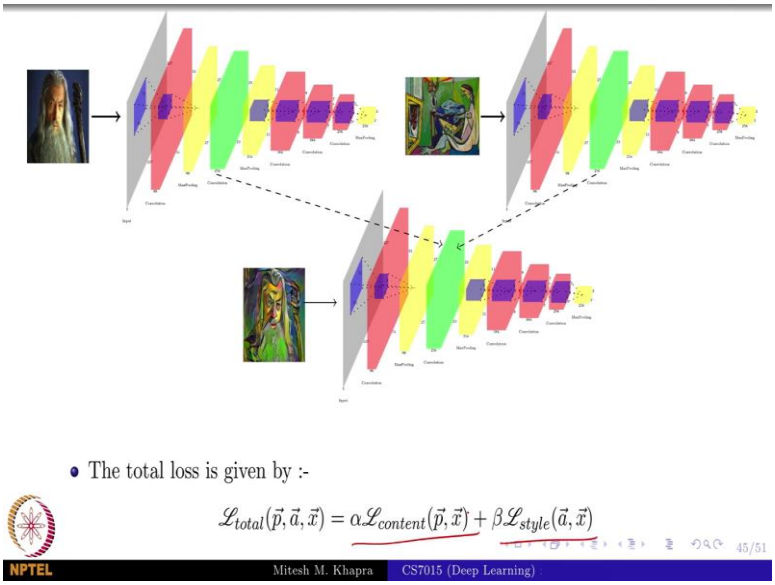
(Refer Slide Time: 03:52)



So, how would I do that? So, this is the content image, this is the actual oh sorry this is a style image correction ok. So, I would just want that this $V^T V$, which captures the style and I could do it for any one of the layers or all layers depending on what I want to do. I just want that this style should be as close to each other.

So, I can have a similar matrix squared error kind of a function right, so that is what this is trying to capture. These are the style gram, so this is $V^T V$ for the style image and this is $V^T V$ for the generated image, if I pass it through the convolutional neural network, I want both of these to match.

(Refer Slide Time: 04:37)



So, I want the content to match I want the style to match, so, then what is my total objective function going to be.

Student: sum of these.

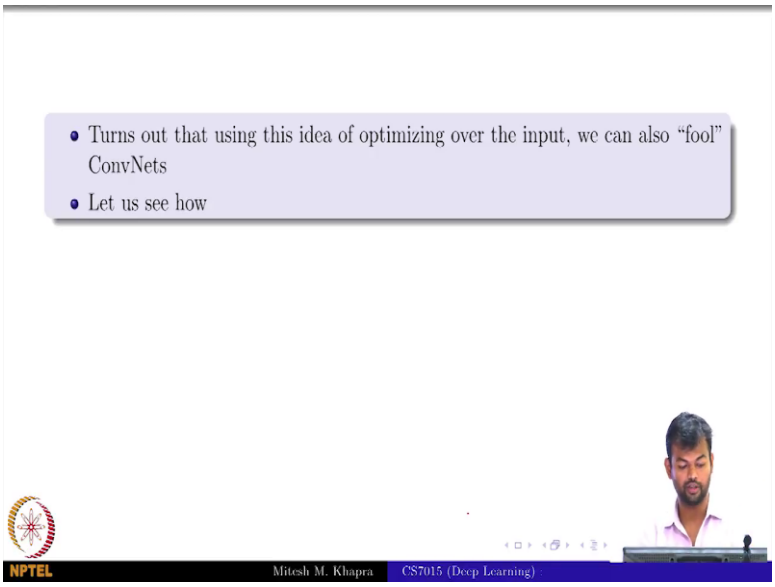
Sum of these right, so this is what my total objective function is going to be, I want the content to match and I also want the style to match. So, I will use an objective function which tries to balance between these 2 and alpha and beta are some hyper parameters ok. And if you do this and train the algorithm and try to modify the pixels along with some other bunch of tricks then, you will get this gandalf rendered in this style that you have given right. So, this is again some code is available for this, you can go and try it out. And it is interesting it is in a very interesting idea that you could have taken these two things and now you could be imaginative right, you could do all sorts of things with if you have two different images, how do you want to combine them and so on right. So, that is the basic key idea here.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture-102
Fooling Deep Convolution Neural Networks

With that we go on to the last module which is Fooling Deep Convolutional Neural Networks.

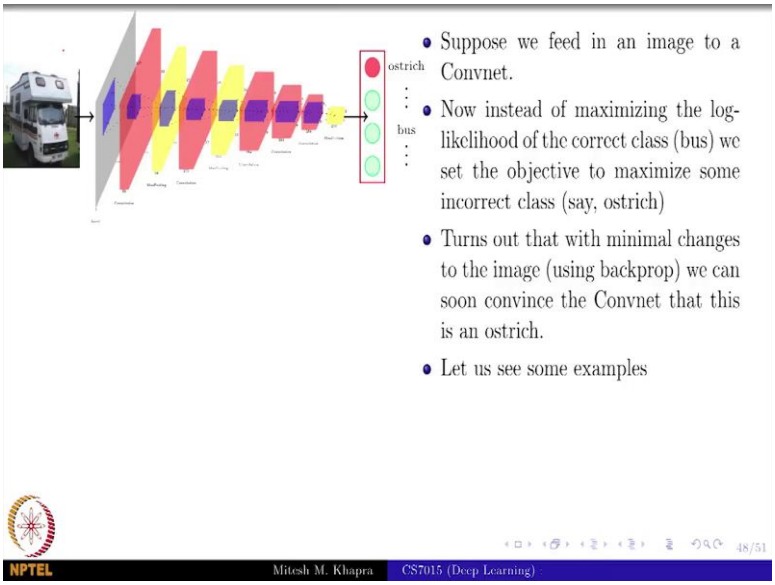
(Refer Slide Time: 00:19)



So, turns out that using this idea of optimization where, we are able to actually change the image to suit our needs right. And these needs were, one was we wanted to change the image so that, it fires for a particular class. The other was deep dream, where we wanted to change the image so that, it is starts seeing patterns which were otherwise not observed in the image. And the other was d part, where we trained the image or we optimized over the image so that, we could produce some artistic images and these are the different optimization problems that we have seen.

But the same idea can actually also be used to full convolutional neural networks and I have already hinted at this earlier. So, let us see how to do that.

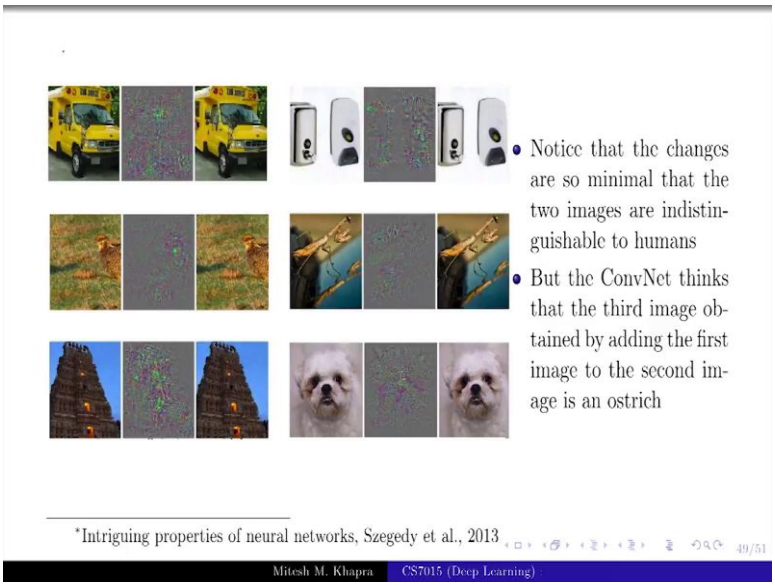
(Refer Slide Time: 00:55)



So, now suppose, we feed an image to a convnet and I know this is the bus image right, but now what I do is, this is a trained convolutional neural network and what I do is instead of setting the cross entropy loss to maximize bus, I will set up the cross entropy loss to maximize Ostrich. And then I will back propagate through the network, I will not modify any of these weights or parameters and I am only change the image right.

So, what I am trying to do is, I know that this is the bus image, but now I am setting the objective that, it should fire for the Ostrich class. So, now, I am going to back propagate and change this image so that, the blog the likelihood of the Ostrich class increases. You get this set up? Its very straight forward, ok. And turns out that if you do this with very minimal changes to the image, you can actually fool the convolutional neural network ok.

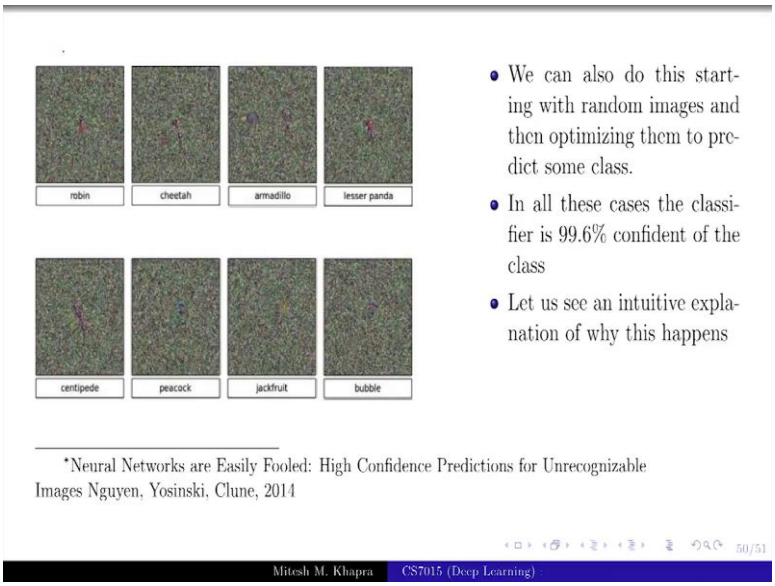
(Refer Slide Time: 01:50)



So, this is the change right, you have the original image, the second image is actually the amount of change you made and the third image is the original image plus this change.

Now, to the human eye there is no distinction here right, you would all of first would still think this is a bus and in fact, I do not even see that, there is a noise in the third bus that you see. Same for some other class they have taken some bird or something like that and added some noise to it and a temple. And in all of these cases, the network actually predicts that the modified image is an Ostrich right or some very random class from the original class. So, why is this happening and before asking that question let me just finish and it need not be that you start with an original image and then try to modify it.

(Refer Slide Time: 02:34)



Actually, you can start with a blank image and do the same experiment where you modify the image minimally so that, p of robin becomes 1 or close to 1. And you will get some very arbitrary noisy looking images, which no in; which to at least you and me do not look like a cheetah or robin or armadillo, but the network thinks that these are the classes that these images belong to.

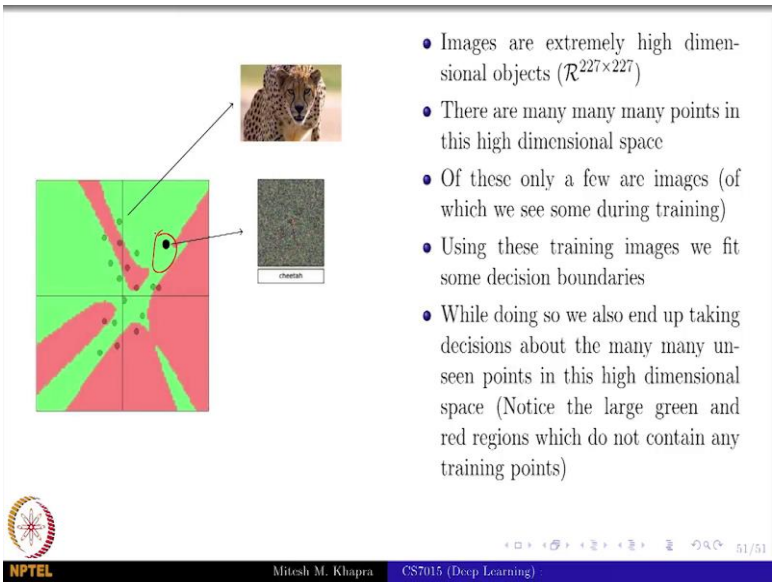
Now, this is definitely a risky? How many of you appreciate that? It is bad ok. Now and a network is not just predicting it, is predicting it with a very high confidence right 99.6 percent confidence. So, why is this happening? Can even think of a reason for that?

Student: (Refer Slide Time: 03:12).

No, but ok, in that case I would have been fine if there are 1000 classes it should have given 1/1000 probability to all the classes right, but this is like worse than random classifier right. It is saying with 99 percent confidence that, this is a Ostrich or whatever class that is So, why is this happening? And the interesting thing is that, this in some sense ties back to the universal approximation theory or at least some ideas with that. Can you think of why this is happening ok.

So, let us try to see a very intuitive explanation for this so on.

(Refer Slide Time: 03:45)



So, this explanation is due to Andree Karpathi, we need to put the acknowledgments; this slide does not have any acknowledgments actually. So, remember that images are extremely high dimensional objects right, they are 227×227 which is a very high dimensional object, high dimensional space. And no matter how much training data you have, you see a only a small sample of this high dimensional space right because, its real numbers 227×227, just imagine the number of possibilities out there, no matter you have 1 million samples 10 million samples for training, this is much smaller than the actual number of samples which exist in this space. Of these only a few are images right.

So, now, think of all 227 $\times$ 227 matrices that you can make and how many of them are actually going to be natural images. The probability of natural images is very small, most of these are random things right, they are just matrices which do not make any sense which actually look like these images that you see here right.

Now, using the training images, we fit some decision boundaries and this is the decision boundaries that we fit right that, this is class 1, the rest of the green part is class 2 and so on. And in fact, we are doing these decision boundaries for some 1000 classes While doing so, we actually end up taking decisions for a large number of points that we have not seen. We have not seen any points in this space, but I have made a decision for them, that all of them belong to the green class. I have not seen any point in this space, but I have ended up taking a decision for them that all of them belong to the red class right.

So, in particular what I have done is, I saw a cheetah class, image from a cheetah class. I saw a few images from the cheetah class and I drew some boundary around it to say that this is the cheetah class, but my boundary also contains images like this. Because, this is a very high dimensional space and in that boundary a lot of points actually fall in and some of these points are these random points, which have no relation to cheetah right. But I have been so aggressive in fitting to the training data, that I have drawn these boundaries which also include a lot of these points and now, all I need to do starting with these rend random images is that, go somewhere inside this boundary and then I am all set right it will start detecting it as cheetah because, the boundaries have been drawn by the classifier. How many if you get this explanation, good. So, that is the intuitive explanation for why this happens. So, this is where we will end the discussion on convolutional neural networks.